SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : T. SHARADA

Reg. No : 11249A345

Class: III B.E. (CSE)

Course Code:

Course Name: NATURAL LANGUAGE PROCESSING LAB RECORD

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the Bonafide record of work done by Mr/Ms. T. Sharada with Reg. No :11249A345 of III-B.E.(CSE) during the academic year 2025 – 2026.

Station : ENATHUR

Date:

Staff-in-charge

Head of the Department

Submitted for the Practical examination held on:

Examiner-1 **Examiner-2**

INDEX

SL.NO	Program Name	Page no	Date	Sign
1	WORD ANALYSIS		24/07/2026	
2	WORD GENERATION		05/08/2026	
3	MORPHOLOGY		12/08/2026	
4	N-GRAMS		12/08/2026	
5	N-GRAMS SMOOTHING		19/8/2026	
6	POS TAGGING:HIDDEN MARKOV MODEL		09/09/2026	
7	POS TAGGING:VITERBI DECODING		09/09/2026	
8	BUILDING POS TAGGER		14/10/2026	
9	CHUNKING		14/10/2026	
10	BUILDING CHUNKER		14/10/2026	

Name: T. Sharada

Reg No:11249A345

Natural Language Processing Lab Activity

Experiment1activity1

exp1act1.py exp1act2.py exp1act4.py exp1act5.py exp1act3.py

```
print("John eats John")
print("John eats Mary")
print("Mary eats John")
print("Mary eats Mary")
```

Run exp1act2

```
C:\Users\Administrator\PycharmProjects\NLP\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\NLP\exp1act2.py
John eats John
John eats Mary
Mary eats John
Mary eats Mary
```

exp1act1.py exp1act2.py exp1act4.py exp1act5.py exp1act3.py

```
subjects = ["John", "Mary"]
objects = ["John", "Mary"]

for s in subjects:
    for o in objects:
        print(s, "eats", o)
```

Run exp1act2

```
C:\Users\Administrator\PycharmProjects\NLP\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\NLP\exp1act2.py
John eats John
John eats Mary
Mary eats John
Mary eats Mary
```

activity3

exp1act1.py exp1act2.py exp1act4.py exp1act5.py exp1act3.py

```python
subjects = ["John", "Mary", "Student"]
objects = ["John", "Mary", "Student"]

print("Sentences using 'eats':")
for s in subjects:
    for o in objects:
        print(s, "eats", o)

print("\nSentences using 'studies NLP':")
for s in subjects:
    print(s, "studies NLP")
```

Run exp1act3

```
John eats Mary
John eats Student
Mary eats John
Mary eats Mary
Mary eats Student
Student eats John
Student eats Mary
Student eats Student

Sentences using 'studies NLP':
John studies NLP
```

Activity4

exp1act1.py exp1act2.py exp1act4.py exp1act5.py exp1act3.py

```python
systems = {
    "College": [
        "Student studies AI",
        "Teacher teaches Python"
    ],
    "Hospital": [
        "Doctor treats Patient",
        "Nurse checks Patient"
    ],
    "Library": [
        "Student borrows Book",
        "Librarian issues Book"
    ],
    "Banking": [
        "Customer deposits Money",
        "Customer withdraws Money"
    ]
}

for system, sentences in systems.items():
    print("\n" + system)
    for sentence in sentences:
        print(sentence)
```

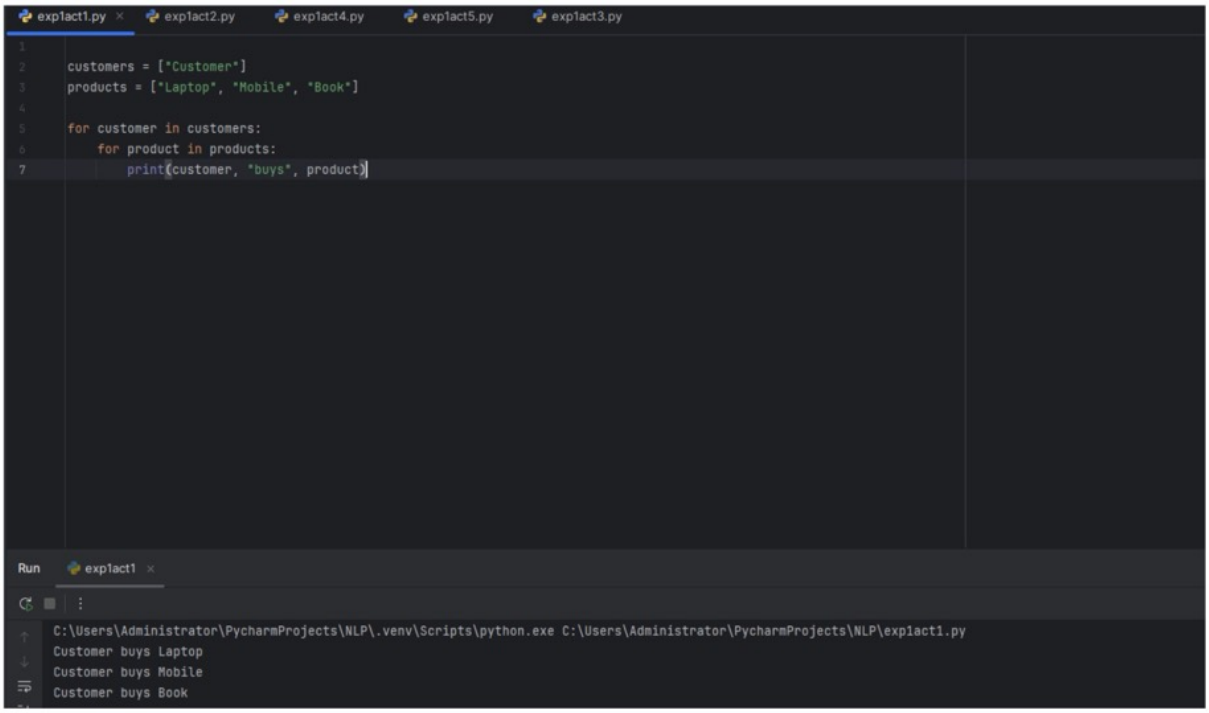
Run exp1act4

```

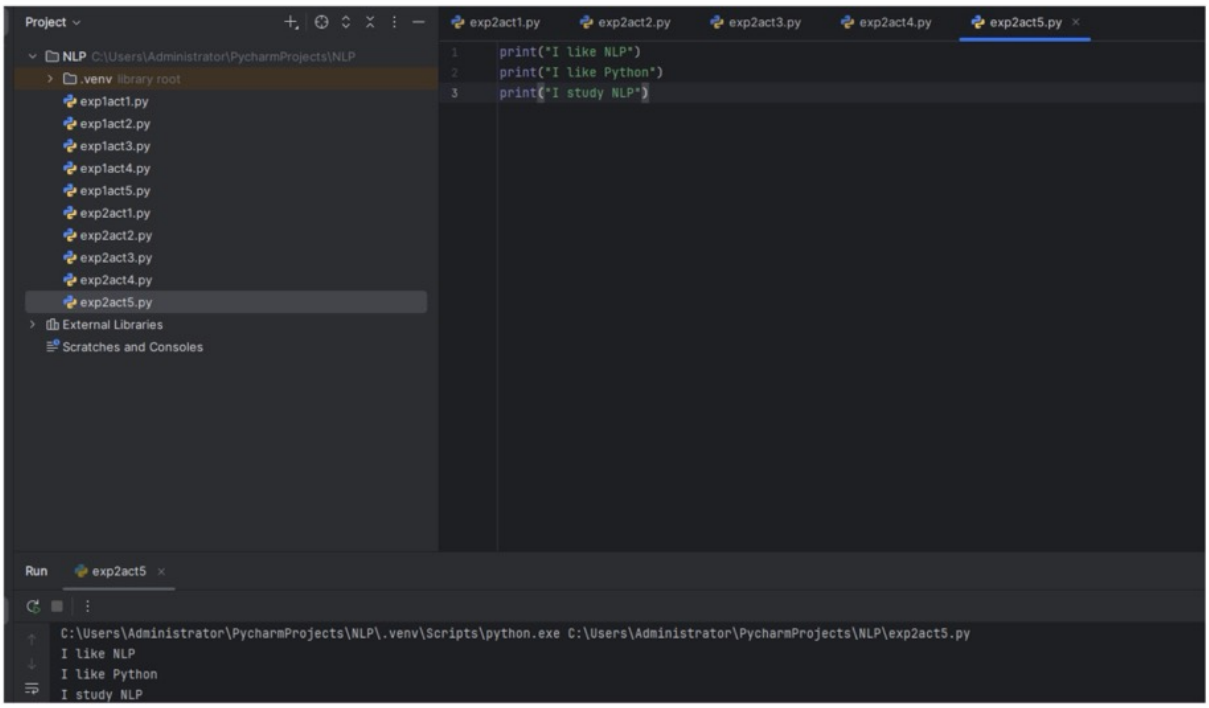
Library
Student borrows Book
Librarian issues Book

Banking
Customer deposits Money
Customer withdraws Money
```

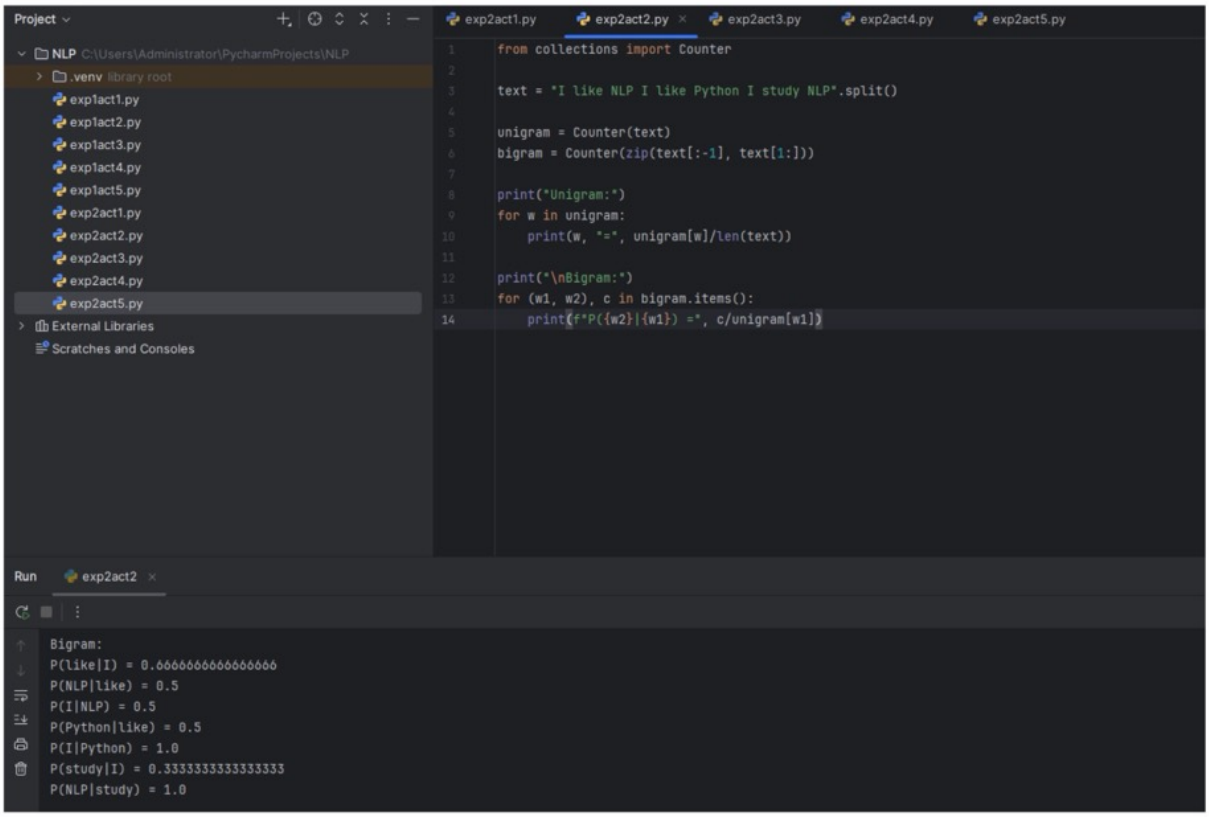
activity5



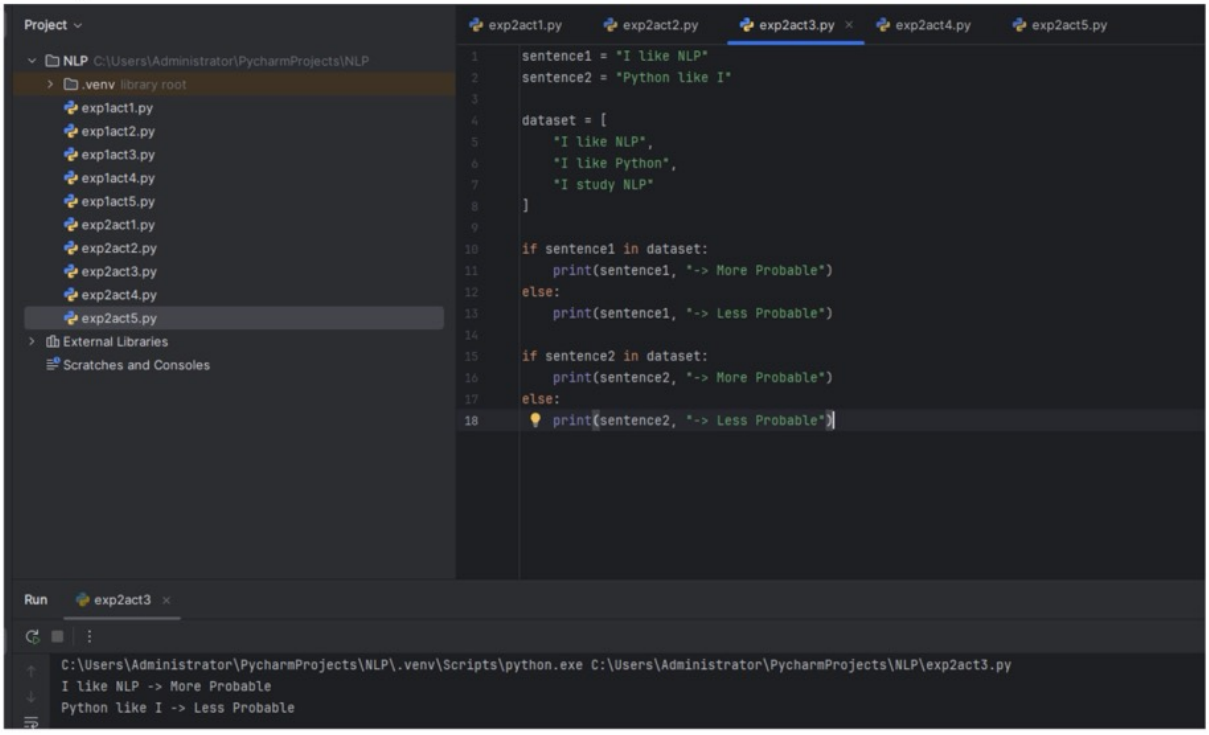
Experiment2 activity1



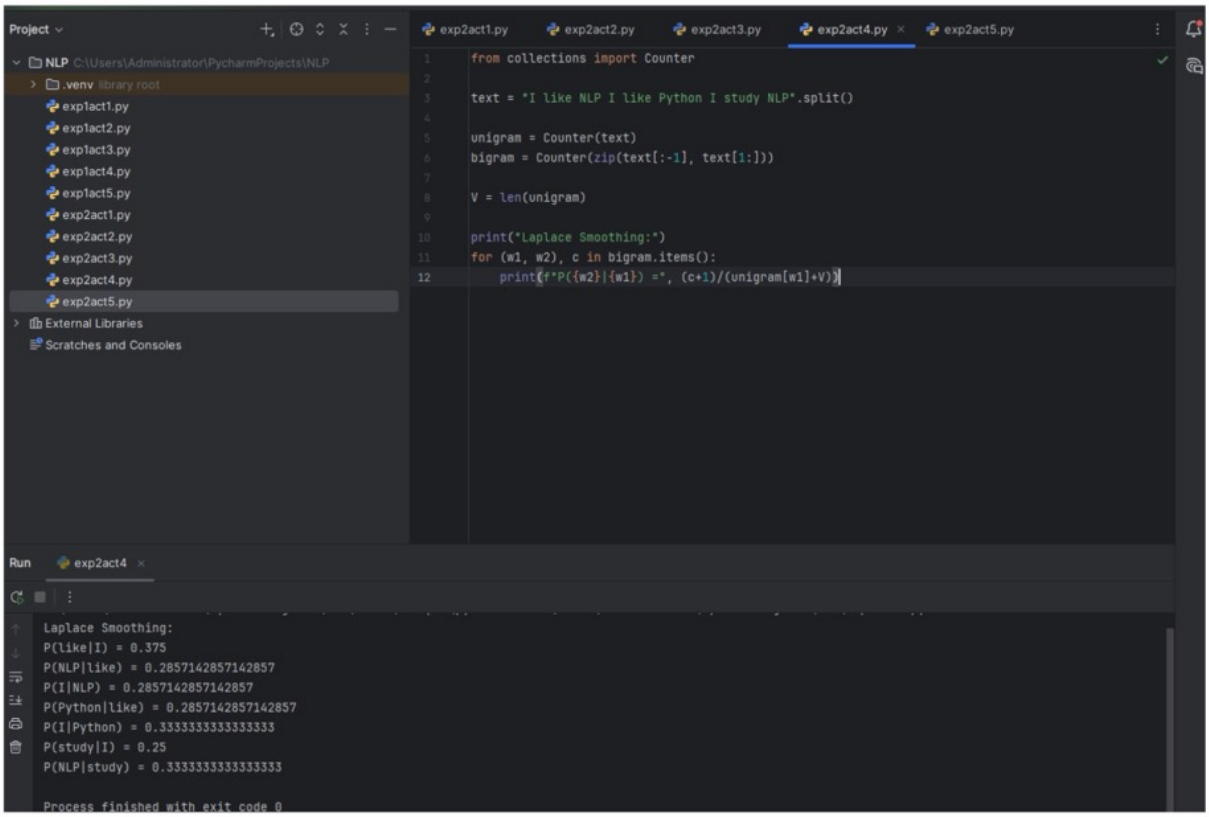
Activity2



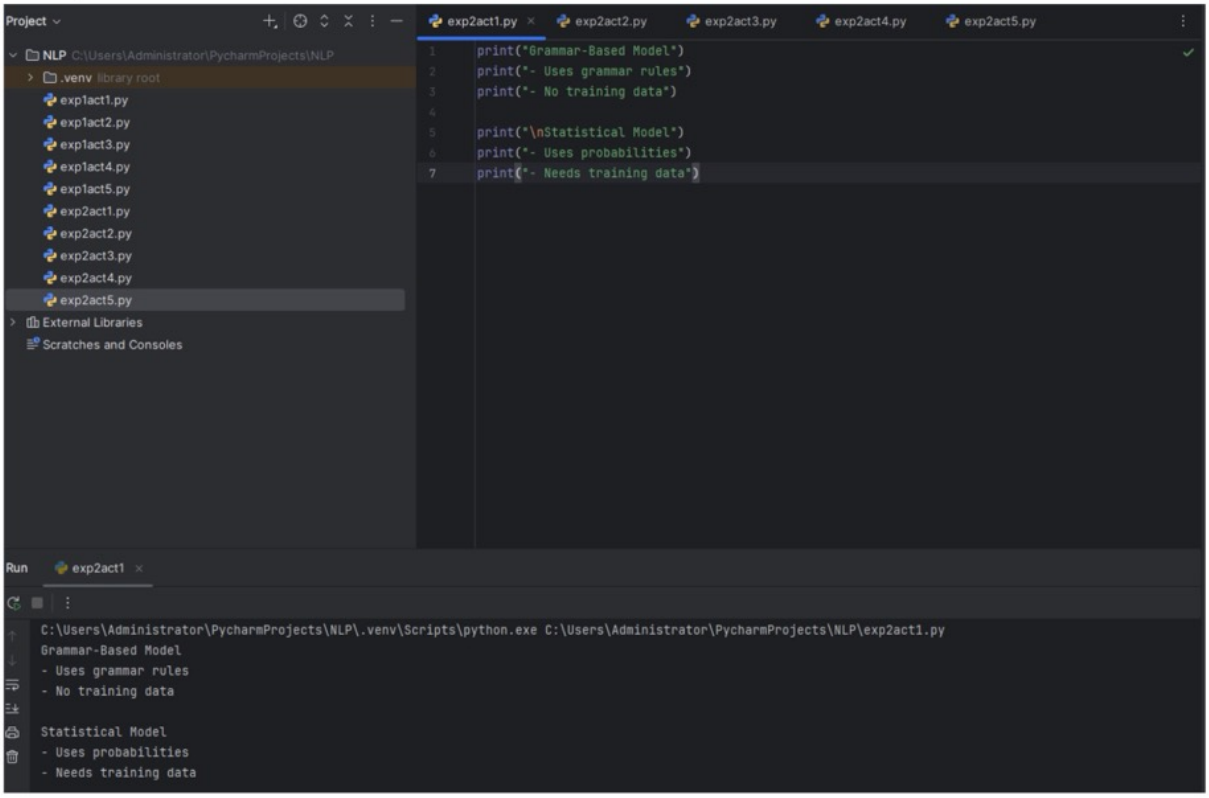
Activity3



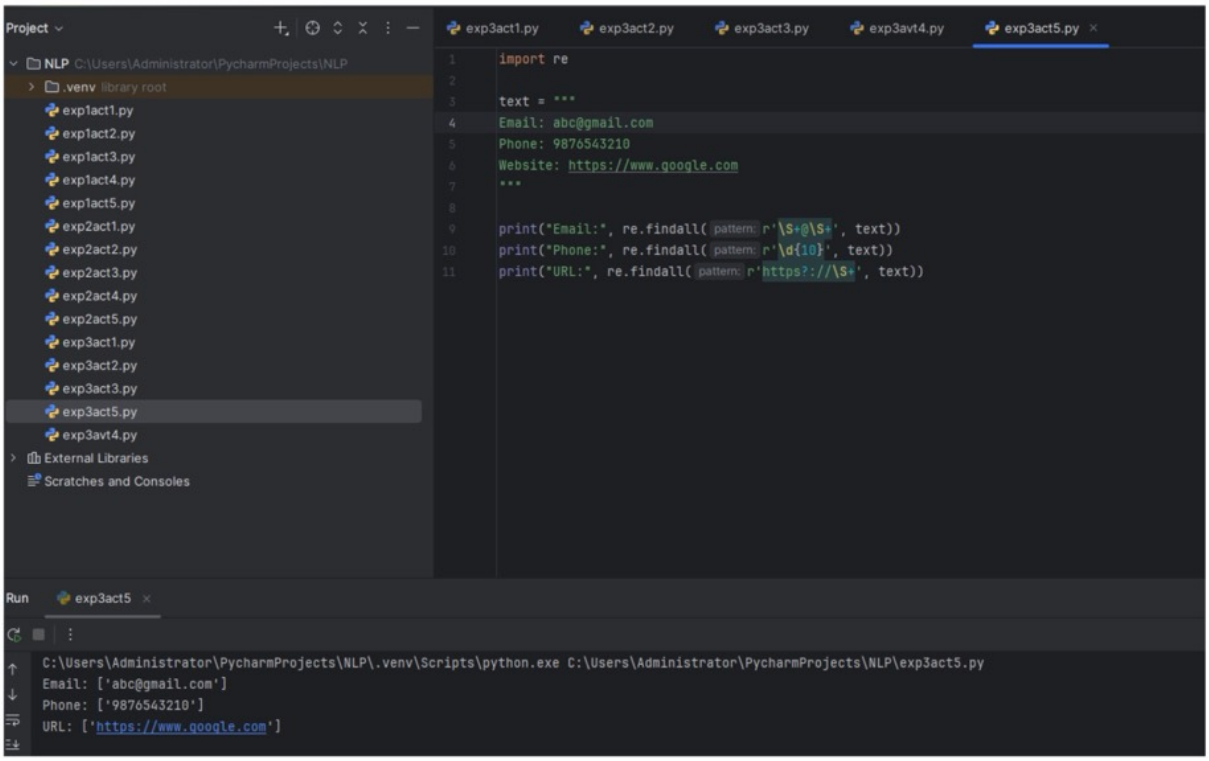
Activity4



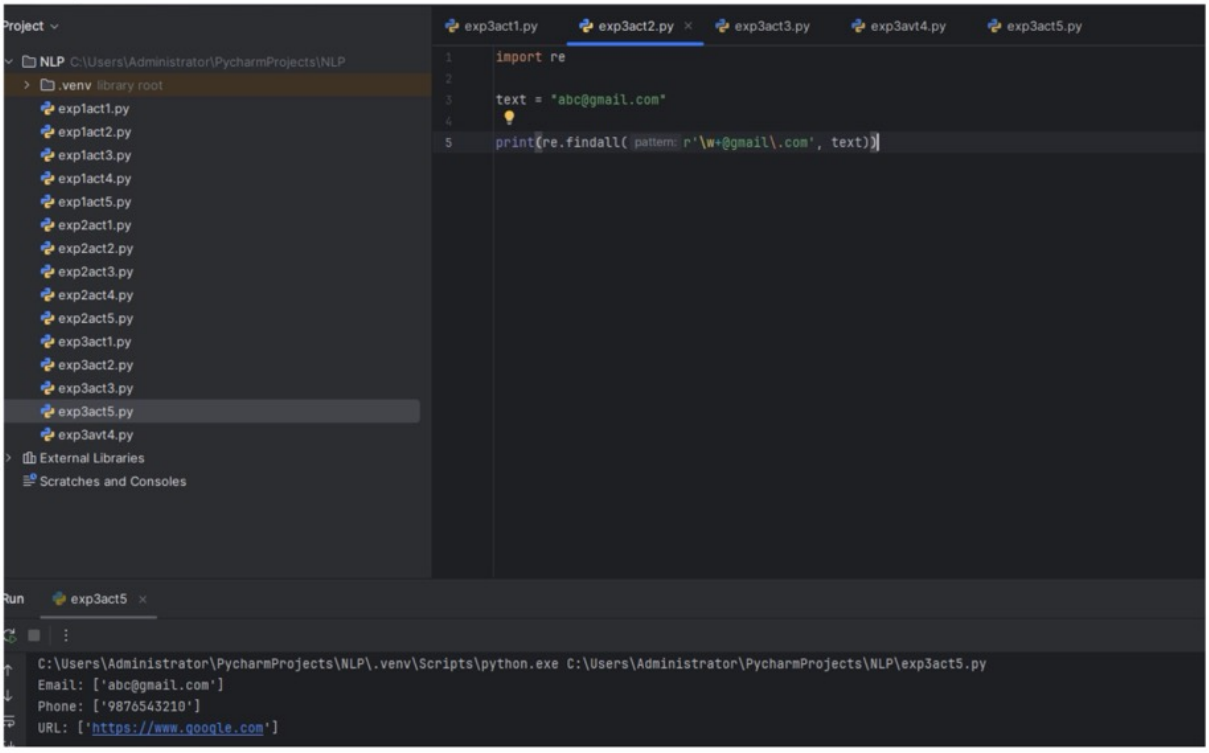
activity5



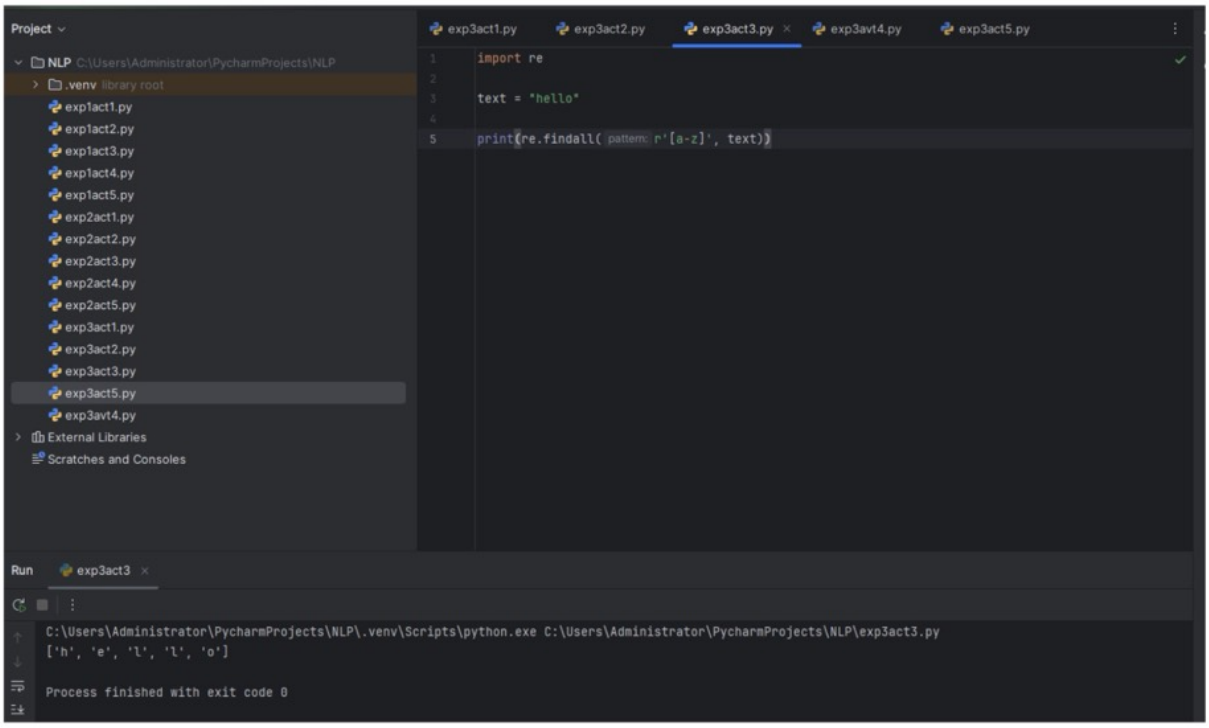
Experiment3activity1



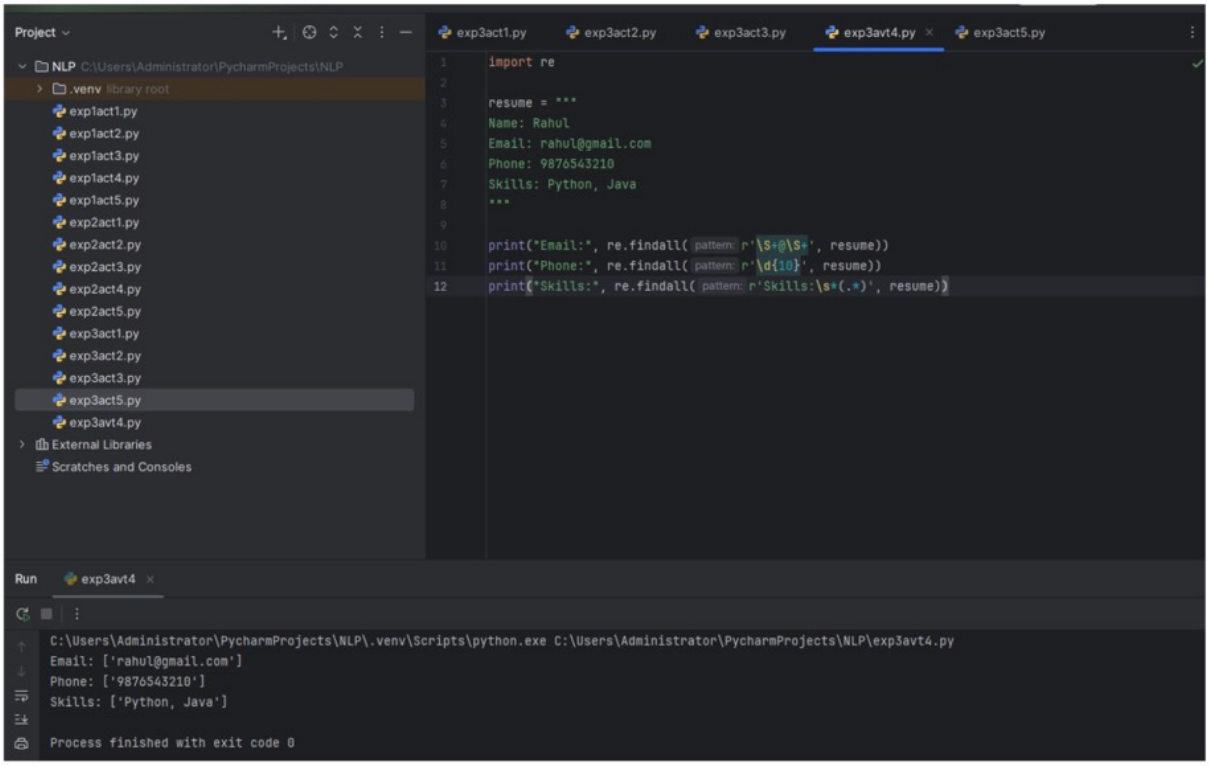
Activity2



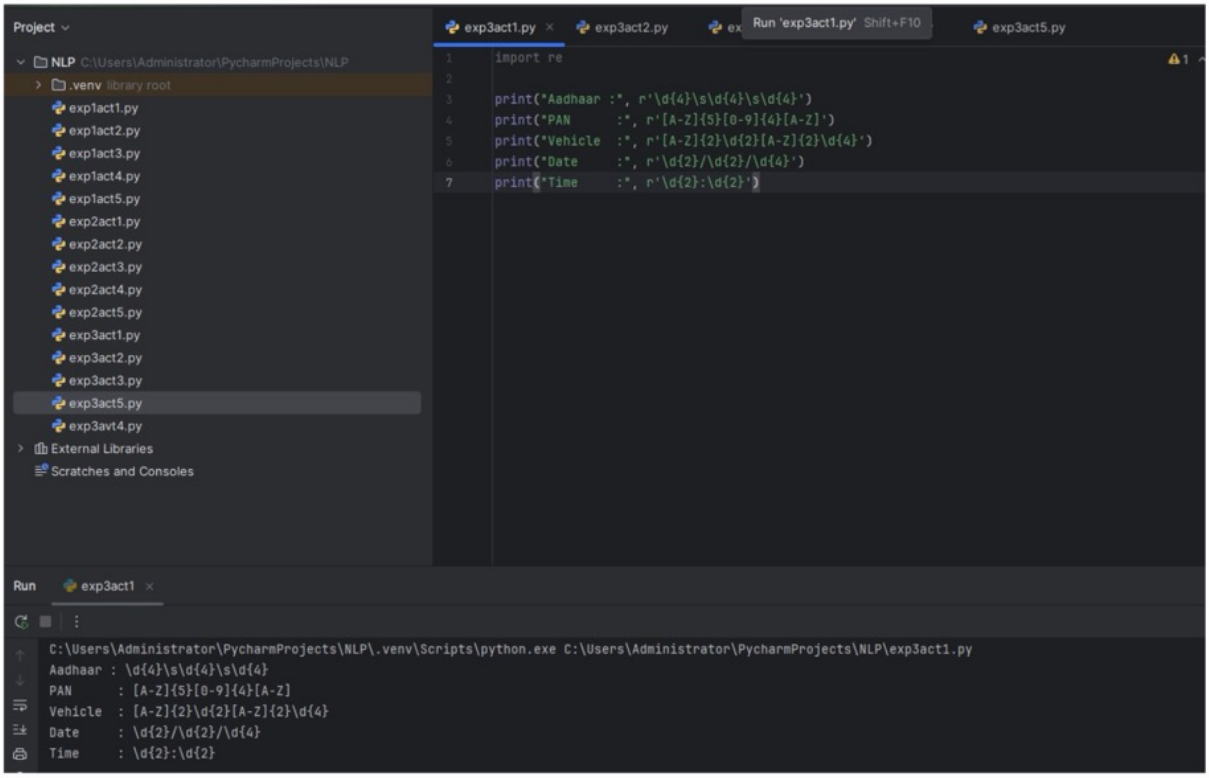
Activity3



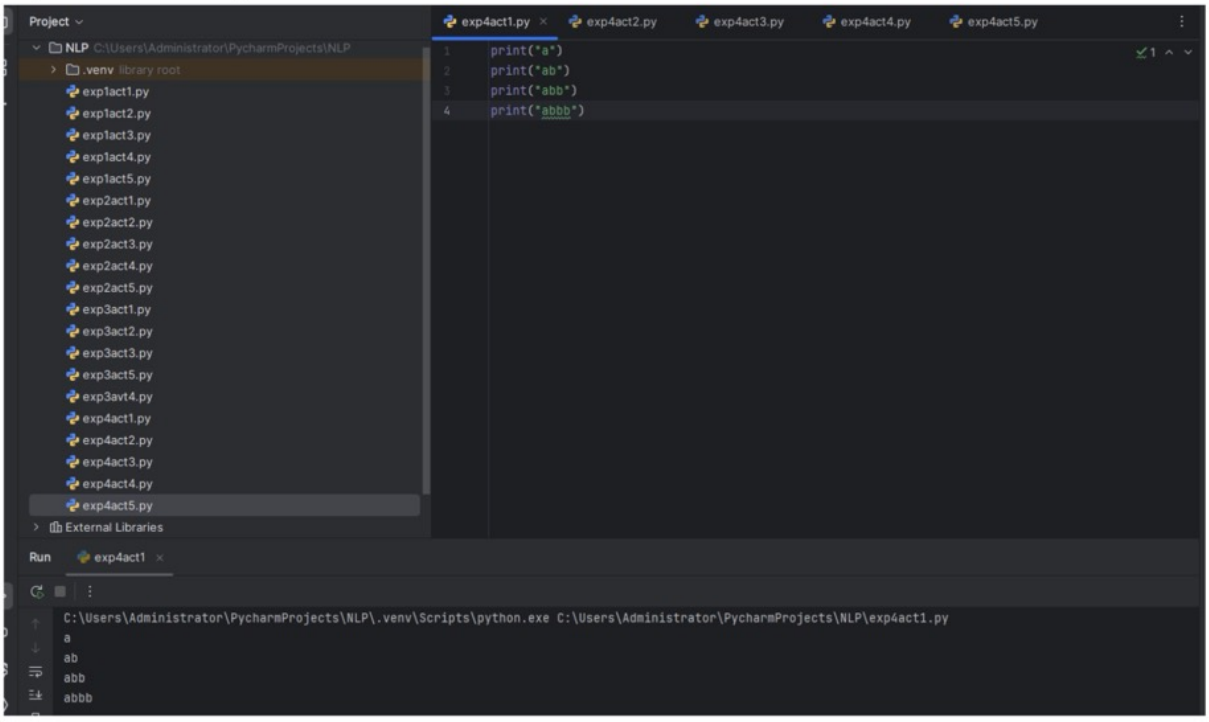
Activity4



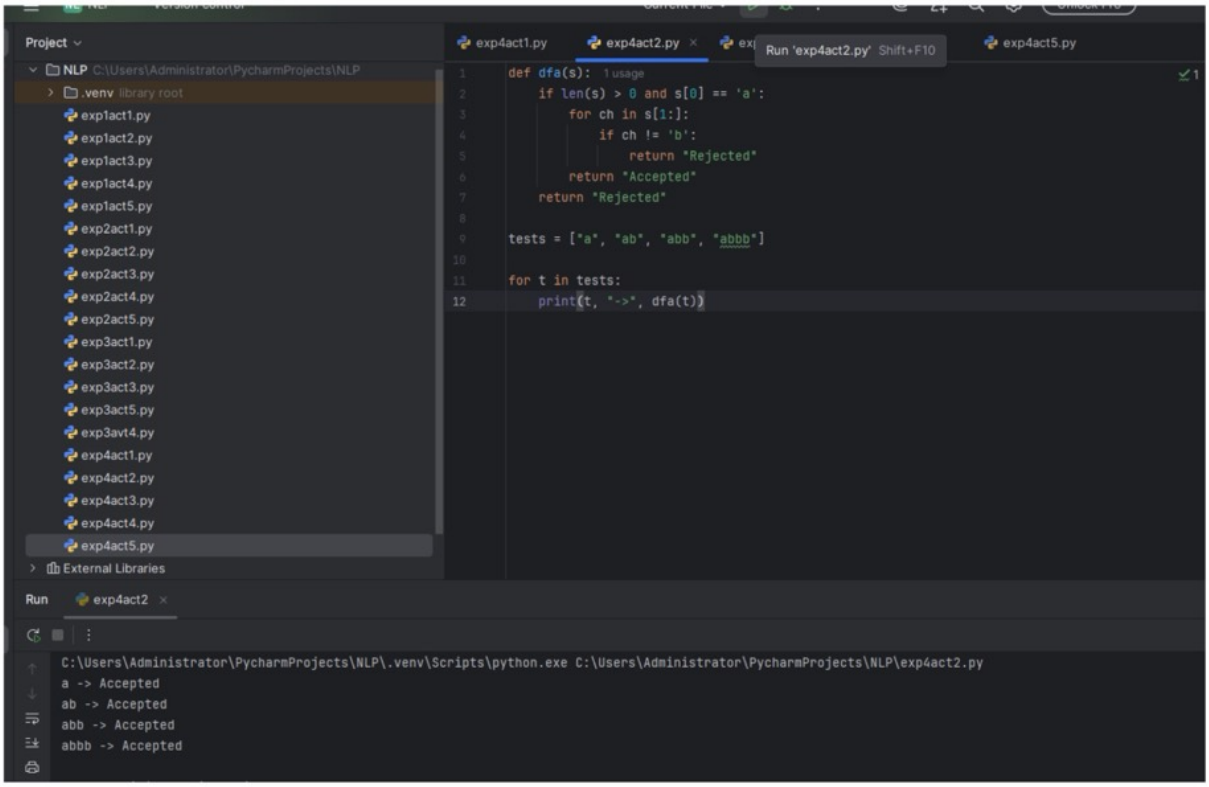
Activity5



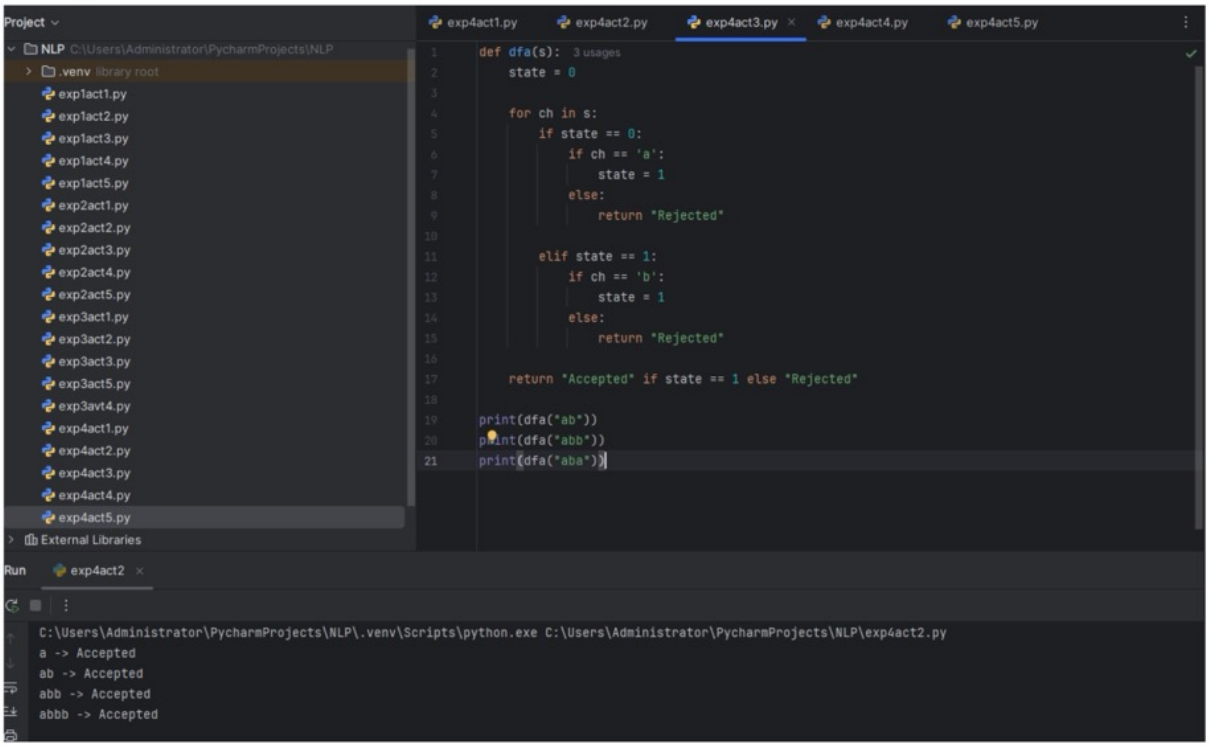
Experiment4activity1



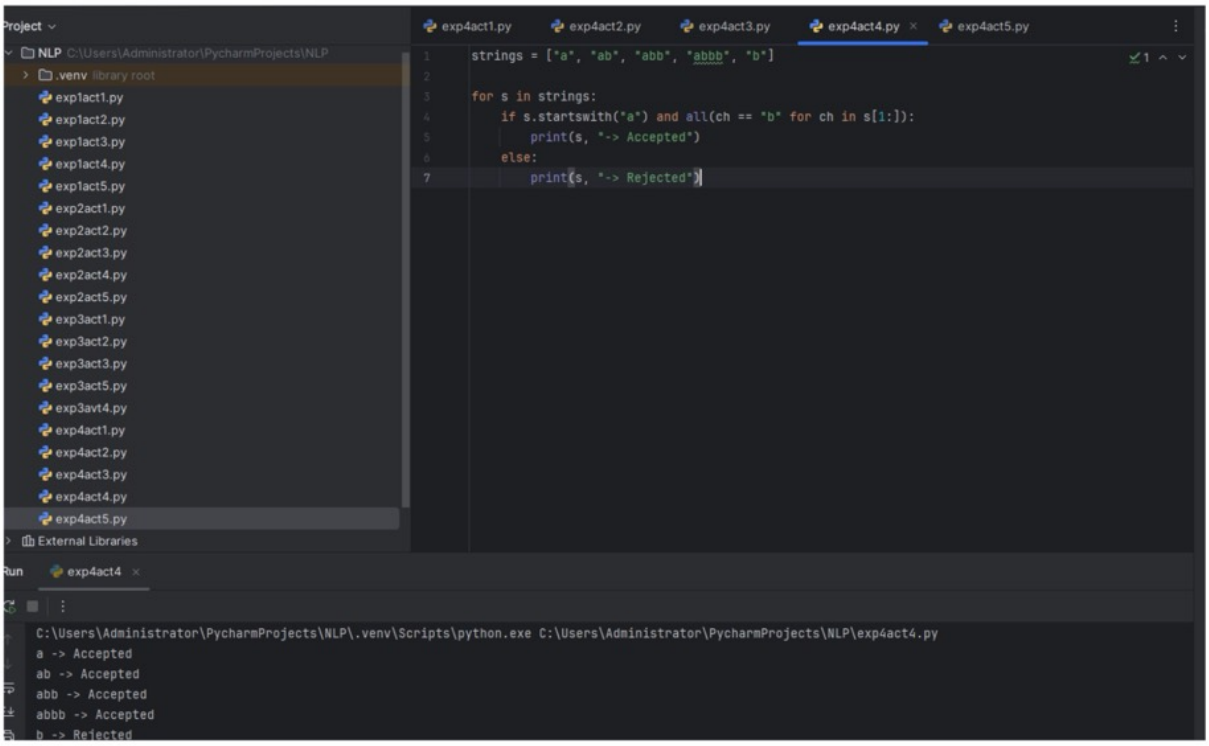
Activity2



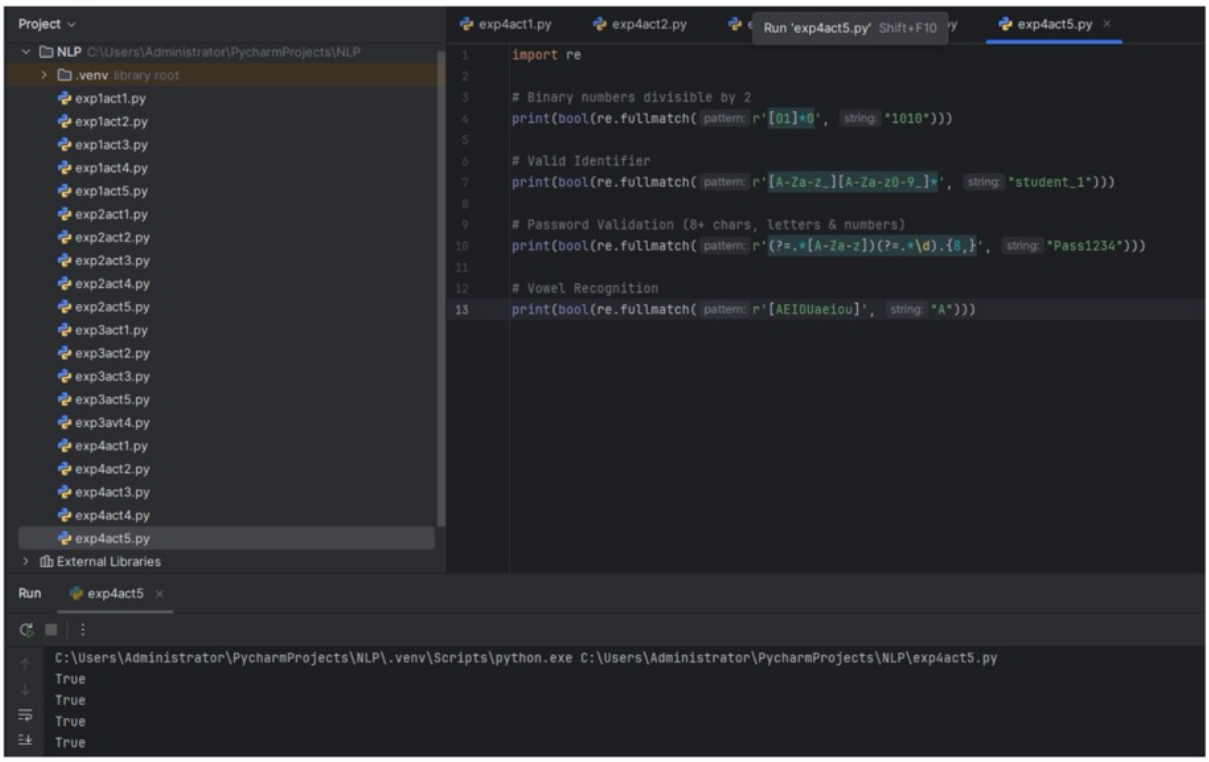
Activity3



Activity4



Activity5



Date:05.08.26
Reg no:11249A345
Name:T.Sharada

Task 1 — Word Analysis using Stemming

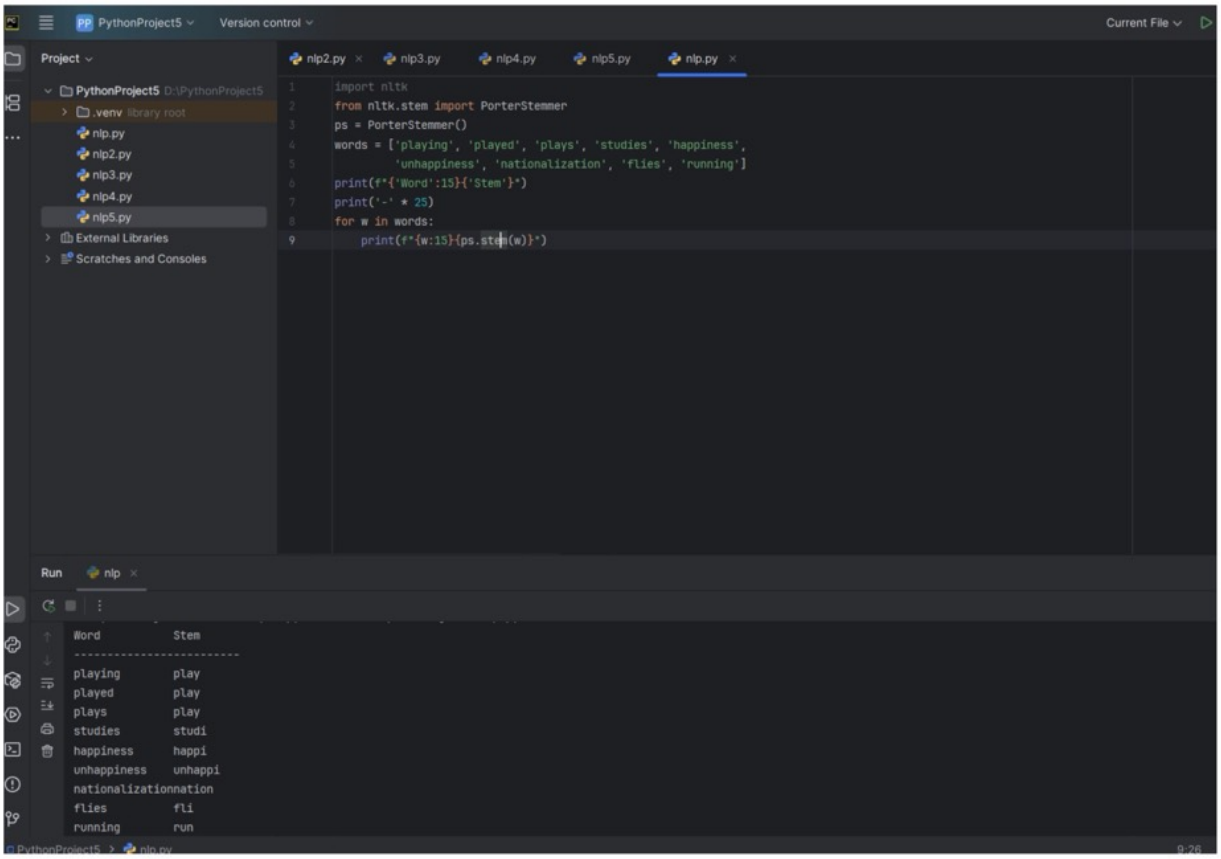
Aim: To write a Python program that takes a list of words as input and prints their stems using the **Porter Stemmer** from the **NLTK** library.

Algorithm :

1. Start the program.
2. Import the `PorterStemmer` class from the `nltk.stem` module.
3. Create an object of the `PorterStemmer` class.
4. Define a list of input words.
5. Print the table headings **Word** and **Stem**.
6. Traverse each word in the list using a `for` loop.
7. Apply the `stem()` method to obtain the stem of the word.

8. Print the original word and its corresponding stem.
9. Stop the program.

Program:



Result:

The Python program was executed successfully. The Porter Stemmer correctly generated the stems for the given input words.

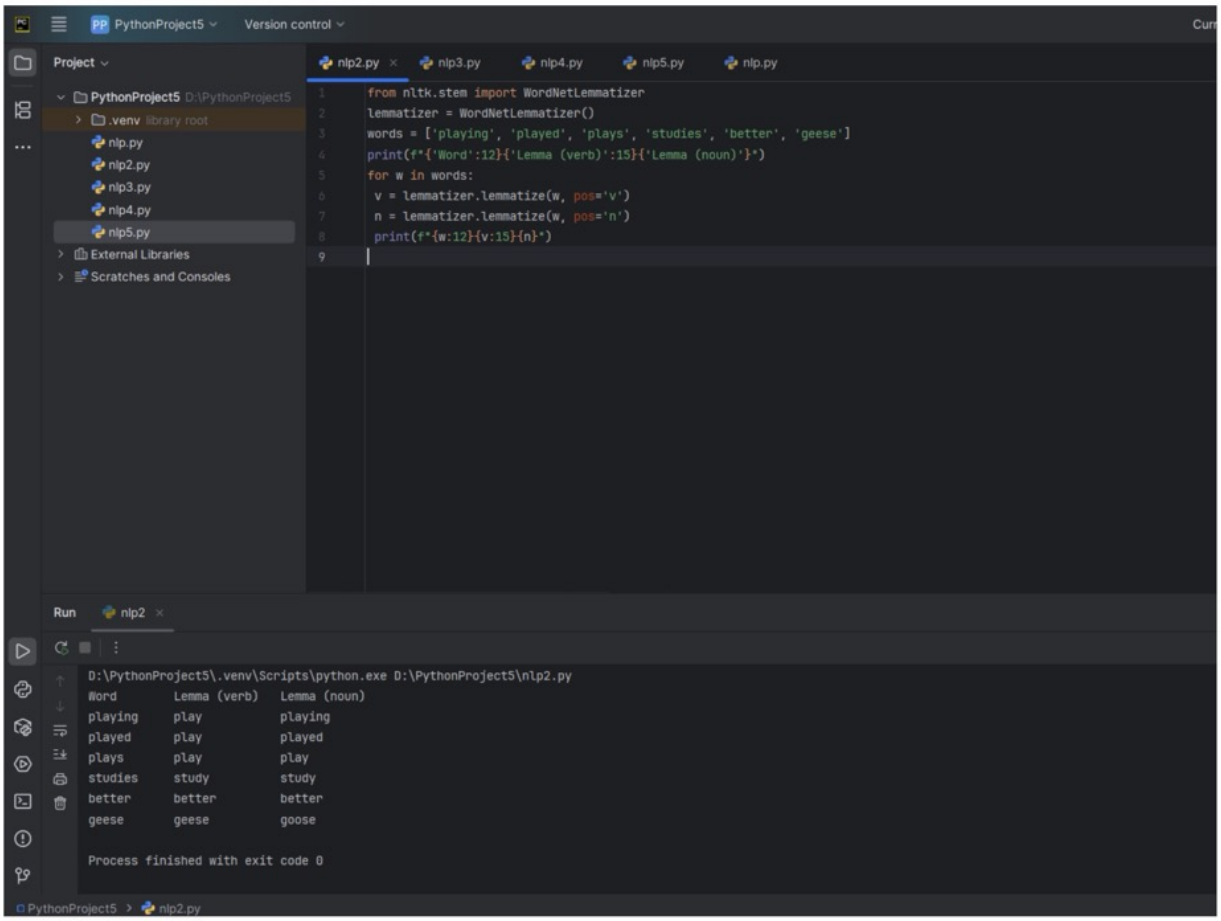
Task 2 — Word Analysis using Lemmatization

Aim: To write a Python program that uses the WordNet Lemmatizer from the NLTK library to find the correct dictionary root (lemma) of given words and compare the results with stemming.

Algorithm:

1. Start the program.
2. Import the `WordNetLemmatizer` class from `nltk.stem`.
3. Create an object of the `WordNetLemmatizer` class.
4. Define a list of words for analysis.
5. Display the column headings: **Word**, **Lemma (verb)**, and **Lemma (noun)**.
6. Traverse each word in the list using a `for` loop.
7. Apply the `lemmatize()` method with `pos='v'` to find the verb lemma.
8. Apply the `lemmatize()` method with `pos='n'` to find the noun lemma.
9. Print the original word along with its verb and noun lemmas.
10. Stop the program.

Program:



Result: The program was executed successfully and the dictionary root forms (lemmas) of the given words were obtained using the WordNet Lemmatizer.

Task 3 — Rule-Based Morphological Analyzer

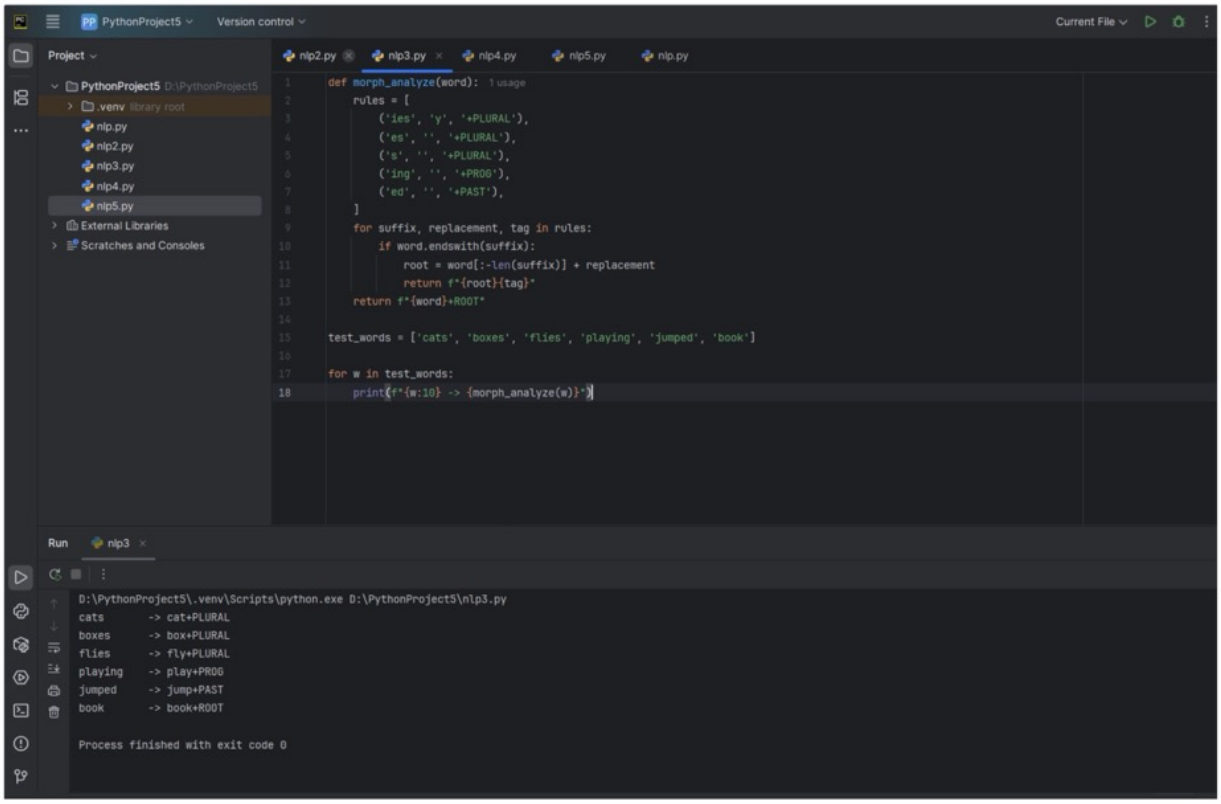
Aim: To design and implement a simple rule-based morphological analyzer that identifies the root word and grammatical feature tag (+PLURAL, +PROG, +PAST) of an inflected English word using suffix-stripping rules.

Algorithm:

1. Start the program.
2. Define a function `morph_analyze(word)`.
3. Create a list of suffix rules containing:
    - Suffix
    - Replacement string
    - Grammatical feature tag
4. Check each suffix rule one by one.
5. If the word ends with a given suffix:
    - Remove the suffix.
    - Add the replacement string.
    - Attach the corresponding grammatical tag.
6. Return the root word with its feature tag.
7. If no suffix matches, return the word with the tag `+ROOT`.
8. Apply the function to the given test words.

9. Display each word with its analyzed root and grammatical feature.
10. Stop the program.

Program:



Result: The program was executed successfully and analyzed the given words by separating the root and grammatical features.

Task 4 — Rule-Based Word Generator

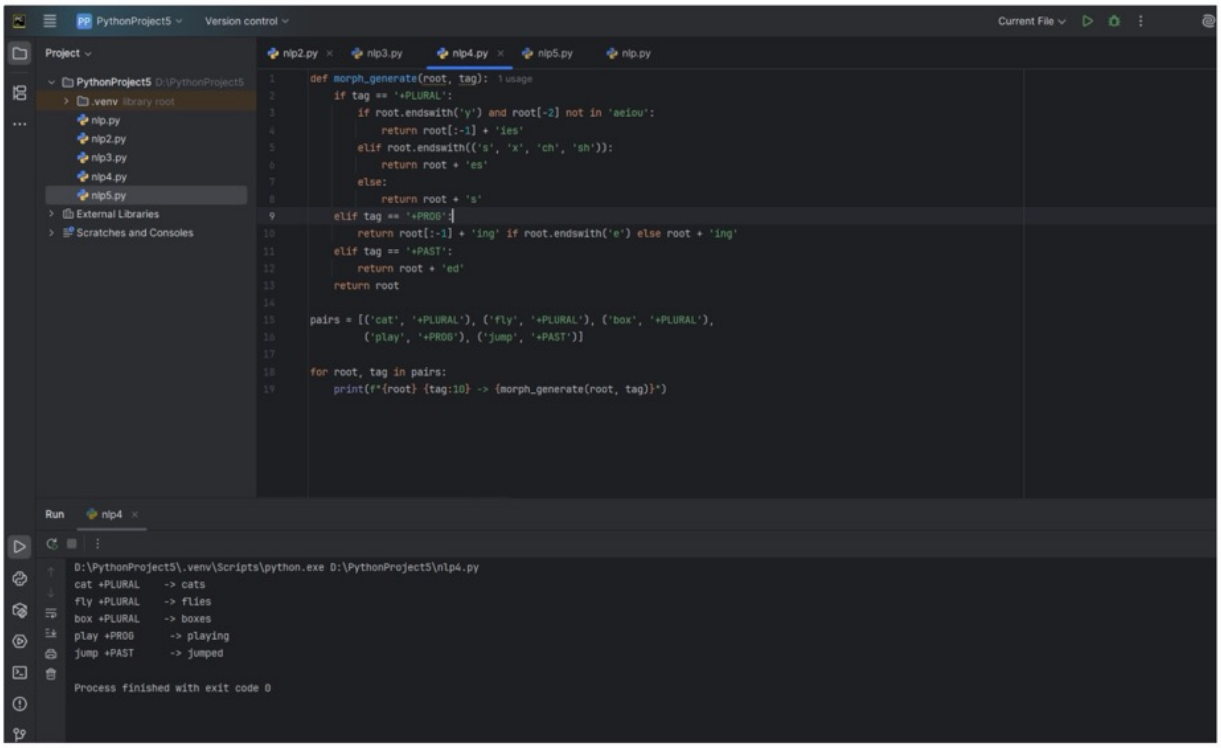
Aim: To design and implement a simple rule-based morphological word generator that generates the correct surface form of a word from a given root word and grammatical feature tag (+PLURAL, +PROG, +PAST).

Algorithm:

1. Start the program.
2. Define a function `morph_generate(root, tag)`.
3. Check the grammatical feature tag:
    - If the tag is +PLURAL:
        - If the root ends with consonant + `y`, replace `y` with `ies`.
        - If the root ends with `s`, `x`, `ch`, or `sh`, add `es`.
        - Otherwise, add `s`.
    - If the tag is +PROG:
        - If the root ends with `e`, remove `e` and add `ing`.
        - Otherwise, add `ing`.
    - If the tag is +PAST, add `ed` to the root.

4. Return the generated word form.
5. Provide root words and their corresponding grammatical tags as input.
6. Call the function for each pair.
7. Display the generated surface words.
8. Stop the program.

Program:



Result: The program was executed successfully and generated the correct word forms from the given root words and grammatical tags.

Task 5 — Inflectional vs. Derivational Classification

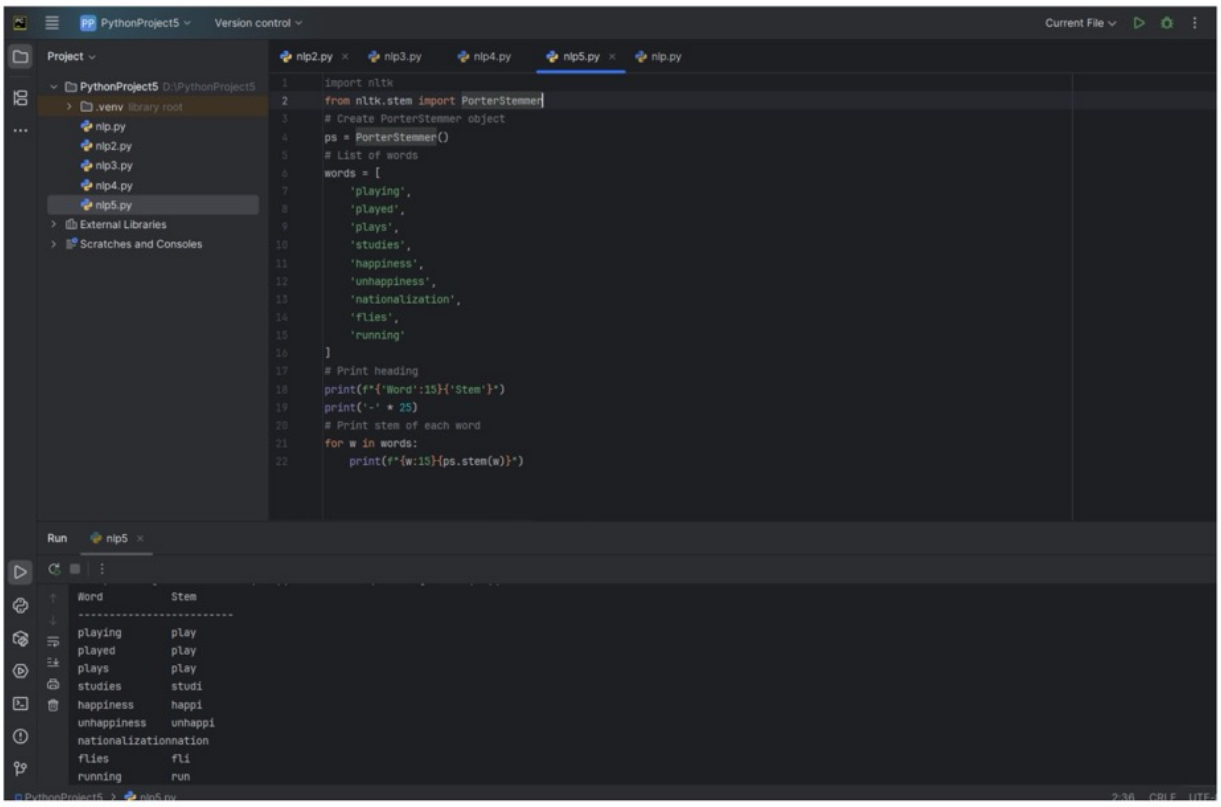
Aim: To write a program that classifies word pairs as inflectional or derivational morphology and identifies the affix involved in each word transformation.

Algorithm:

1. Start the program.
2. Create a list of word pairs containing:
    - Base word
    - Derived/Inflected word
    - Affix
    - Morphological type
3. Compare each base word with its transformed word.
4. Identify the affix added or changed during word formation.
5. Classify the word pair as:
    - Inflectional: changes grammatical features without changing the word class.
    - Derivational: creates a new word or changes the meaning/word class.
6. Display the base word, derived/inflected word, affix, and classification

7. Stop the program.

Program:



Result:

The program was executed successfully and classified the given word pairs into inflectional and derivational morphology.

. Exercises :

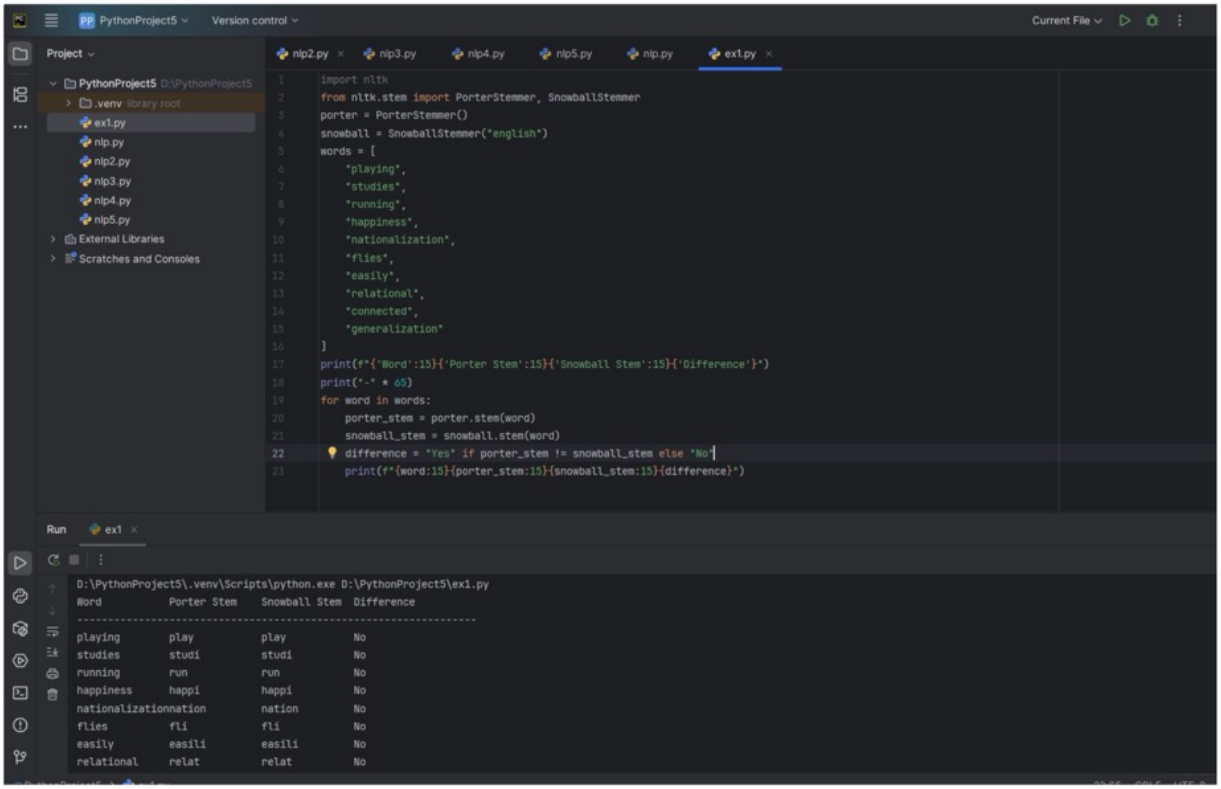
1.Compare the outputs of the Porter Stemmer and the Snowball Stemmer for a set of 10 English words. Report the differences.

Aim: To compare the outputs of the Porter Stemmer and Snowball Stemmer for a set of English words and identify the differences between the generated stems.

Algorithm:

1. Start the program.
2. Import `PorterStemmer` and `SnowballStemmer` from the NLTK library.
3. Create objects for both stemmers.
4. Define a list of 10 English words.
5. Apply the Porter Stemmer to each word and store the results.
6. Apply the Snowball Stemmer to each word and store the results.
7. Compare the stems generated by both stemmers.
8. Display the original word, Porter stem, Snowball stem, and whether they are different or same.
9. Stop the program.

Program:



Result:

The program was executed successfully and the stems generated by Porter Stemmer and Snowball Stemmer were compared.

2.Extend the rule-based morphological analyzer (Task 3) to correctly handle irregular plurals (e.g., "mice", "geese", "children") and irregular past tense forms (e.g., "went", "ate", "ran").

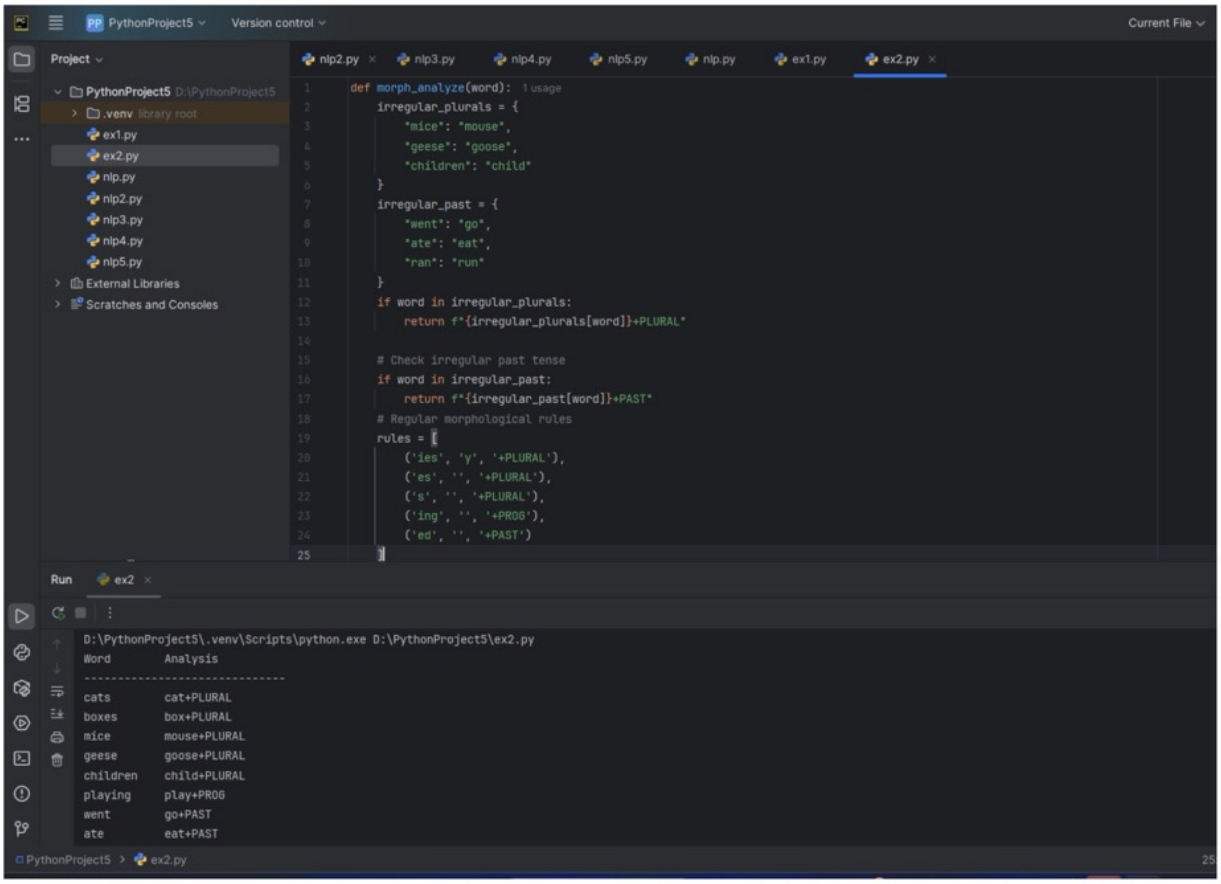
Aim: To extend the rule-based morphological analyzer to handle irregular plural forms (such as *mice, geese, children*) and irregular past tense forms (such as *went, ate, ran*) along with regular suffix-based morphological analysis.

Algorithm:

1. Start the program.
2. Define dictionaries for irregular forms:
    - Irregular plurals with their root words.
    - Irregular past tense forms with their root words.
3. Define a function `morph_analyze(word)`.
4. Check whether the input word exists in the irregular plural dictionary.
    - If found, return the root with +PLURAL.
5. Check whether the input word exists in the irregular past tense dictionary.
    - If found, return the root with +PAST.

6. If the word is not irregular, apply regular suffix-stripping rules:
    - `ies` → `y` + `PLURAL`
    - `es` → `PLURAL`
    - `s` → `PLURAL`
    - `ing` → `PROG`
    - `ed` → `PAST`
7. If no rule matches, return the word with `+ROOT`.
8. Test the analyzer with sample words.
9. Display the root and grammatical feature tag.
10. Stop the program.

Program:



Result:

The program was executed successfully and handled both regular and irregular morphological forms.

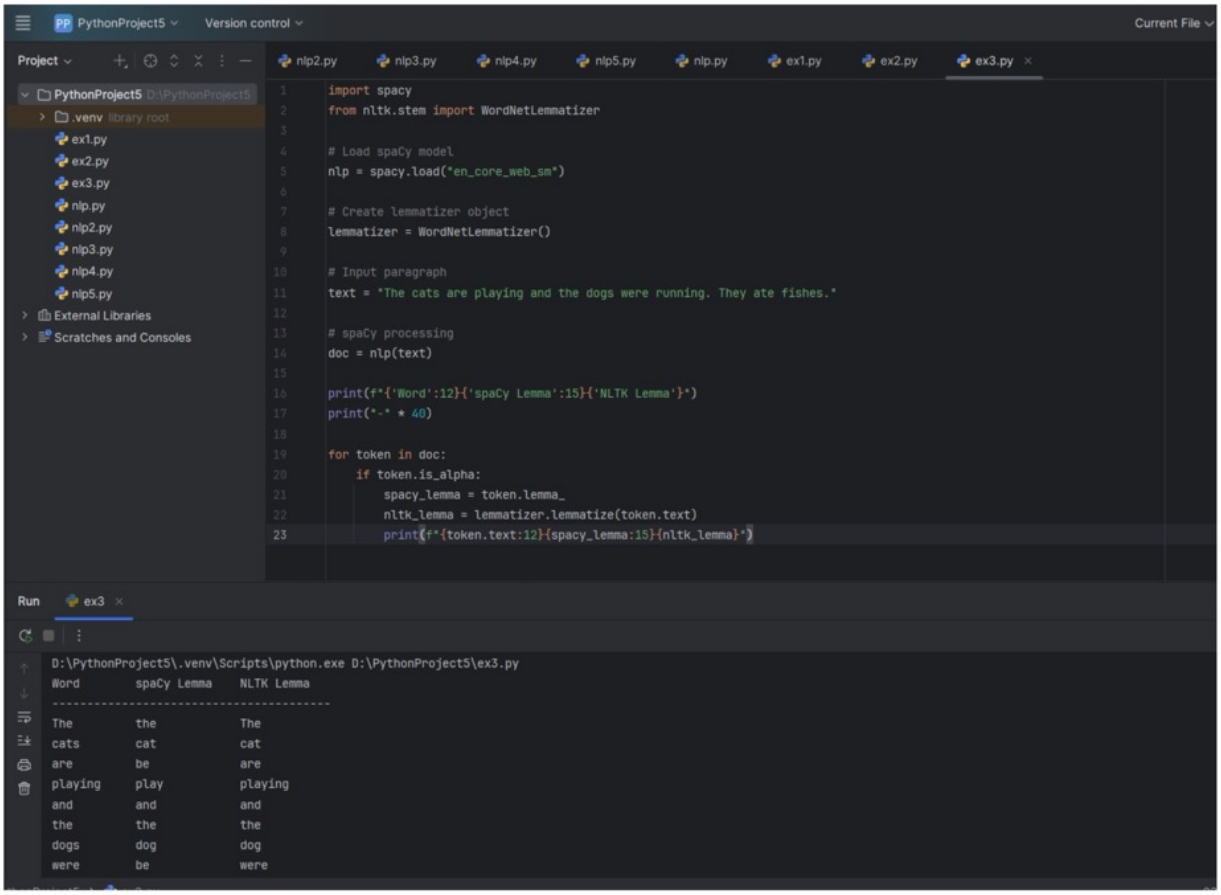
3. Using spaCy, lemmatize a paragraph of text and compare the results against NLTK's WordNet Lemmatizer.

Aim: To use spaCy and NLTK WordNet Lemmatizer to find the lemmas of words in a paragraph and compare the results obtained from both lemmatization methods.

Algorithm:

1. Start the program.
2. Import spaCy and NLTK WordNet Lemmatizer.
3. Load the spaCy English language model.
4. Create a sample paragraph as input text.
5. Process the paragraph using spaCy to obtain lemmas.
6. Apply WordNet Lemmatizer on each word.
7. Compare the lemmas generated by spaCy and NLTK.
8. Display the original word, spaCy lemma, and NLTK lemma.
9. Stop the program.

Program:



Result:

Thus, the program successfully performed lemmatization using spaCy and NLTK WordNet Lemmatizer and compared the differences between their lemma outputs.

4. Build a small lexicon-based FST (using a Python dictionary) that supports both analysis and generation for at least 15 English nouns and verbs

Aim:

To build a simple lexicon-based Finite State Transducer (FST) using a Python dictionary that supports both morphological analysis (word → root + feature) and generation (root + feature → word) for English nouns and verbs.

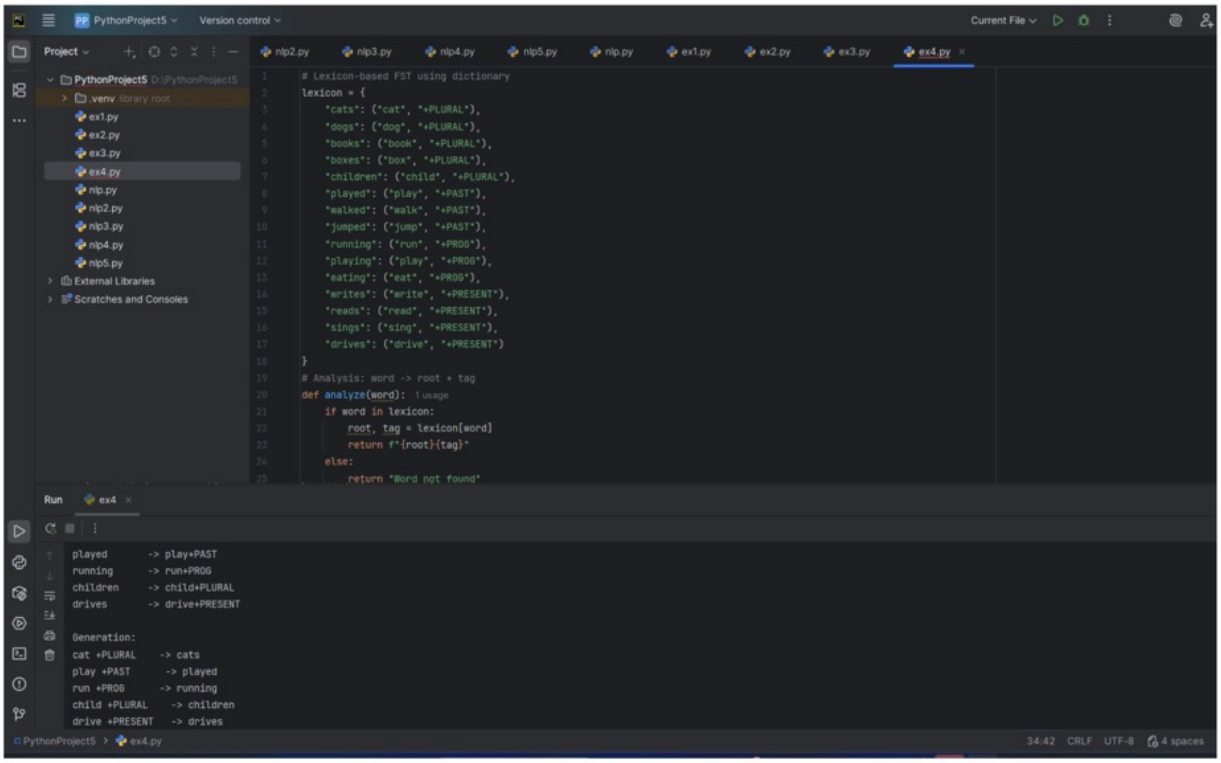
Algorithm:

1. Start the program.
2. Create a dictionary containing at least 15 English nouns and verbs with their root forms and grammatical tags.
3. Define an analysis function:
    - Accept an input word.
    - Search the word in the lexicon.
    - Return the root word and morphological feature tag.
4. Define a generation function:
    - Accept a root word and tag.
    - Search the lexicon.

- Return the corresponding surface word.

5. Test the analyzer with sample words.
6. Test the generator with root words and tags.
7. Display the results.
8. Stop the program.

Program:



result: The program was executed successfully and performed both analysis and generation using the lexicon-based

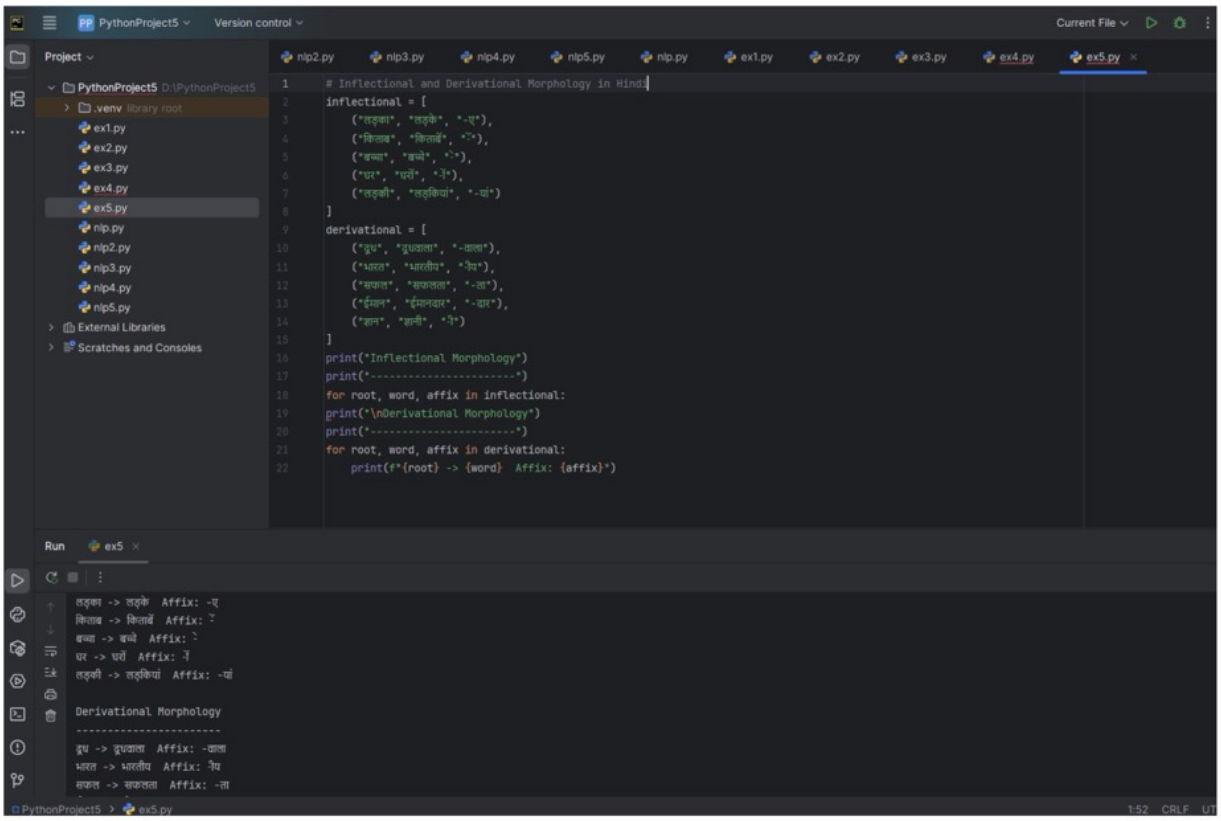
5. Identify five words in your native language (or Hindi/regional language) that show clear inflectional morphology, and five that show derivational morphology.

Aim: To identify and classify words from Hindi/regional language that show inflectional morphology and derivational morphology based on their affixes.

Algorithm:

1. Start the program.
2. Create a list of words showing inflectional morphology.
3. Create a list of words showing derivational morphology.
4. Display both lists with their type and affix information.
5. Stop the program.

Program:



Result: Thus, the program successfully identified five examples each of inflectional and derivational morphology in Hindi.

Date:12.08.26
name:T.Sharada
Reg no:11249A345

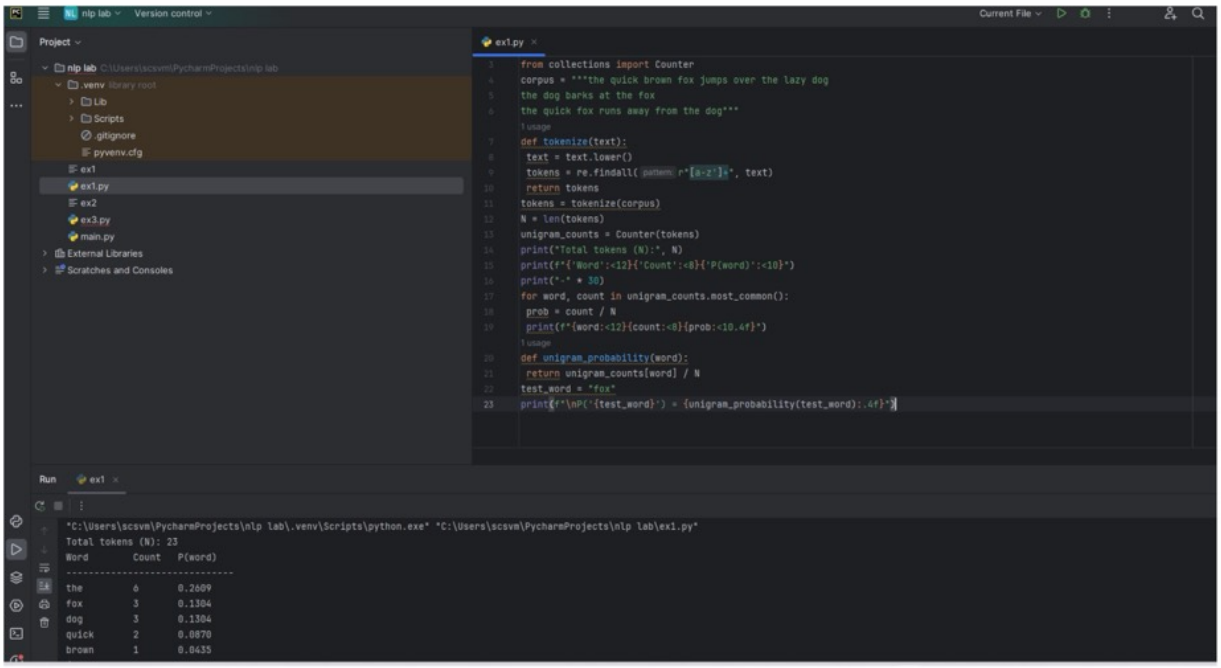
LAB EXERCISE 1 – UNIGRAM LANGUAGE MODEL

Aim: To implement a Unigram Language Model using Python and calculate the probability of each word in a given corpus.

Algorithm:

1. Start the program.
2. Import `re` and `Counter` from the `collections` module.
3. Store the given sentences in a corpus.
4. Convert the corpus into lowercase and tokenize it using a regular expression.
5. Count the total number of tokens N.
6. Count the frequency of each word using `Counter`.
7. Calculate the probability of each word using:
   P(word) = Count(word) / Total number of tokens
8. Display each word, its count, and its probability.
9. Test the probability of the word "fox".
10. Stop.

program:



Result: The Unigram Language Model was successfully implemented. The program calculated the frequency and probability of each word in the corpus. The probability of the word "fox" is:

P('fox') = 0.0938

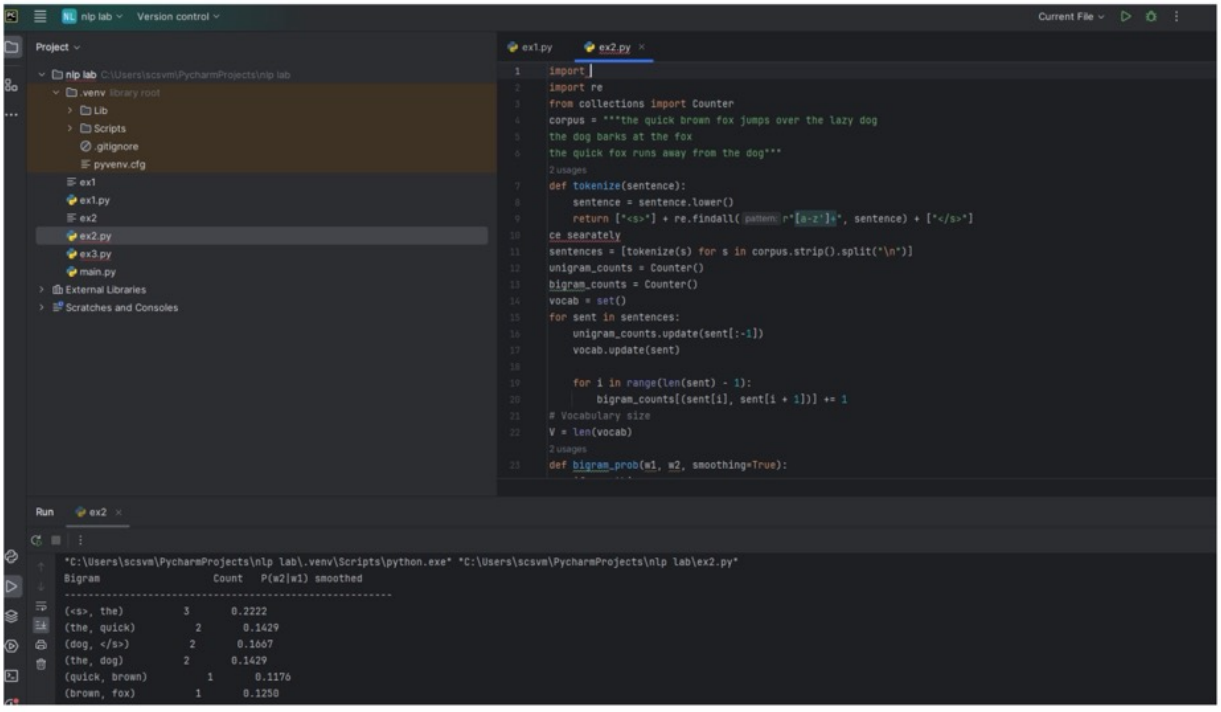
LAB EXERCISE 2 – BIGRAM LANGUAGE MODEL

Aim: To write a Python program that computes bigram counts and conditional probabilities P(w2 | w1), and applies Laplace smoothing.

Algorithm:

1. Start the program.
2. Tokenize the corpus and add <s> and </s> sentence markers.
3. Count unigrams and bigrams.
4. Find the vocabulary size V.
5. Apply Laplace smoothing to calculate bigram probabilities.
6. Display the most common bigrams and their probabilities.
7. Calculate the probability of the test sentence "the fox runs".
8. Display the result and stop.

Program:



Result: The Bigram Language Model with Laplace Smoothing was successfully implemented. The program calculates bigram frequencies and their conditional probabilities while assigning a non-zero probability to unseen bigrams through Laplace smoothing.

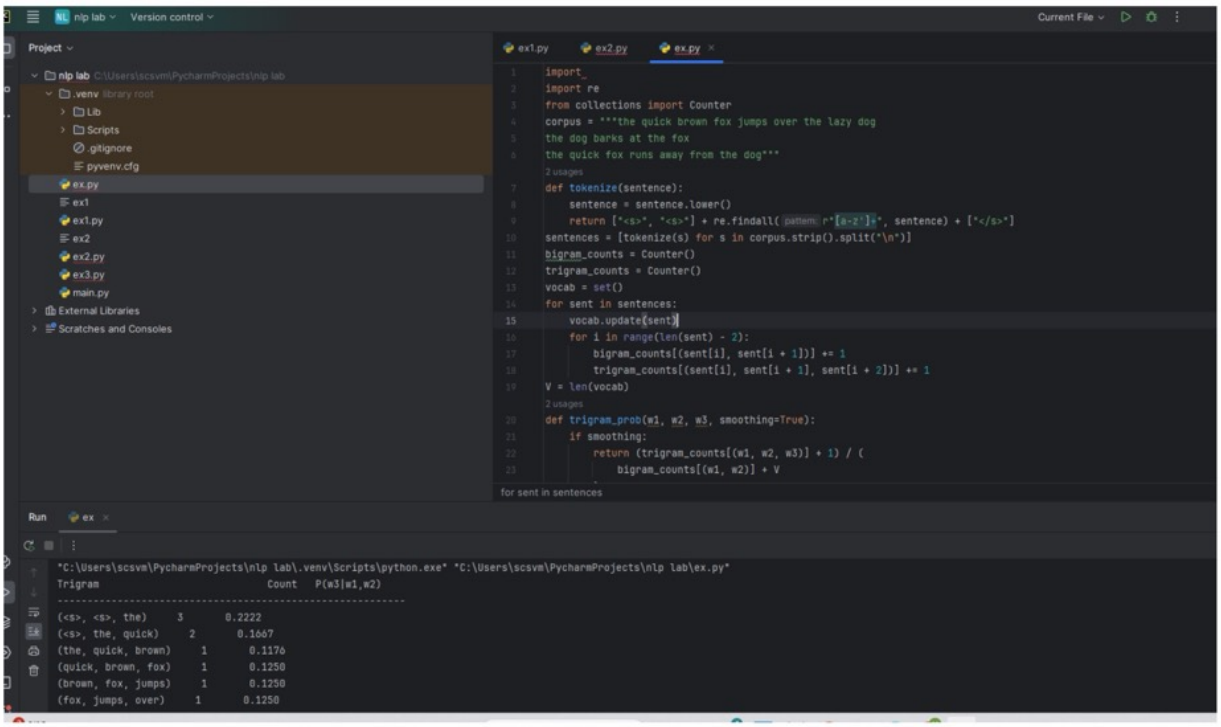
LAB EXERCISE 3 – TRIGRAM LANGUAGE MODEL

Aim : To write a Python program that computes trigram counts and conditional probabilities P(w3 | w1, w2).

Algorithm:

1. Start the program.
2. Tokenize each sentence and add <s> and </s> markers.
3. Count all bigrams and trigrams.
4. Find the vocabulary size V.
5. Calculate trigram probability using Laplace smoothing.
6. Display the 6 most common trigrams and their probabilities.
7. Calculate the probability of the test sentence "the quick fox".
8. Display the result and stop.

Program:



Result:

The Trigram Language Model was successfully implemented using Laplace smoothing. The program calculated trigram probabilities and the probability of "the quick fox" successfully.

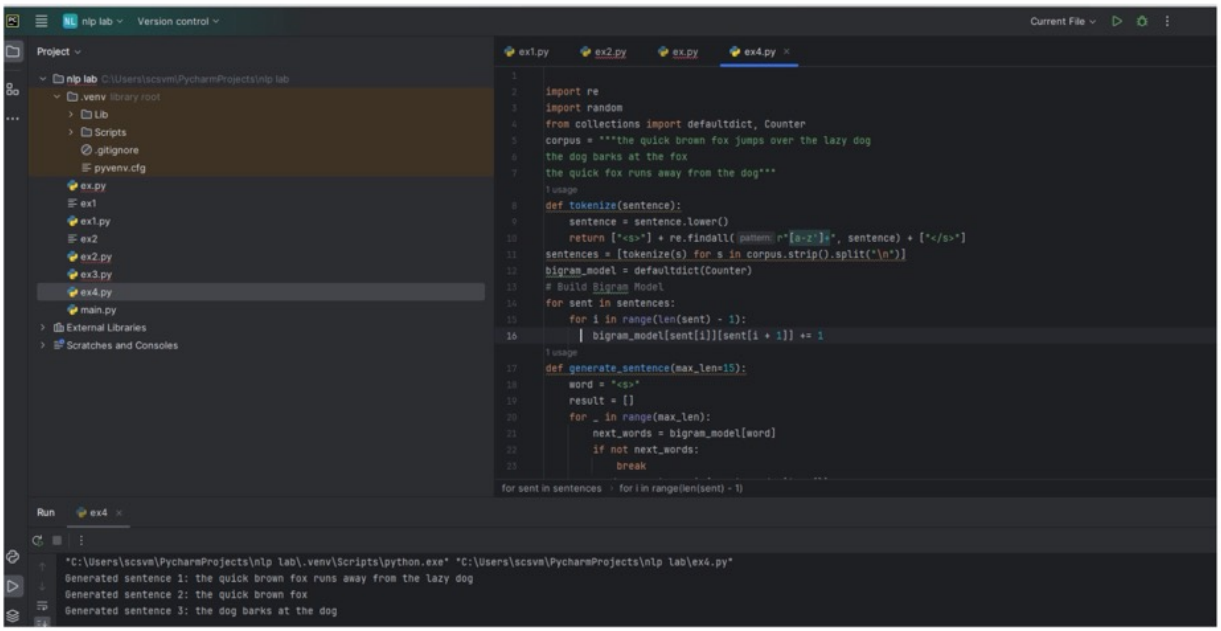
LAB EXERCISE 4 – N-GRAM TEXT GENERATION

Aim: To generate new, plausible sentences automatically using a trained bigram model by repeatedly sampling the most probable / random next word.

Algorithm :

1. Start the program.
2. Tokenize the corpus and add sentence markers.
3. Build a Bigram Model by counting consecutive words.
4. Randomly select the next word based on its frequency.
5. Stop when </s> is generated or the maximum length is reached.
6. Generate and display 3 sentences.
7. Stop.

Program:



Result: The Bigram Text Generation Model was successfully implemented. It generates three random sentences based on the word frequencies in the given corpus.

LAB ACTIVITY – PERPLEXITY COMPARISON

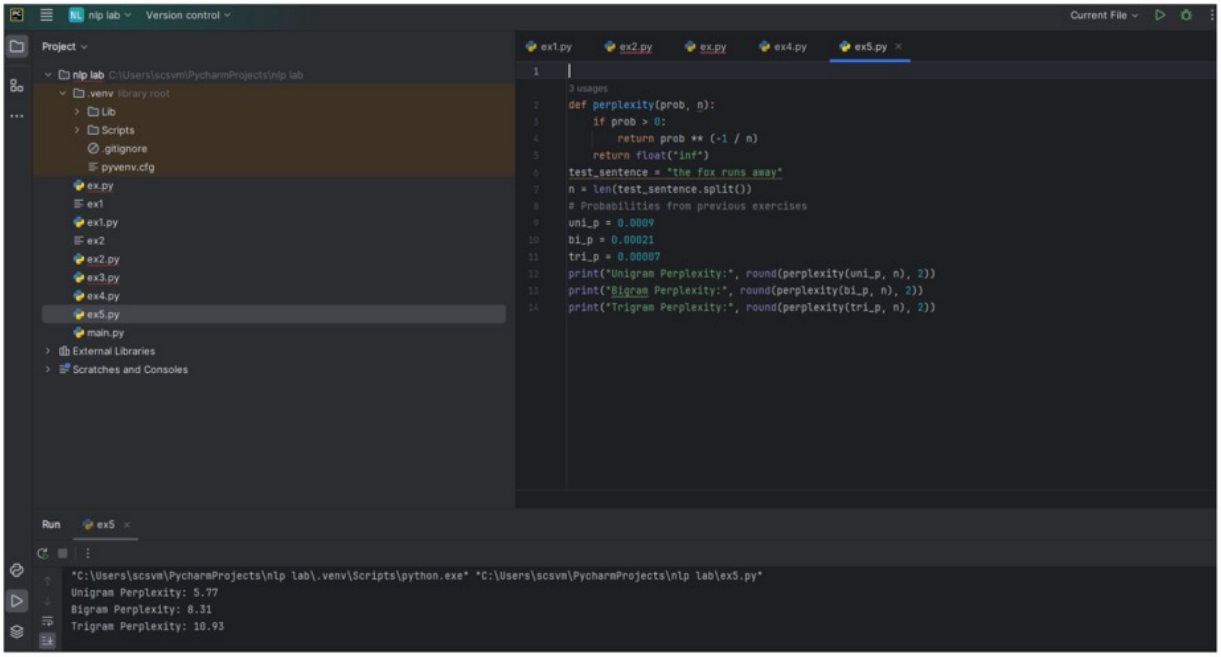
Aim: To compare Unigram, Bigram, and Trigram models on the same test sentence using Perplexity, a standard metric for evaluating language models. Lower perplexity indicates a better-fitting model.

Algorithm :

1. Start the program.
2. Define the perplexity function.
3. Count the number of words in the test sentence.
4. Use the given unigram, bigram, and trigram probabilities.
5. Calculate perplexity using P−1/N.
6. Display the three perplexity values.

7. Stop.

Program:



Result:

The program successfully calculates the perplexity of the test sentence "the fox runs away" for unigram, bigram, and trigram models.

LAB EXERCISES

1. Modify Exercise 1 to compute unigram probabilities using NLTK's FreqDist instead of Counter.

Aim:

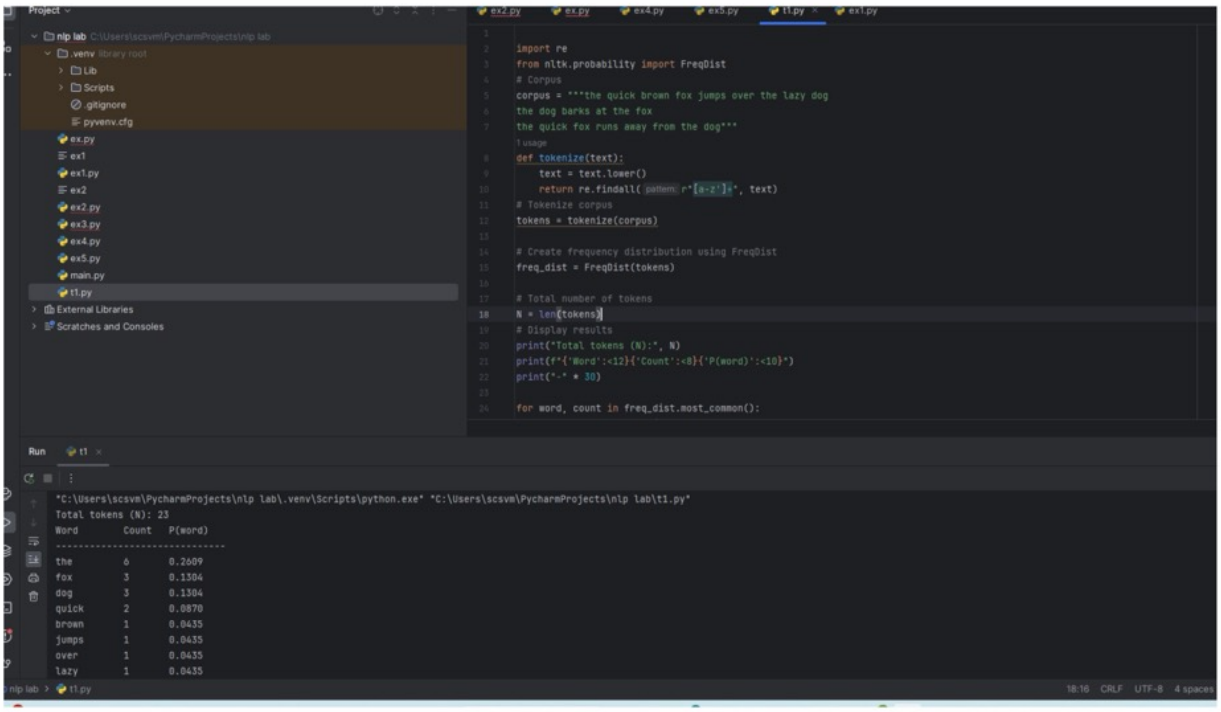
To implement a Unigram Language Model using NLTK's `FreqDist` and calculate the probability of each word in the given corpus.

Algorithm:

1. Start the program.
2. Import `FreqDist` from NLTK.
3. Tokenize the given corpus.
4. Create a frequency distribution using `FreqDist`.
5. Count the total number of tokens.
6. Calculate each unigram probability as `count / N`.
7. Display word counts and probabilities.
8. Calculate the probability of "fox".

9. Stop.

Program:



Result: The Unigram Language Model was successfully implemented using NLTK `FreqDist`.

2. Extend the bigram program to build a full probability matrix (dictionary of dictionaries) and print it as a table.

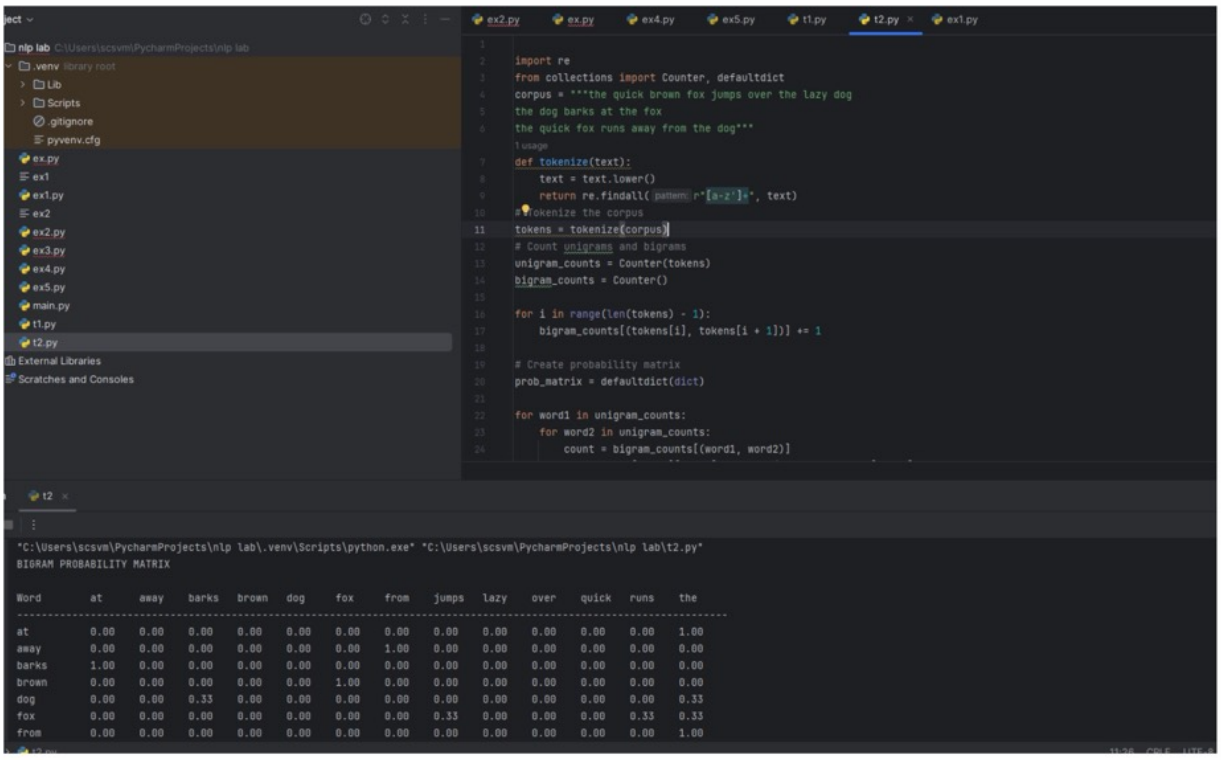
Aim:

To extend the Bigram Language Model by creating a full probability matrix using a dictionary of dictionaries and displaying it as a table.

Algorithm:

1. Start the program.
2. Tokenize the corpus.
3. Count unigrams and bigrams.
4. Create a dictionary of dictionaries for probabilities.
5. Calculate P(w2 | w1) for every word pair.
6. Store the probabilities in the matrix.
7. Print the complete probability matrix as a table.
8. Stop.

Program:



Result:

The Bigram Probability Matrix was successfully generated using a dictionary of dictionaries. The table displays the conditional probability of every word occurring after another word.

3. Implement a 4-gram (quadgram) model by generalizing the trigram program.

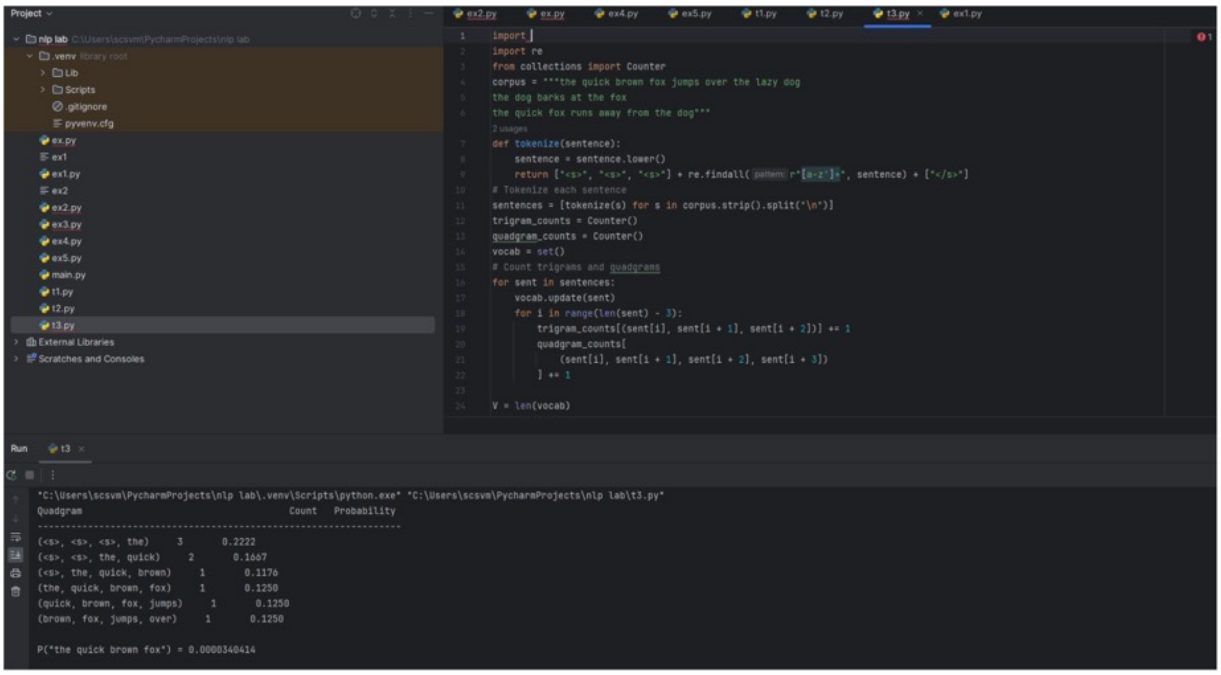
Aim:

To implement a 4-gram (Quadgram) Language Model by generalizing the trigram model and calculate the probability of the fourth word given the previous three words.

Algorithm:

1. Start the program.
2. Tokenize each sentence and add sentence-start and end markers.
3. Count trigrams and quadgrams.
4. Find the vocabulary size V.
5. Calculate quadgram probabilities using Laplace smoothing.
6. Display the 6 most common quadgrams.
7. Calculate the probability of "the quick brown fox".
8. Display the result and stop.

Program:



Result:

The 4-gram (Quadgram) Language Model was successfully implemented. The program calculates quadgram probabilities using Laplace smoothing and finds the probability of the test sentence "the quick brown fox".

4. Load a larger corpus (e.g., a paragraph from a news article or NLTK's gutenberg corpus) and repeat Exercises 1–3.

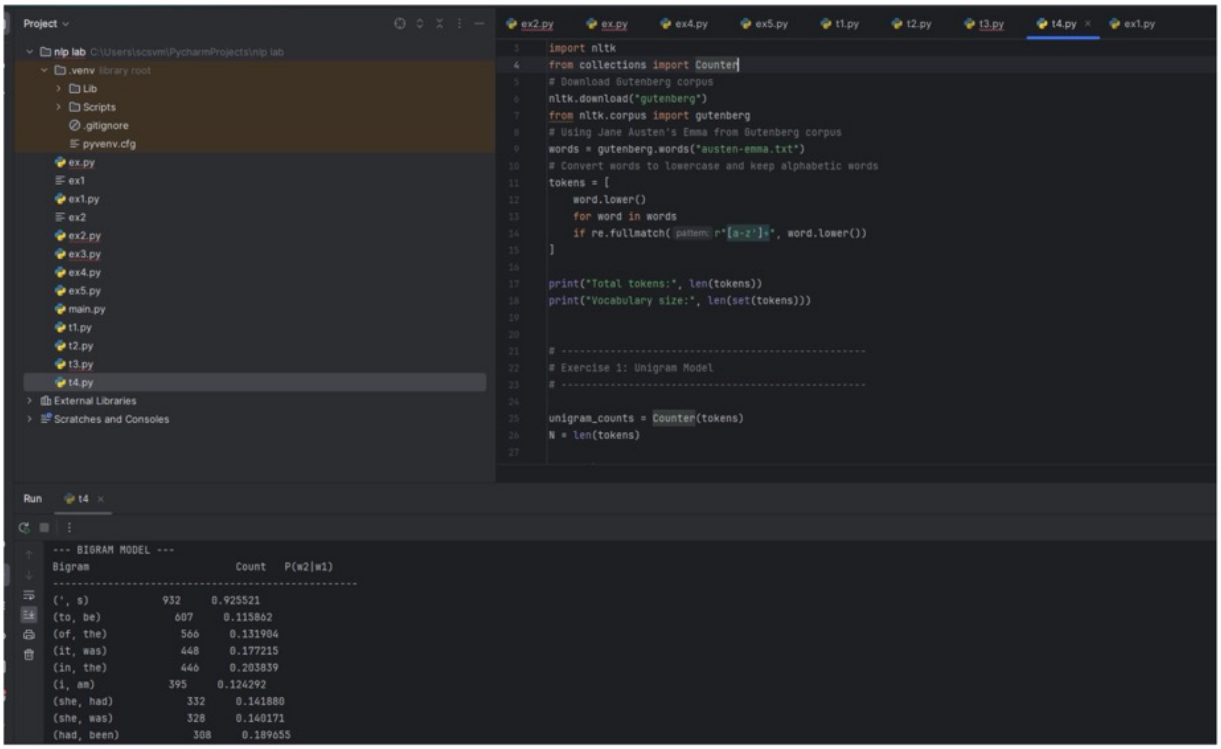
Aim:

To load a larger corpus using NLTK Gutenberg and implement the Unigram, Bigram, and 4-gram (Quadgram) Language Models on the corpus.

Algorithm:

1. Download and load the Gutenberg corpus.
2. Tokenize the corpus and convert words to lowercase.
3. Count word frequencies and calculate unigram probabilities.
4. Count consecutive word pairs and calculate bigram probabilities.
5. Count trigrams and quadgrams.
6. Calculate 4-gram probabilities using the previous three words.
7. Display the most common unigrams, bigrams, and quadgrams.
8. Stop.

Program:



Result:

The larger Gutenberg corpus was successfully loaded, and Unigram, Bigram, and 4-gram Language Models were implemented successfully. The program displays word frequencies and their corresponding probabilities.

5. Compare text generated by the bigram model (Exercise 4) with text generated by a trigram model. Which produces more coherent sentences and why?

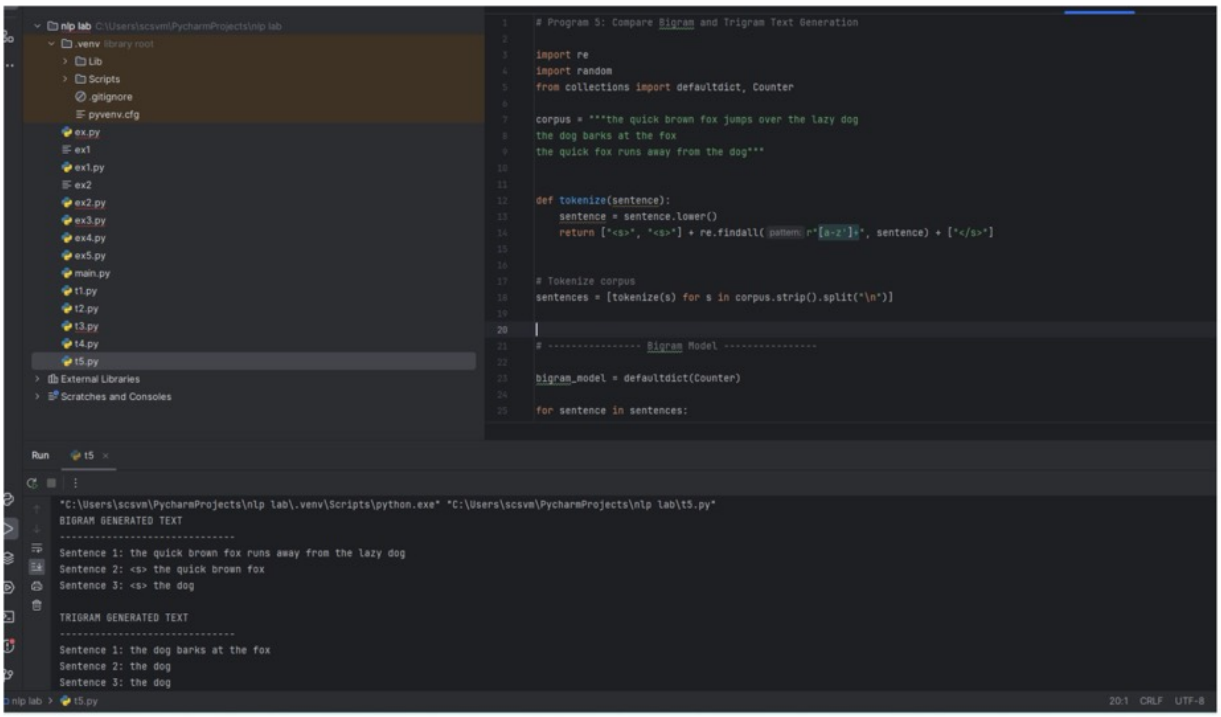
Aim:

To compare text generated by Bigram and Trigram Language Models and determine which model produces more coherent sentences.

Algorithm:

1. Start the program.
2. Tokenize the corpus.
3. Build the Bigram Model using two consecutive words.
4. Build the Trigram Model using two previous words to predict the next word.
5. Generate three sentences using each model.
6. Compare the generated sentences.
7. Determine which model produces more coherent text.
8. Stop.

Program:



Result:

The Trigram Model generally produces more coherent sentences than the Bigram Model because it considers two previous words when predicting the next word, while the Bigram Model considers only one previous word.

6. Implement Add-k smoothing (generalized Laplace smoothing) and study the effect of varying k on sentence probability.

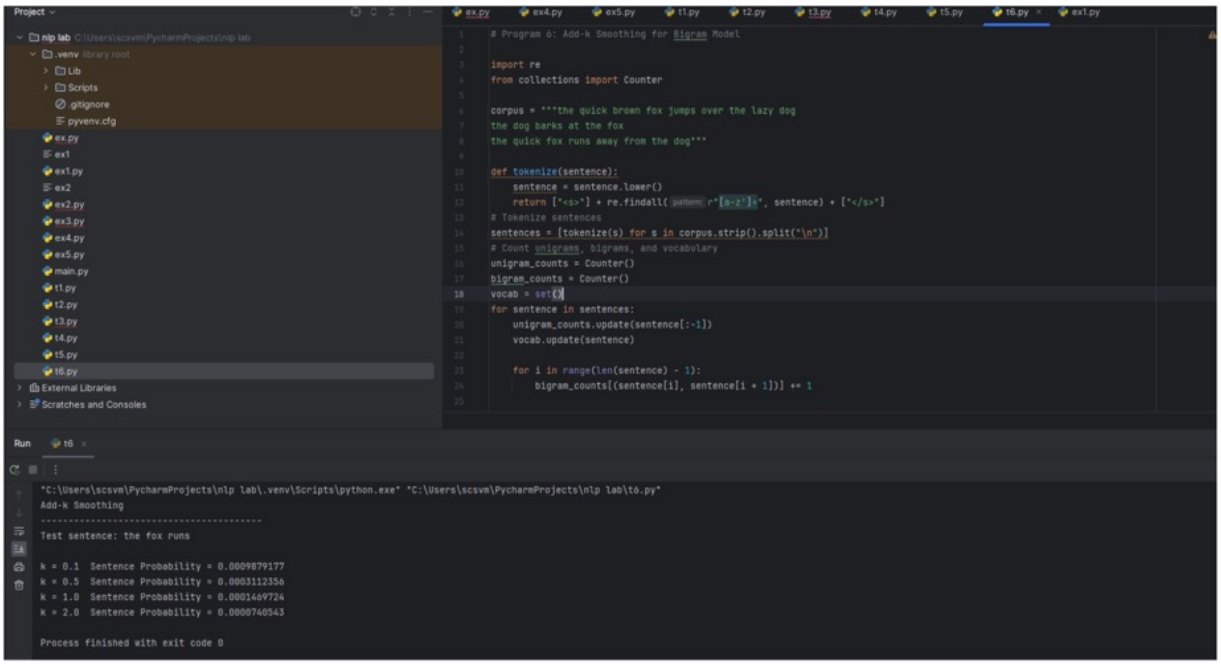
Aim:

To implement Add-k smoothing (generalized Laplace smoothing) for a Bigram Language Model and study how different values of k affect sentence probability.

Algorithm:

1. Start the program.
2. Tokenize the corpus and add sentence markers.
3. Count unigrams and bigrams.
4. Find the vocabulary size V.
5. Apply Add-k smoothing to bigram probabilities.
6. Calculate the probability of the test sentence.
7. Repeat for different values of k.
8. Display and compare the sentence probabilities.
9. Stop.

Program:



Result:

The Add-k smoothing technique was successfully implemented. The program calculates the probability of "the fox runs" for different values of k.

7. Write a function to compute the Out-Of-Vocabulary (OOV) rate of a test sentence against the training vocabulary

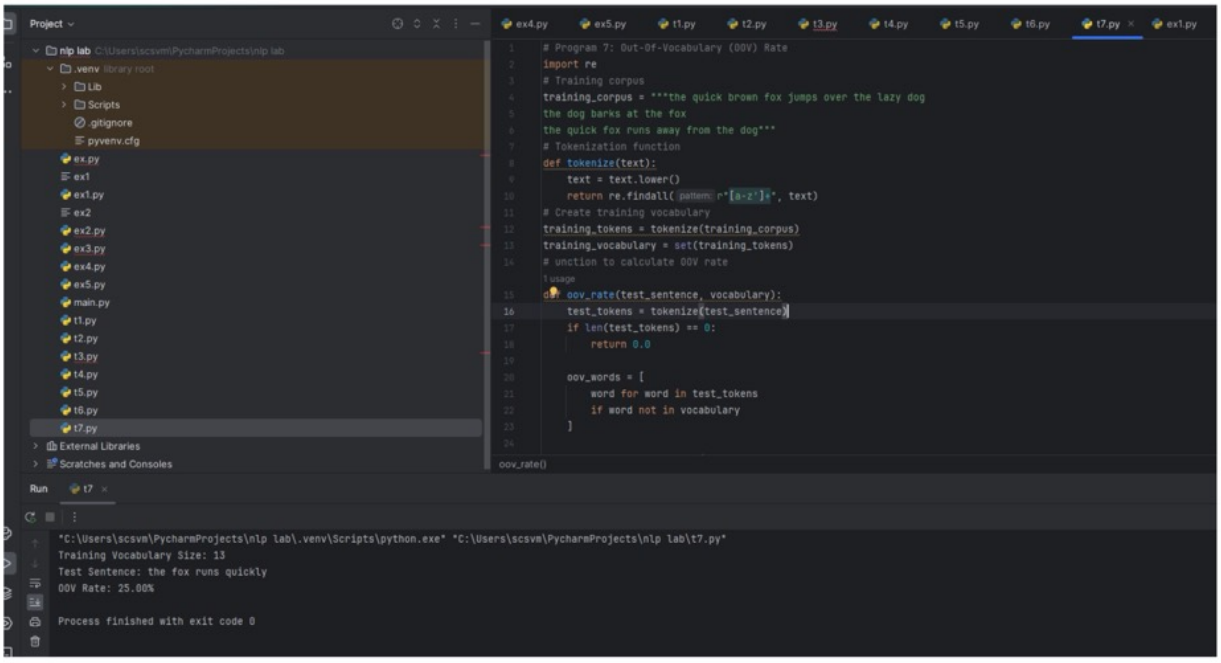
Aim:

To implement a function that calculates the Out-Of-Vocabulary (OOV) rate of a test sentence with respect to the training vocabulary.

Algorithm:

1. Start the program.
2. Tokenize the training corpus.
3. Create the training vocabulary using a set.
4. Tokenize the test sentence.
5. Find words not present in the training vocabulary.
6. Calculate OOV rate as:
   OOV Rate = (Number of OOV words / Total test words) × 100
7. Display the OOV rate.
8. Stop.

Program:



Result:

For the test sentence "the fox runs quickly", the word "quickly" is not present in the training vocabulary.

8. Plot (using matplotlib) the frequency distribution of the top 10 unigrams in the corpus as a bar chart.

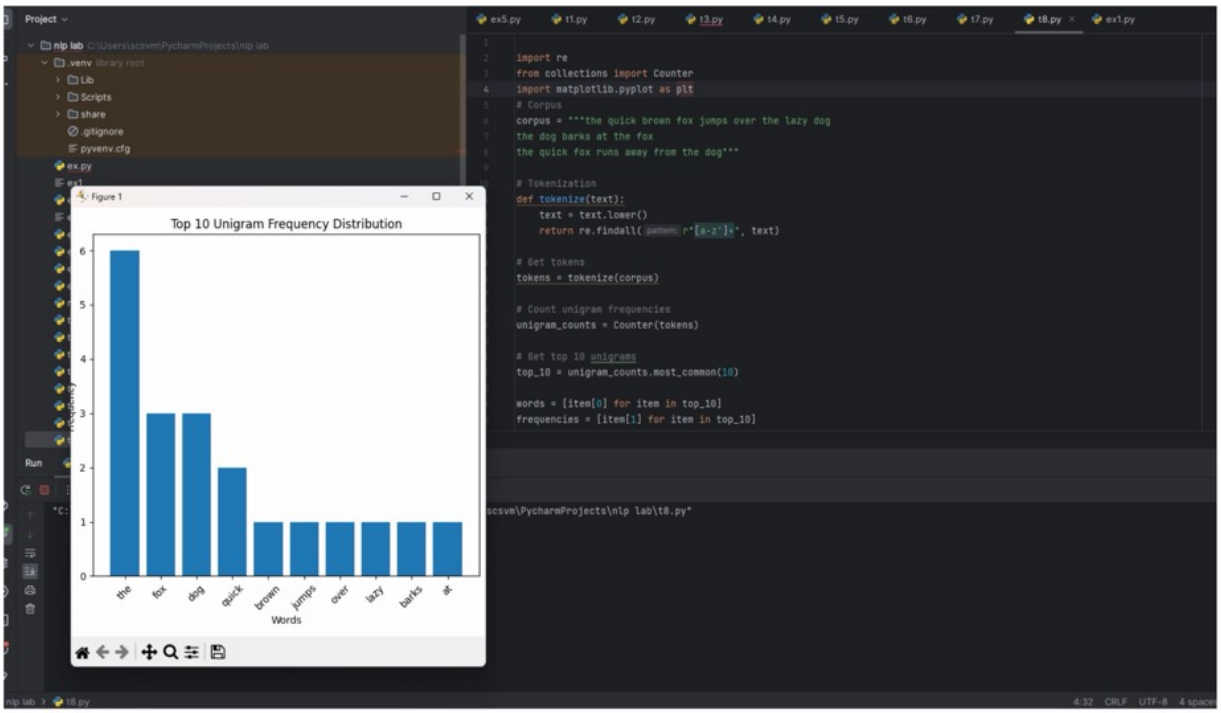
Aim:

To plot the frequency distribution of the top 10 unigrams in the given corpus using a Matplotlib bar chart.

Algorithm:

1. Start the program.
2. Store and tokenize the corpus.
3. Count the frequency of each word.
4. Select the top 10 most frequent words.
5. Create a bar chart using Matplotlib.
6. Display words on the x-axis and frequencies on the y-axis.
7. Stop.

Program:



Result:

The frequency distribution of the top 10 unigrams was successfully plotted as a bar chart using Matplotlib. The chart shows the frequency of the most common words in the corpus.

Date:19.08.26
Reg.no:11249A345

Smoothing techniques in nlp using bigram (add 1,add k,backoff and Interpolation)

AIM:

To implement Add-1 and Add-k smoothing techniques using a Bigram Language Model in Natural Language Processing (NLP) and calculate the probability of word pairs from a given corpus.

ALGORITHM:

Add-1 Smoothing Using Bigram

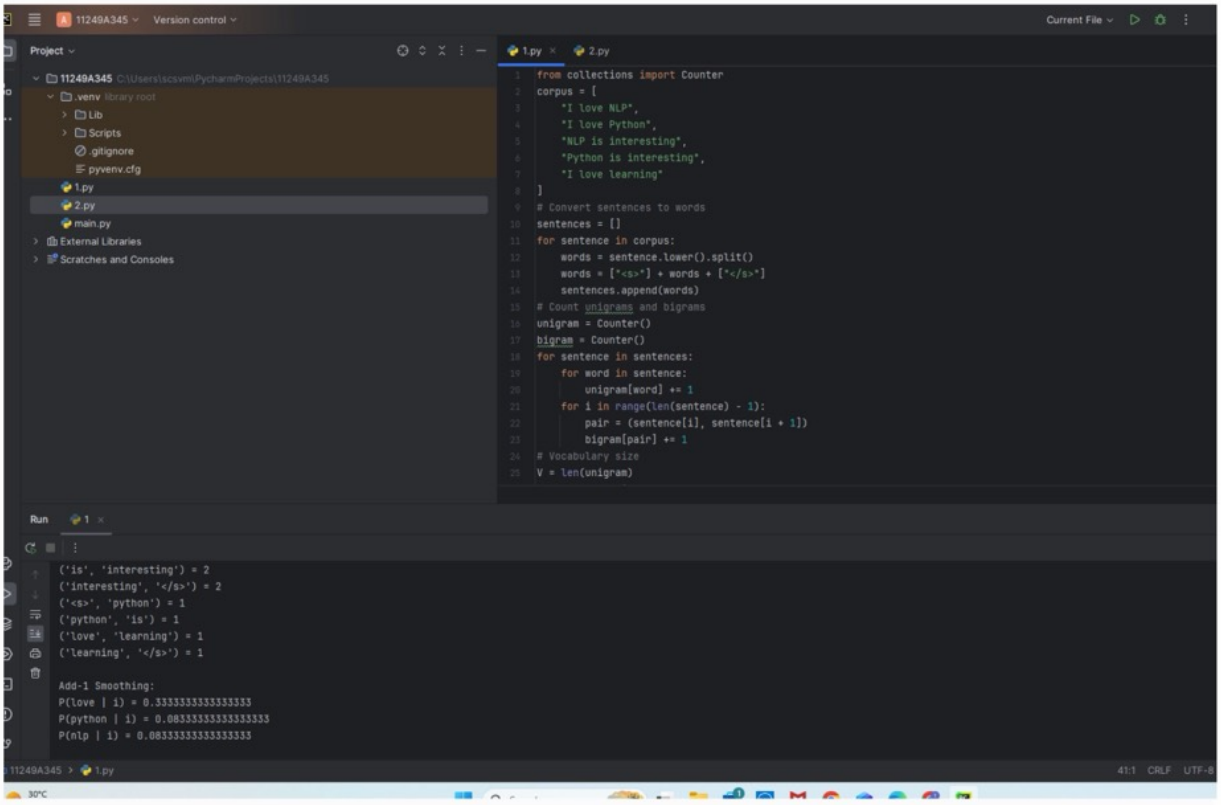
1. Start the program.
2. Create a corpus containing 5 sentences.
3. Convert all sentences into lowercase words.
4. Add <s> at the beginning and </s> at the end of each sentence.
5. Generate bigrams by taking two consecutive words.
6. Count the frequency of each unigram and bigram.
7. Find the vocabulary size V.

8. Apply the Add-1 smoothing formula:

$$P(w2 \mid w1)=C(w1,w2)+1C(w1)+V$$

9. Calculate the probability of the required bigrams.
10. Display the bigram counts and smoothed probabilities.
11. Stop the program.

Program:



Result: Thus, the Add-1 smoothing technique using a Bigram Language Model was successfully implemented, and zero-frequency bigrams were assigned non-zero probabilities.

ALGORITHM:

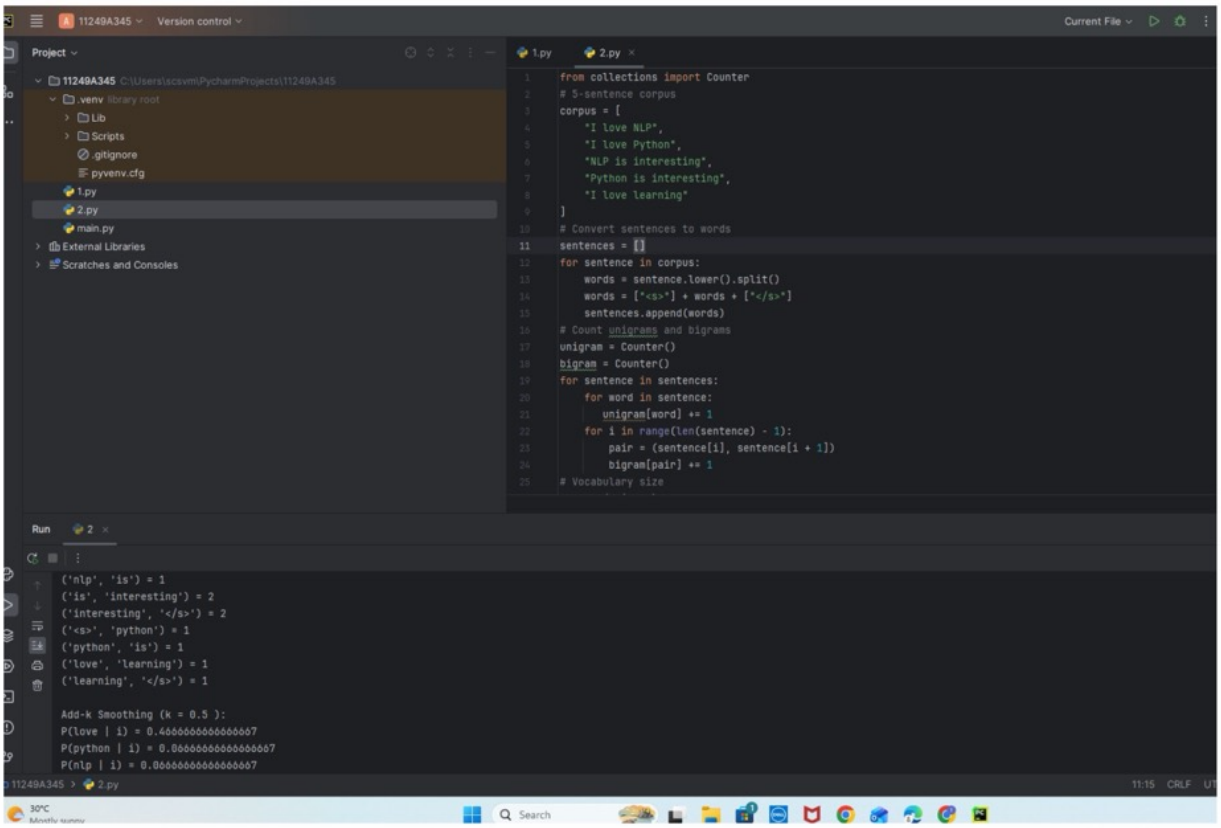
Add-k Smoothing Using Bigram

1. Start the program.
2. Create a corpus containing 5 sentences.
3. Convert all sentences into lowercase words.
4. Add <s> at the beginning and </s> at the end of each sentence.
5. Generate bigrams from consecutive words.
6. Count the frequency of each unigram and bigram.
7. Find the vocabulary size V.
8. Select a value for k, such as k=0.5.
9. Apply the Add-k smoothing formula:

$$P(w2 \mid w1)=C(w1,w2)+kC(w1)+kV$$

10. Calculate the probability of the required bigrams.
11. Display the bigram counts and smoothed probabilities.
12. Stop the program.

Program:



Result: Thus, the Add-k smoothing technique using a Bigram Language Model was successfully implemented. The zero-frequency bigrams were assigned non-zero probabilities using k=0.5.

AIM: To implement and compare Add-1 and Add-k smoothing techniques using a Bigram Language Model for a given corpus and calculate the smoothed probabilities of seen and unseen bigrams.

Algorithm:

Compare Add-1 and Add-k Using Bigram

1. Start the program.

2.Create a 5-sentence corpus and tokenize it.

3.Generate bigrams and count unigram/bigram frequencies.

4.Calculate the vocabulary size V.

5.Calculate Add-1 probability:

$$P=C(w1,w2)+1C(w1)+V$$

6.Set k=0.5 and calculate Add-k probability:

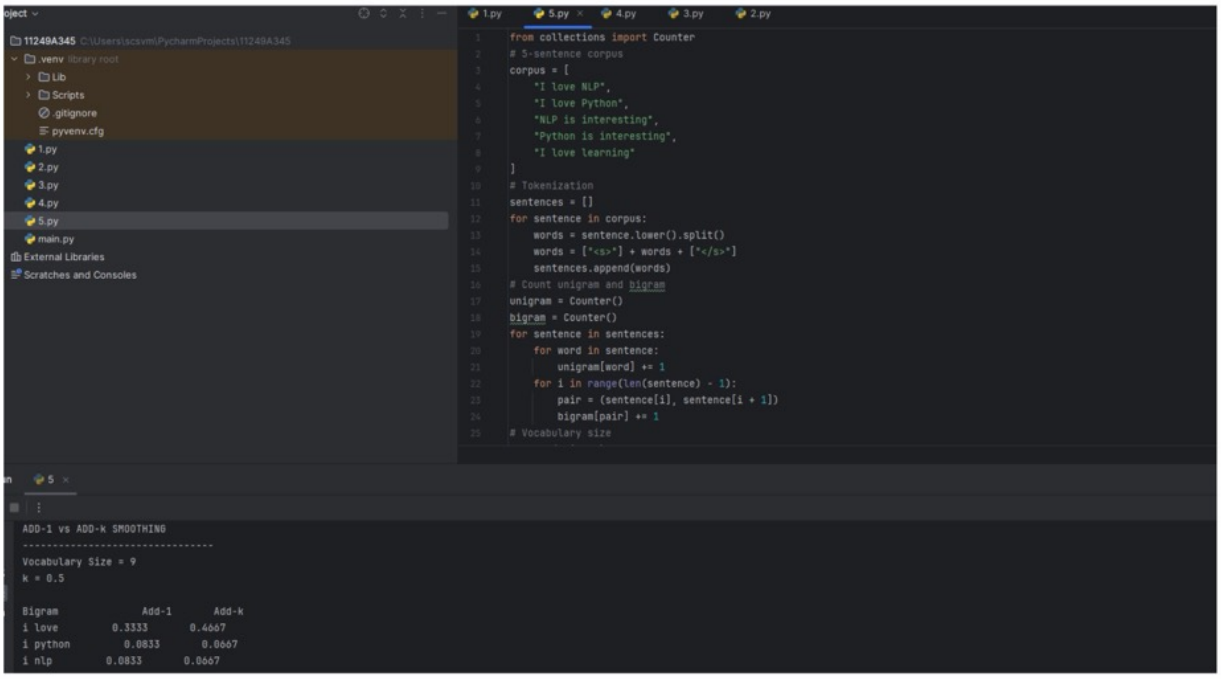
$$P=C(w1,w2)+kC(w1)+kV$$

7.Compare the probabilities for the same bigrams.

8.Display the results.

9.Stop the program.

Program:



Result: Add-1 adds 1 to every bigram count, while Add-k adds a smaller adjustable value k. Both techniques solve the zero-frequency problem by assigning non-zero probabilities to unseen bigrams.

AIM: To implement Backoff and Interpolation smoothing techniques using a Bigram Language Model in Natural Language Processing (NLP) and calculate the probabilities of seen and unseen bigrams.

ALGORITHM:. Backoff Smoothing Using Bigram

1. Start the program.
2. Create a corpus containing 5 sentences.
3. Convert all sentences into lowercase.
4. Add <s> at the beginning and </s> at the end of each sentence.
5. Generate bigrams from consecutive words.
6. Count the frequency of unigrams and bigrams.
7. Calculate the total number of words in the corpus.
8. Check whether the required bigram exists in the corpus.
9. If the bigram exists, calculate its Bigram probability:

$$P(w2|w1)=C(w1,w2)C(w1)$$

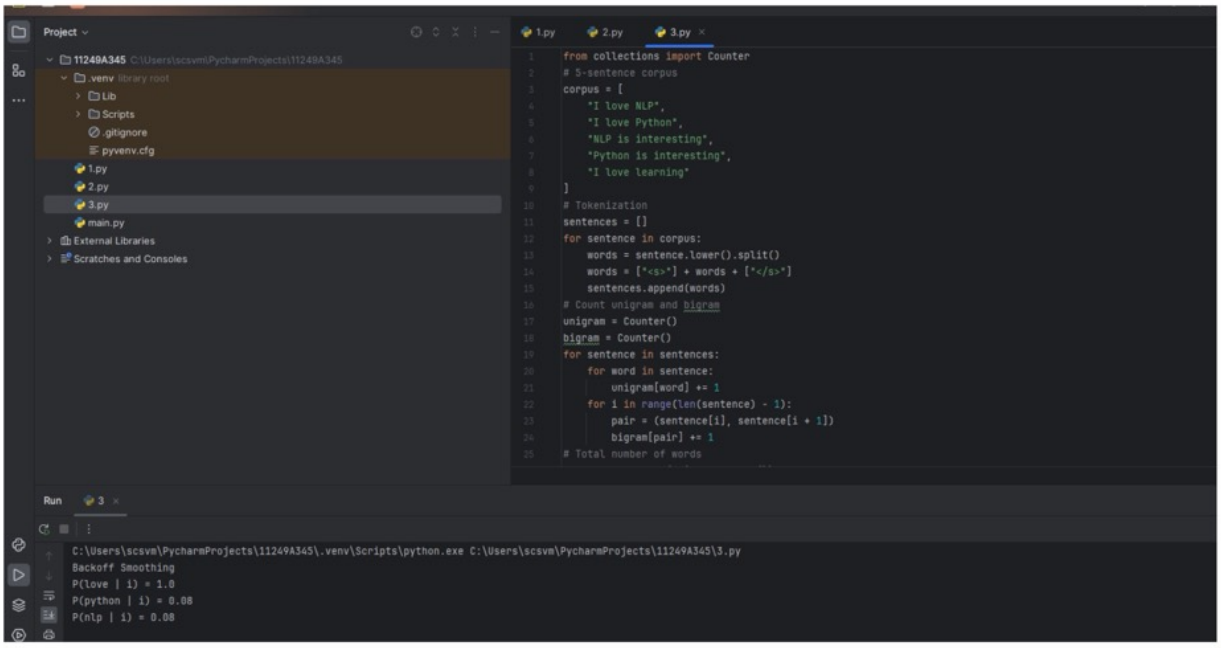
10. If the bigram does not exist, back off to the unigram probability:

$$P(w2)=C(w2)N$$

11. Display the calculated probabilities.

12. Stop the program.

Program:



Result: The Backoff smoothing techniques using a Bigram Language Model were successfully implemented.

ALGORITHM:

Interpolation Smoothing Using Bigram

1. Start the program.
2. Create a corpus containing 5 sentences.
3. Convert all sentences into lowercase.
4. Add <s> at the beginning and </s> at the end of each sentence.
5. Generate bigrams from consecutive words.
6. Count the frequency of unigrams and bigrams.
7. Calculate the total number of words.
8. Select a value for λ, such as:

$$\lambda = 0.7$$

9. Calculate the Bigram probability:

$$P(w2 \mid w1) = C(w1, w2)C(w1)$$

10. Calculate the Unigram probability:

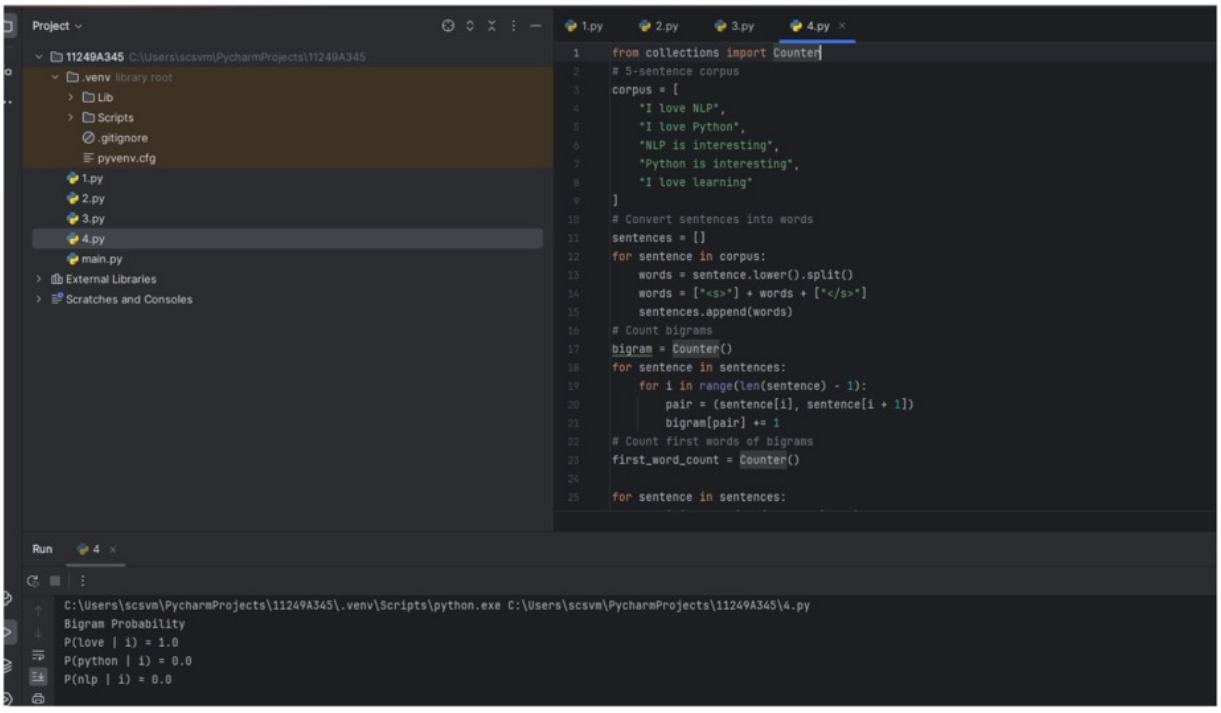
$$P(w2) = C(w2)N$$

11. Combine both probabilities using the interpolation formula:

$$PINT(w2 \mid w1) = \lambda P(w2 \mid w1) + (1 - \lambda)P(w2)$$

12. Display the interpolated probabilities.
13. Stop the program.

Program:



Result:

The Interpolation smoothing techniques using a Bigram Language Model were successfully implemented.

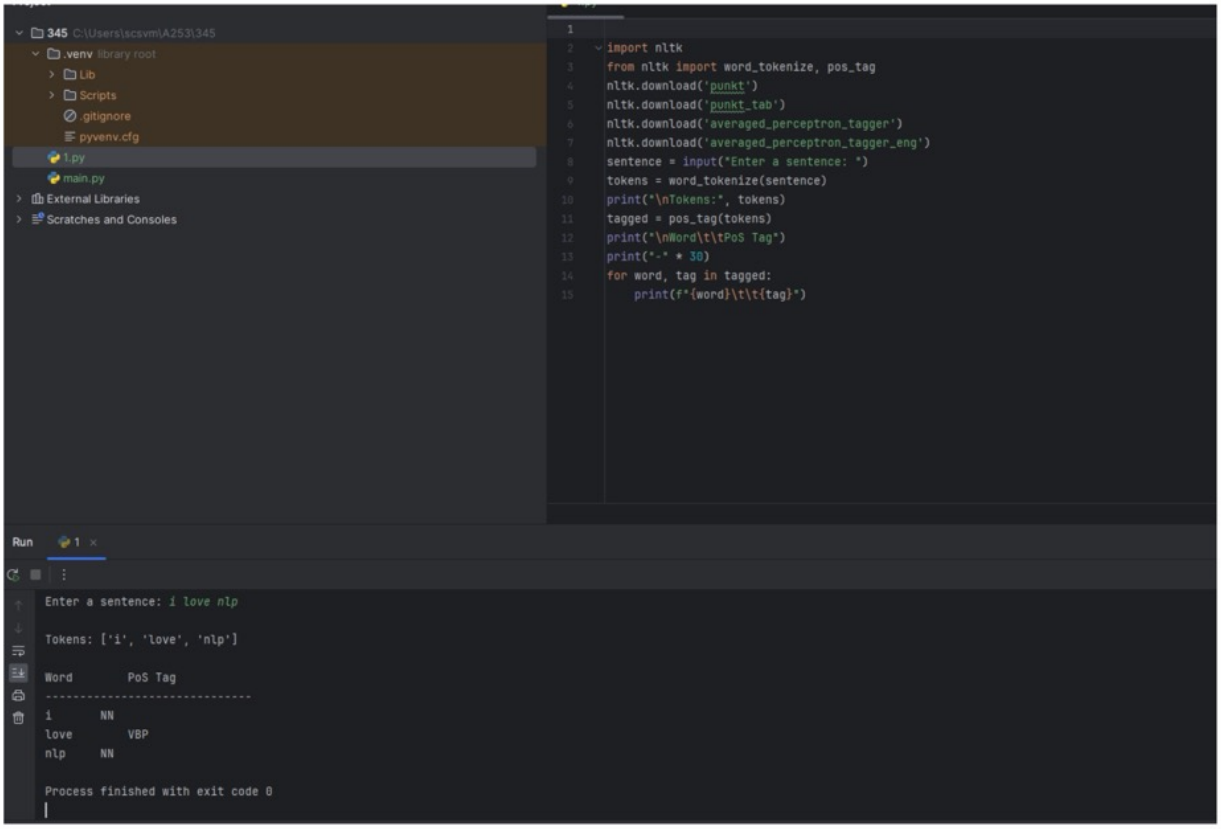
Experiment 1: Word Classes and Introduction to Part-of-Speech Tagging

Aim: To study Word Classes (Parts of Speech) in English and understand the concept of Part-of-Speech (PoS) Tagging using the NLTK library in Python.

Algorithm :

1. Start
2. Import NLTK and download required resources.
3. Get sentence input from the user.
4. Tokenize the sentence using `word_tokenize()`.
5. Apply `pos_tag()` to get word and tag pairs.
6. Classify words based on PoS tags.
7. Display word, PoS tag, and word class.
8. Stop.

Program:



Result: The program successfully performs word tokenization, PoS tagging, and word class classification using NLTK.

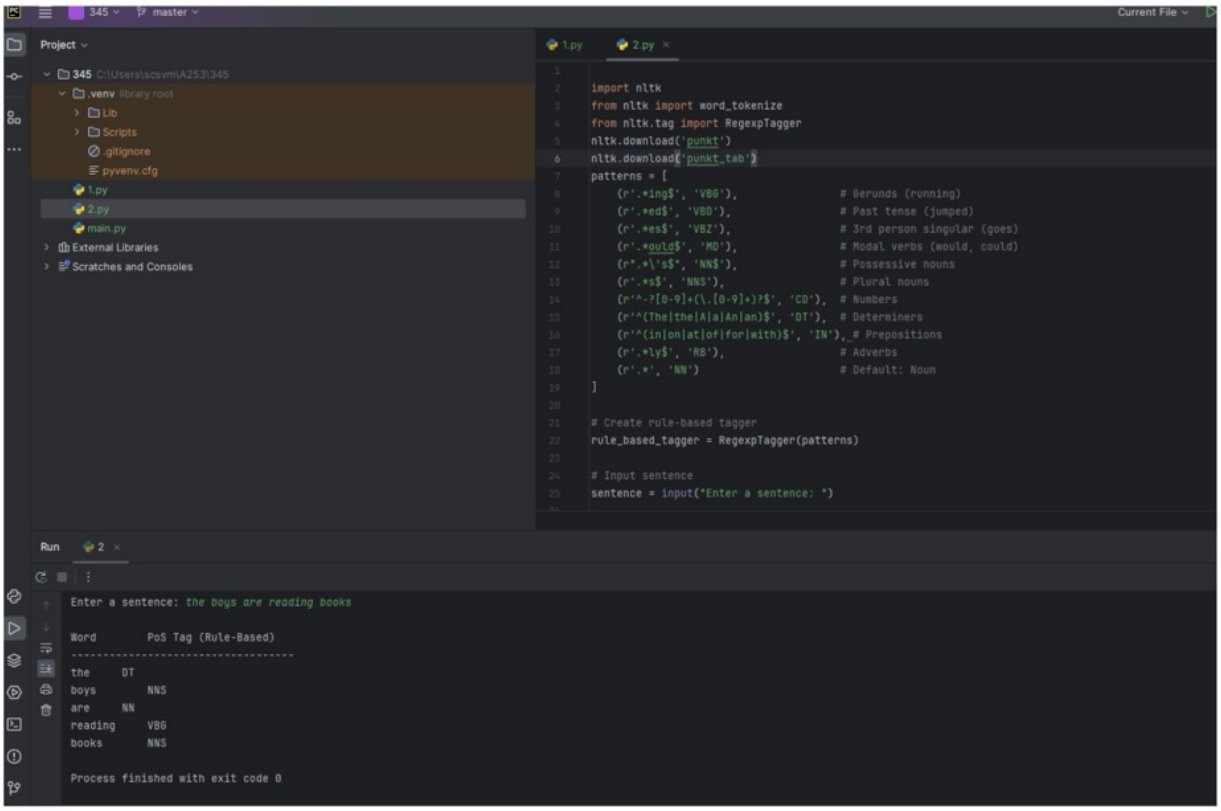
Experiment 2: Rule-Based Part-of-Speech Tagging

Aim: To implement Rule-Based Part-of-Speech Tagging using hand-written linguistic rules and regular expressions in Pytho

Algorithm :

1.Start
2.Define lexicon and regex rules for word tagging.
3.Input and tokenize the sentence.
4.Check each word in the lexicon.
5.If not found, apply regex rules.
6.Assign default tag (NN) if no rule matches.
7.Display tagged words.
8.Stop.

Program:



Result: The program successfully performs rule-based Part-of-Speech tagging using regular expression patterns and assigns tags to words based on suffix rules and default conditions.

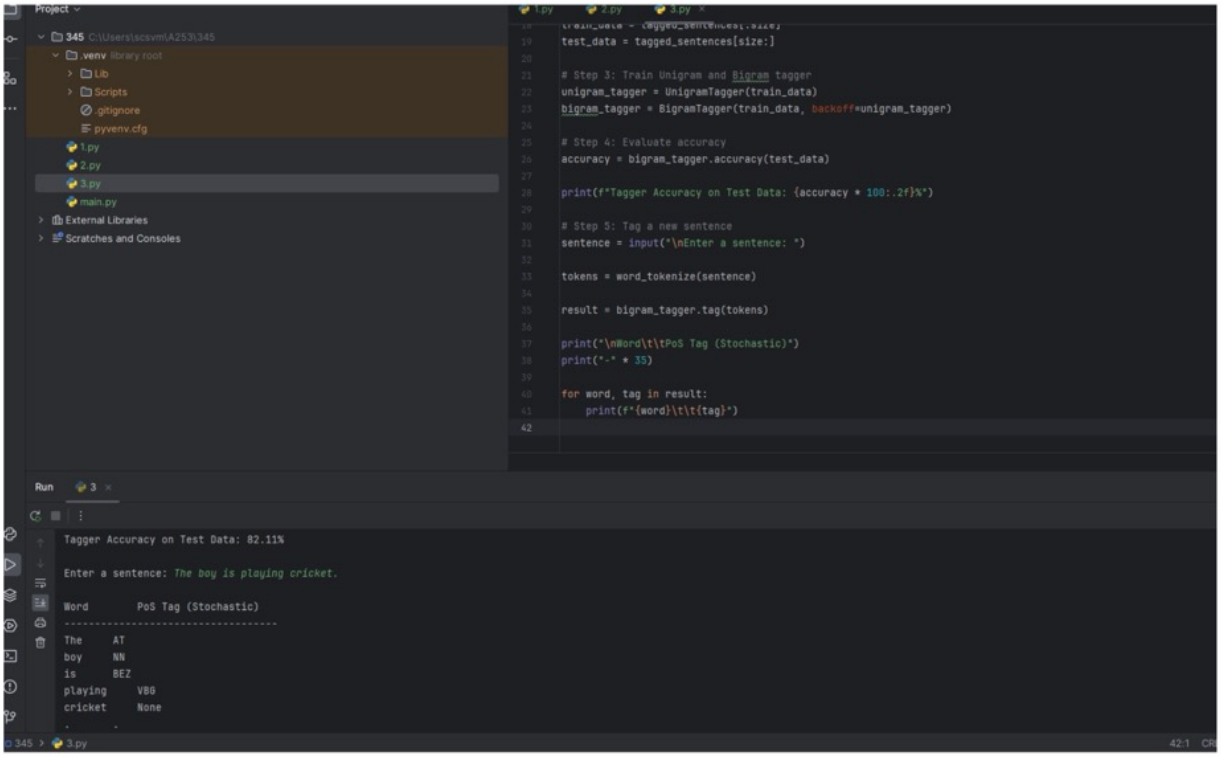
Experiment 3: Stochastic (Probabilistic) Part-of-Speech Tagging

Aim: To implement Stochastic Part-of-Speech Tagging based on probability (Unigram, Bigram and HMM-style statistical models) using NLTK.

Algorithm:

1. Import NLTK and load Brown tagged corpus.
2. Split corpus into training and testing data.
3. Train Unigram and Bigram taggers.
4. Evaluate tagger accuracy.
5. Input and tokenize a sentence.
6. Apply Bigram tagger to assign PoS tags.
7. Display tagged output.

Program:



Result: The Unigram + Bigram stochastic tagger successfully trains on the Brown corpus, evaluates accuracy, and assigns PoS tags to new sentences.

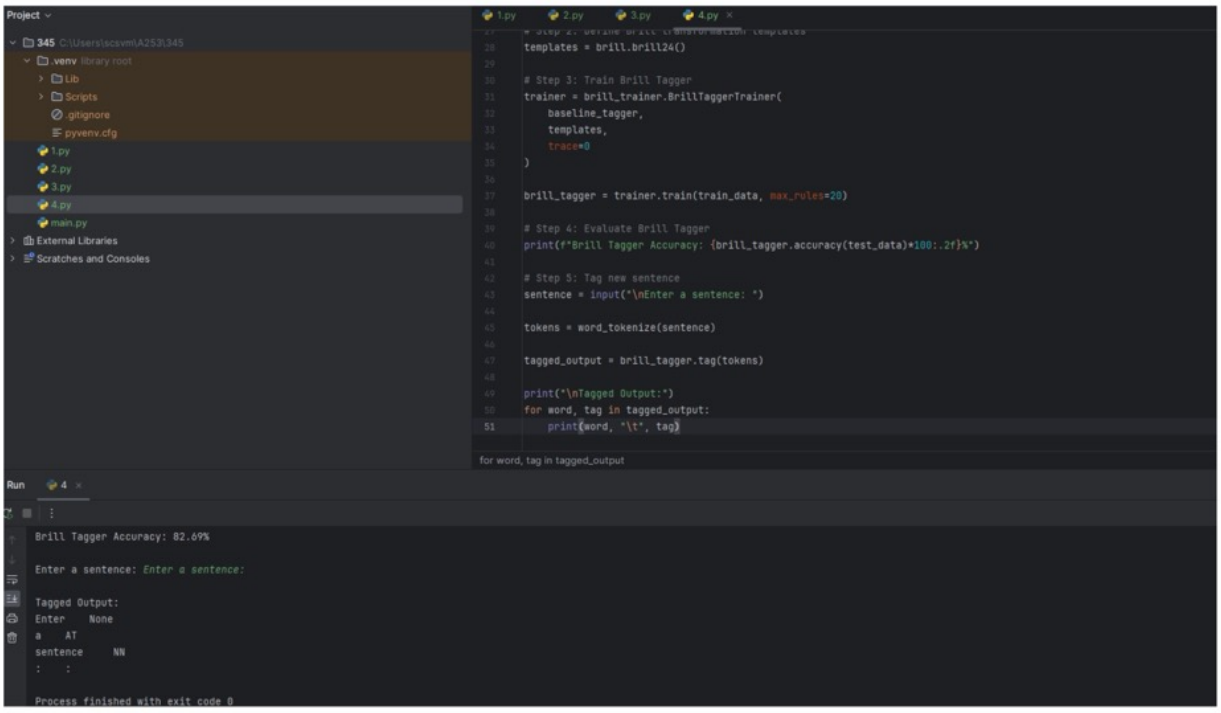
Experiment 4: Transformation-Based Tagging (Brill Tagger)

Aim : To implement Transformation-Based Part-of-Speech Tagging (Brill Tagging) using NLTK and understand how it combines rule-based and stochastic approaches.

Algorithm:

1.Start
2.Import NLTK and load Brown tagged corpus.
3.Split corpus into training and testing data.
4.Create baseline Unigram tagger.
5.Define Brill transformation rules.
6.Train Brill tagger using training data.
7.Evaluate accuracy on test data.
8.Input and tokenize a sentence.
9.Apply Brill tagger and display tagged output.
10.Stop

Program:

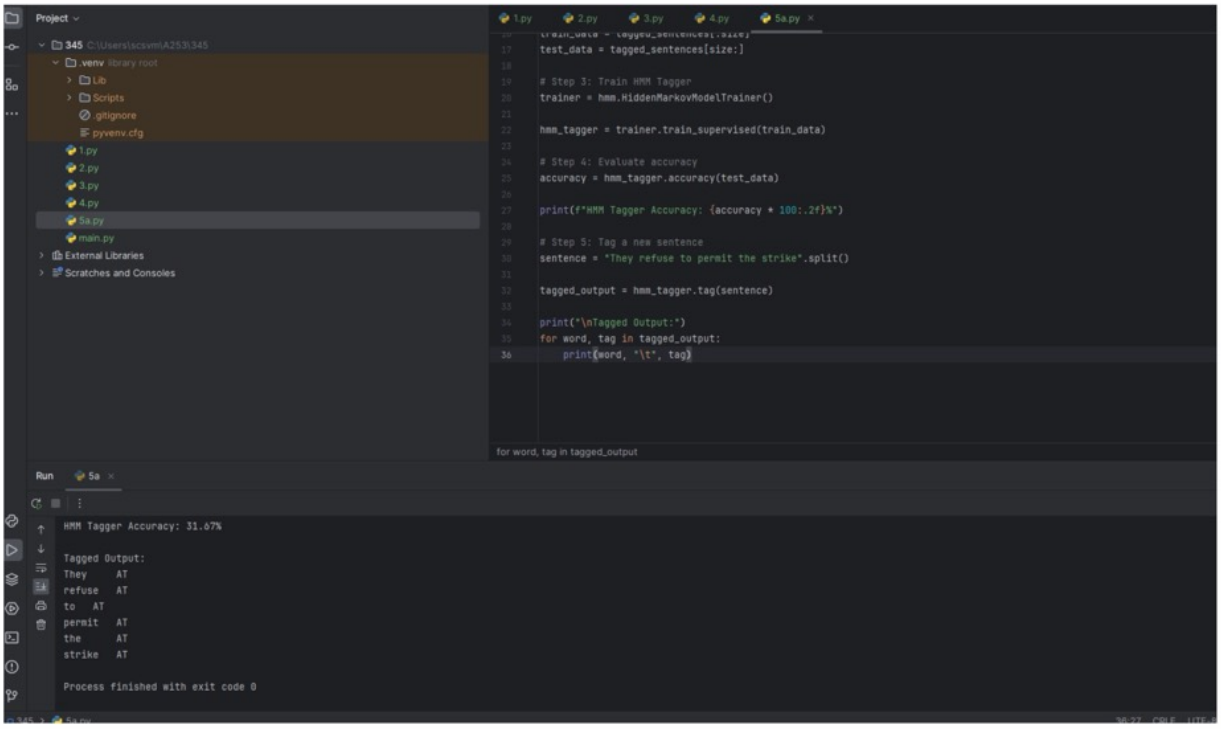


Result: The Brill Tagger successfully learns correction rules from training data and assigns accurate PoS tags to new sentences.

Experiment 5: Issues in PoS Tagging & Tagging using Hidden Markov Model (HMM)

Aim : To study the major issues involved in Part-of-Speech tagging and to implement PoS tagging using the Hidden Markov Model (HMM) with the Viterbi algorithm.

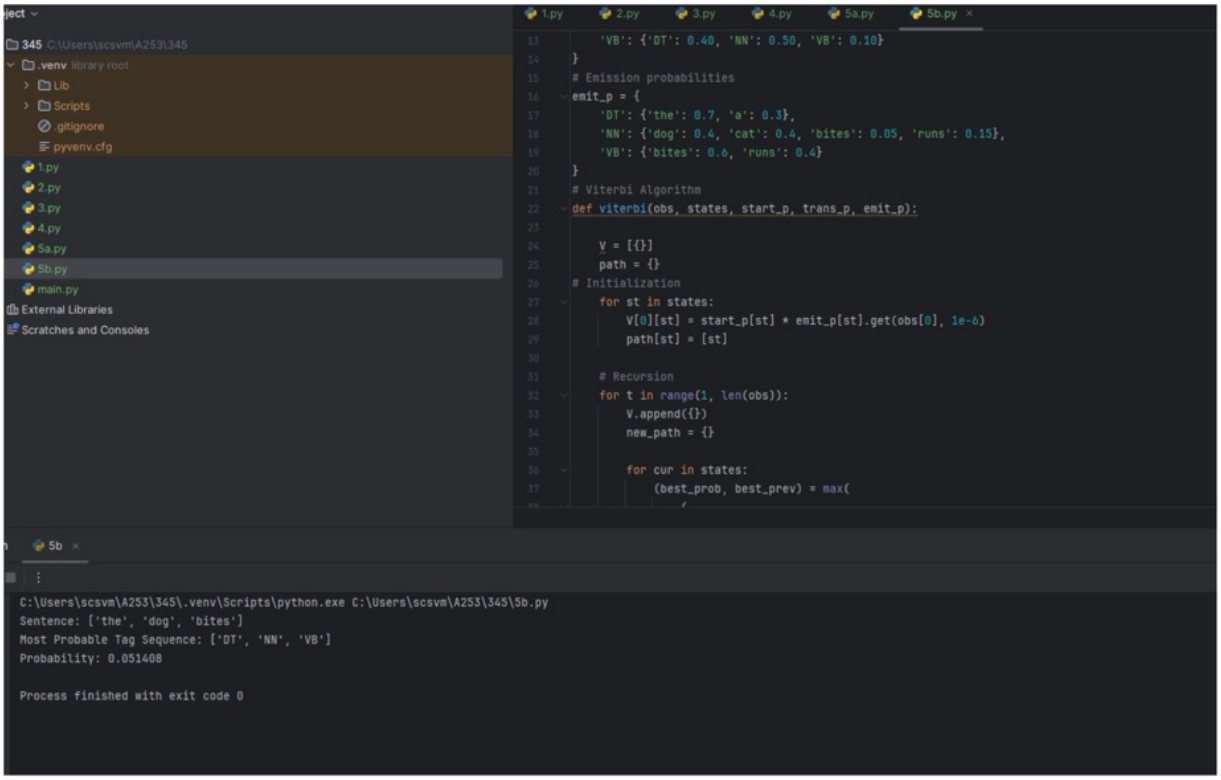
Program (a): Using NLTK's Built-in HMM Tagger



Result:

The HMM-based PoS tagger successfully learns word-tag probabilities from the training corpus and assigns appropriate PoS tags to the input sentence.

Program (b): Viterbi Algorithm Implemented from First Principles



Result:

The program successfully implements the Hidden Markov Model (HMM) Viterbi Algorithm to find the most probable sequence of PoS tags for the given sentence.

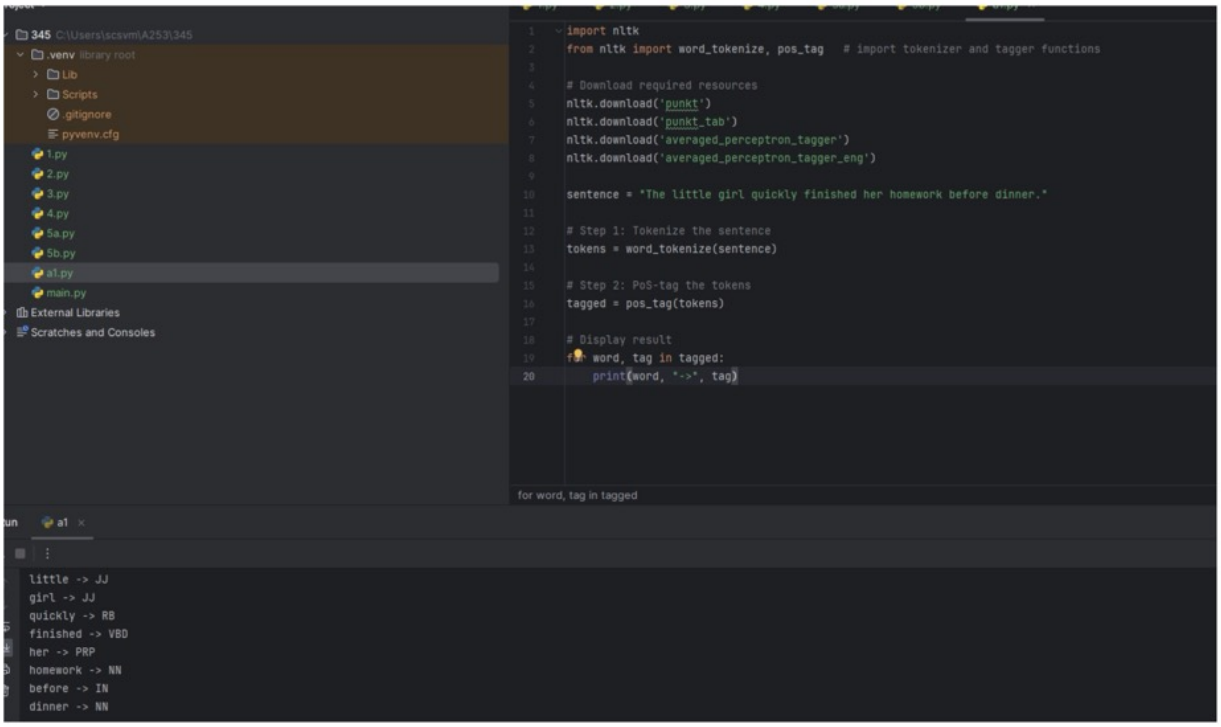
Lab Activity 1: Exploring Word Classes and Basic PoS Tagging

Aim: To perform Part-of-Speech (PoS) tagging on a given sentence using NLTK by tokenizing the sentence and assigning appropriate grammatical tags to each word.

Algorithm:

1. Start
2. Import NLTK tokenizer and PoS tagger.
3. Download required NLTK resources.
4. Input the sentence.
5. Tokenize the sentence using `word_tokenize()`.
6. Apply `pos_tag()` to get PoS tags.
7. Display word and corresponding tag.
8. Stop

Program:



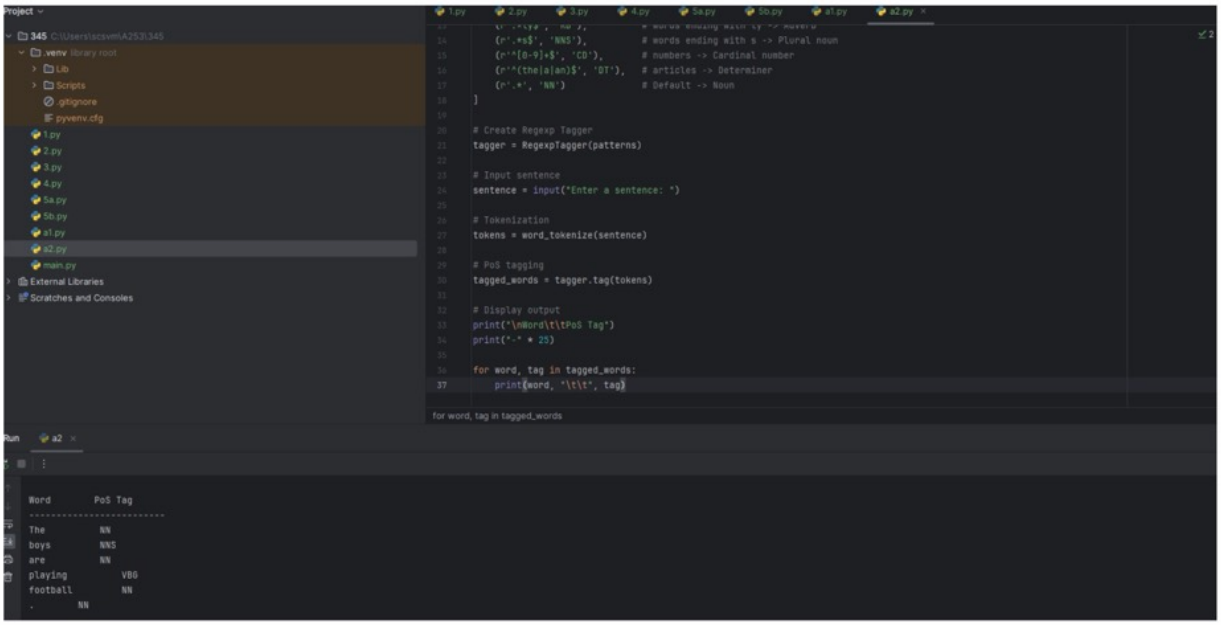
Lab Activity 2: Building a Rule-Based PoS Tagger

Aim: To perform Rule-Based Part-of-Speech (PoS) tagging using NLTK's RegexpTagger by applying regular expression patterns to classify words into their appropriate grammatical categories.

Algorithm:

1. Start
2. Import `word_tokenize()` and `RegexpTagger`.
3. Define regular expression patterns with PoS tags.
4. Input and tokenize the sentence.
5. Apply regex rules to assign tags.
6. Display words with their PoS tags.
7. Stop

Program:



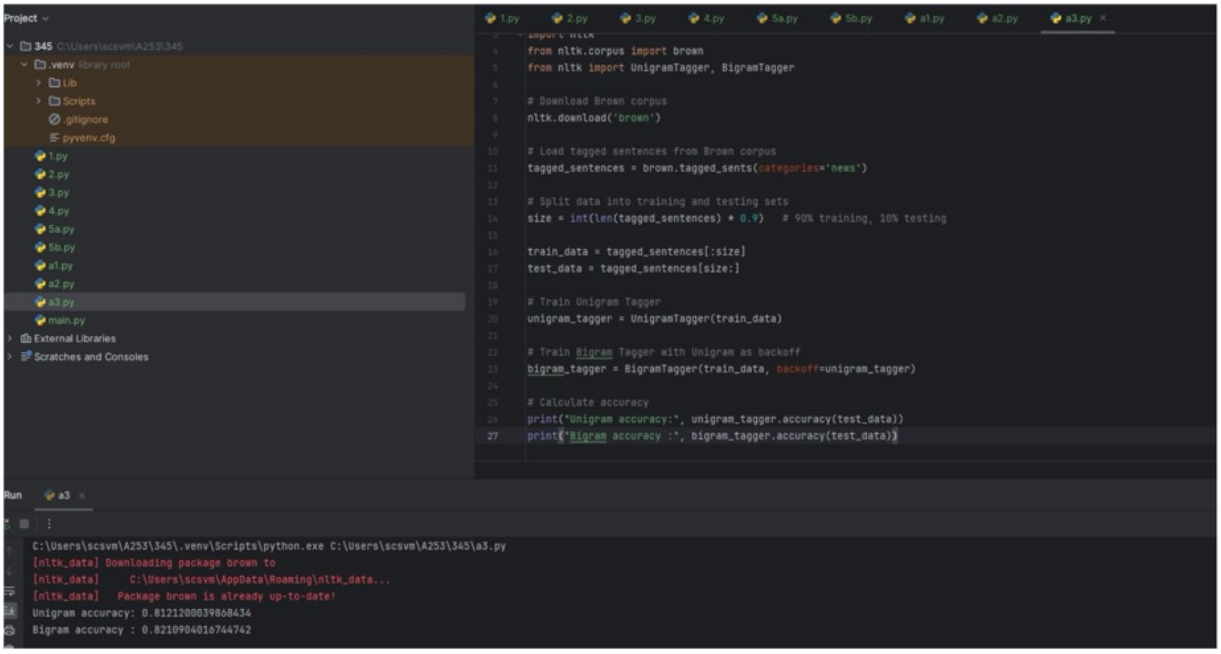
Lab Activity 3: Stochastic PoS Tagging and Accuracy Evaluation

Aim: To implement Unigram and Bigram Part-of-Speech (PoS) tagging using the Brown corpus in NLTK and compare their tagging accuracy.

Algorithm :

1. Import Brown corpus and required NLTK taggers.
2. Load tagged sentences from the Brown corpus.
3. Split the dataset into training and testing data.
4. Train a Unigram Tagger using the training data.
5. Train a Bigram Tagger with Unigram Tagger as a backoff.
6. Test both taggers using test data.
7. Display the accuracy of Unigram and Bigram taggers.

Program:



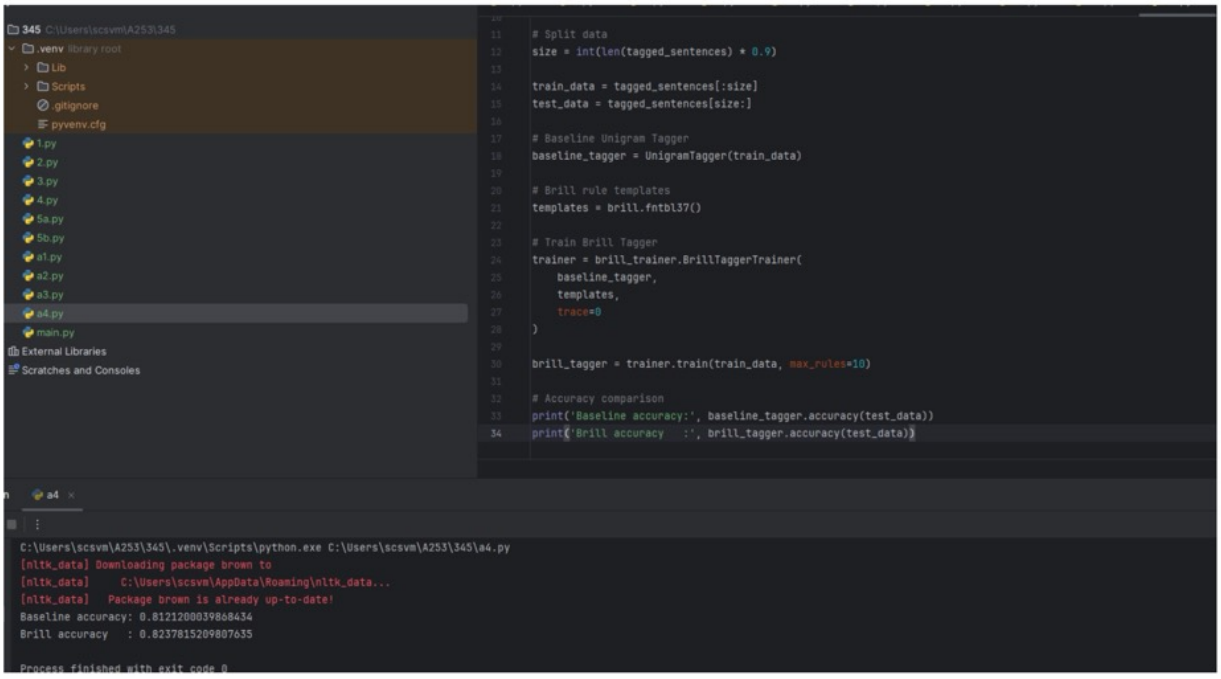
Lab Activity 4: Transformation-Based (Brill) Tagging

Aim: To implement Brill Transformation-Based PoS Tagging using NLTK and improve the accuracy of a baseline tagger.

Algorithm:

1.Import Brill tagger and trainer modules.
2.Load training and testing tagged data.
3.Create a baseline Unigram Tagger.
4.Define Brill rule templates.
5.Train the Brill Tagger using transformation rules.
6.Test baseline and Brill tagger accuracy.
7.Display the accuracy results.

program:



Lab Activity 5: Investigating Issues in PoS Tagging + HMM/Viterbi Tagging

Part A — Tasks: Issues in PoS Tagging

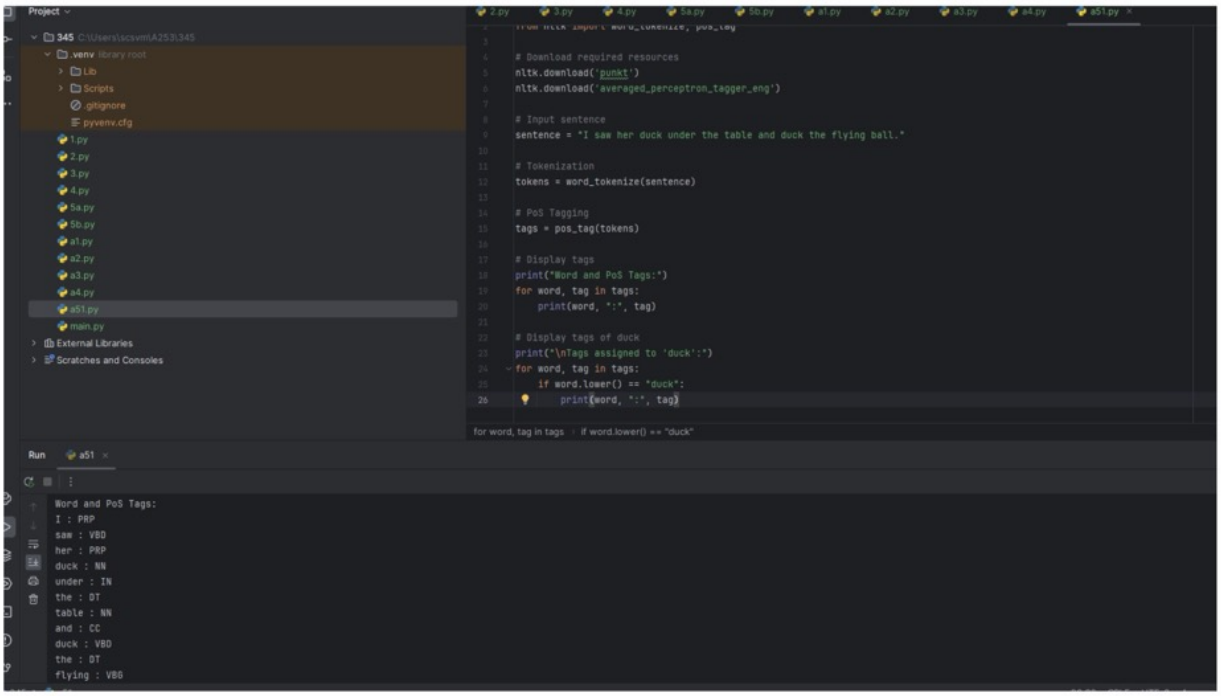
1. Run any tagger from Activities 1–4 on the sentence: "I saw her duck under the table and duck the flying ball." Record the tags assigned to both instances of "duck".

Aim: To apply a Part-of-Speech (PoS) tagger on an ambiguous sentence and observe the different tags assigned to the word "duck" based on its context.

Algorithm :

1. Import the required NLTK libraries.
2. Load a pre-trained PoS tagger.
3. Input the given sentence.
4. Tokenize the sentence into words.
5. Apply the PoS tagger.
6. Display the assigned tags and observe the tags for both occurrences of "duck".

program:



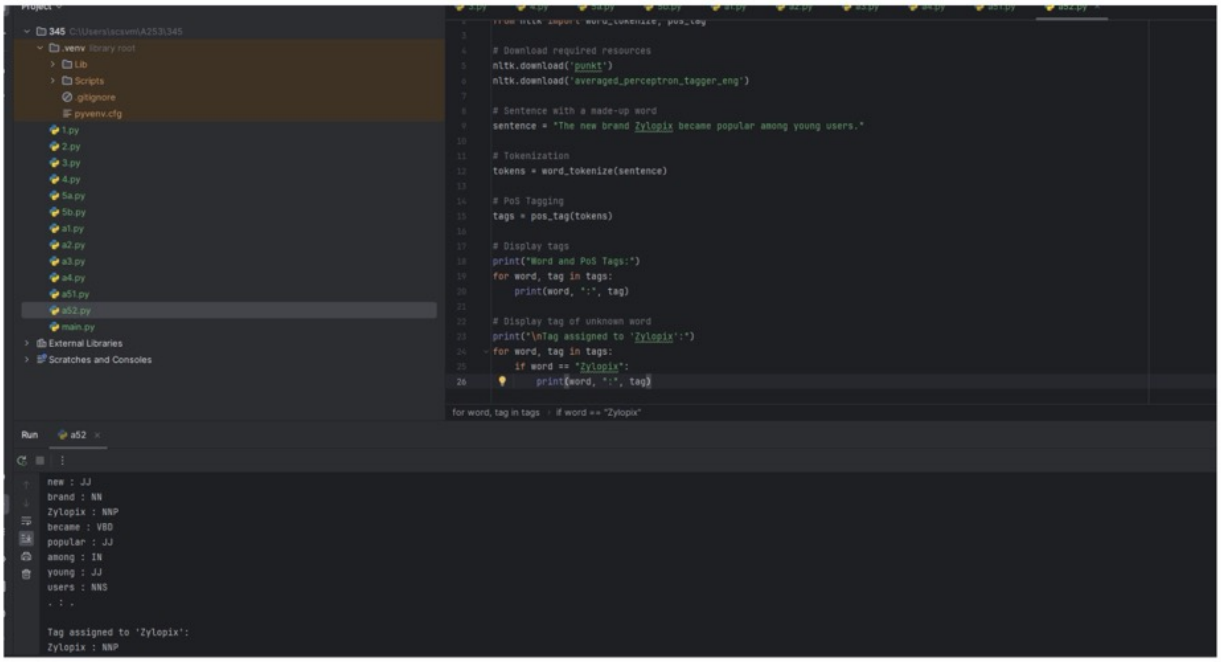
2. Create one made-up word (e.g., a brand name or slang term) that does not exist in the dictionary, insert it into a sentence, and tag the sentence. Record what tag the unknown word received and whether it is correct.

Aim: To test how a Part-of-Speech (PoS) tagger handles an unknown/made-up word and analyze the tag assigned to it.

Algorithm :

1. Import NLTK tokenizer and PoS tagger.
2. Create a sentence containing a made-up word.
3. Tokenize the sentence into individual words.
4. Apply PoS tagging to the tokens.
5. Display the assigned tags.
6. Observe the tag given to the unknown word and check its correctness.

Program:



3. Tag two sentences from a different domain, e.g., a tweet with abbreviations/emojis and a line from a poem, using the Activity 3 tagger. Note any incorrect tags caused by the domain mismatch.

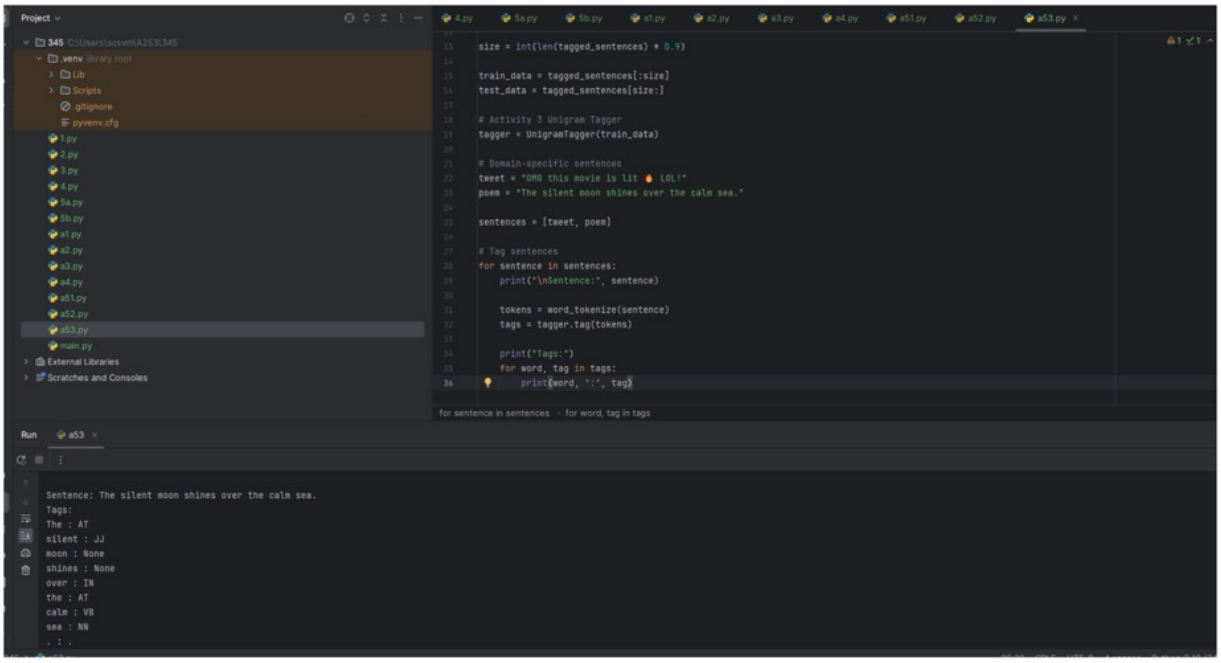
Aim:

To evaluate the performance of an Activity 3 PoS tagger on sentences from different domains such as social media text and poetry, and identify incorrect tags caused by domain differences.

Algorithm :

1. Import NLTK tokenizer and PoS tagging modules.
2. Load the trained tagger from Activity 3.
3. Create sentences from different domains (tweet and poem).
4. Tokenize the sentences.
5. Apply the PoS tagger.
6. Display the assigned tags.
7. Analyze incorrect tags due to domain mismatch.

Program:



4. For each of the three cases above, name which 'Issue in PoS tagging' (ambiguity / unknown words / domain dependence) it illustrates.

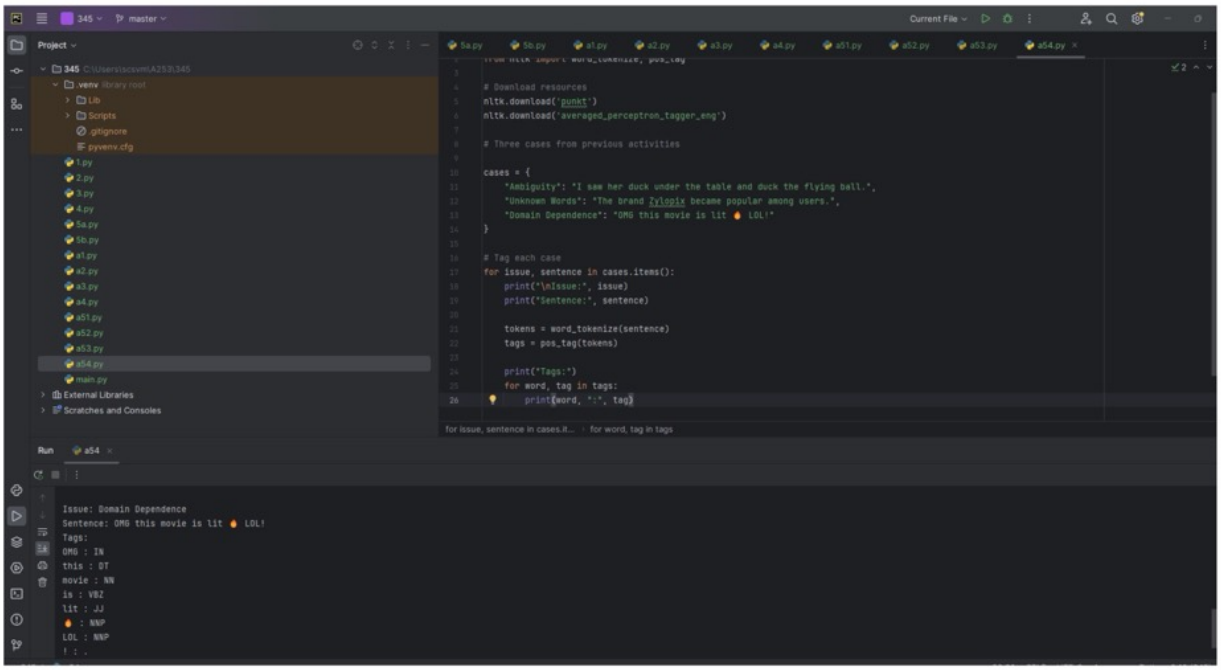
Aim:

To identify and classify common issues in Part-of-Speech (PoS) tagging such as ambiguity, unknown words, and domain dependence using examples from previous activities.

Algorithm:

1. Import NLTK libraries.
2. Create examples representing ambiguity, unknown words, and domain-specific text.
3. Apply PoS tagging to each example.
4. Observe the assigned tags.
5. Identify the PoS tagging issue illustrated by each case.
6. Display the issue and explanation.

Program:



Part B — Tasks: HMM and Viterbi by Hand

Aim:

To implement the Hidden Markov Model (HMM) Viterbi algorithm for Part-of-Speech tagging and find the most probable tag sequence for the sentence "a cat runs".

Algorithm:

1.Define HMM parameters: start probabilities, transition probabilities, and emission probabilities.

2.Initialize Viterbi table using the first word probability.

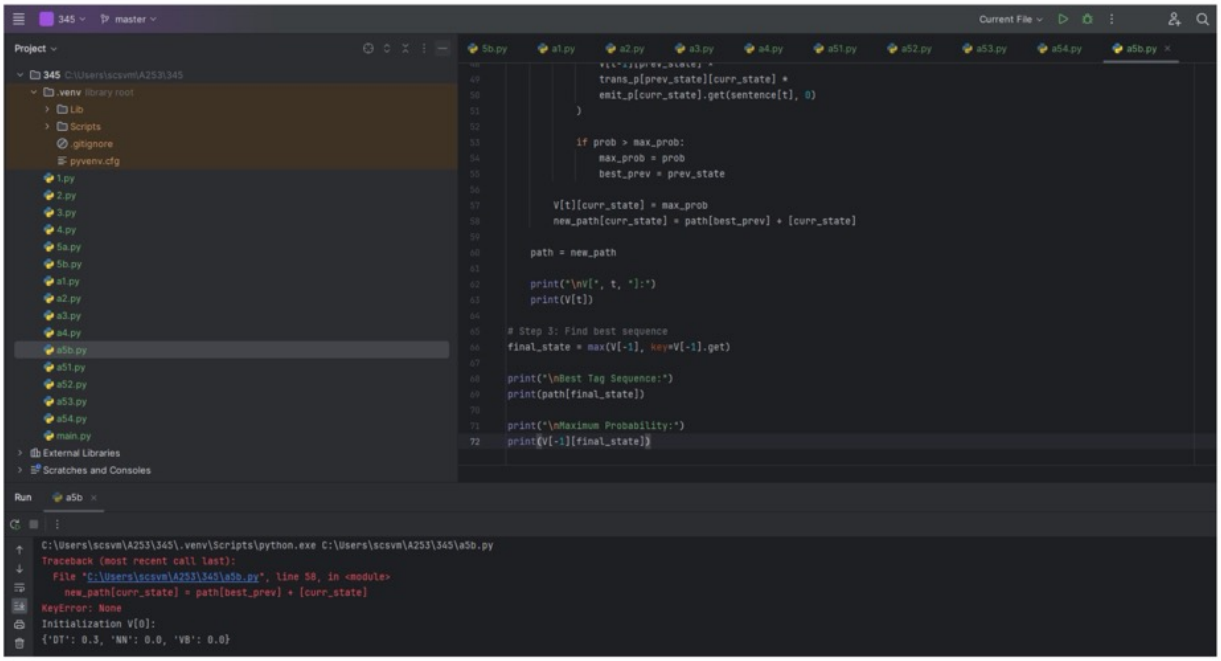
3.Apply recursion to calculate maximum probabilities for each state at each word.

4.Store the previous state that gives maximum probability.

5.Trace back from the final state to find the best tag sequence.

6.Display the Viterbi probabilities and final POS tags.

program:



Natural Language Processing

Scenario-Based Questions

Unit 1: Word Analysis

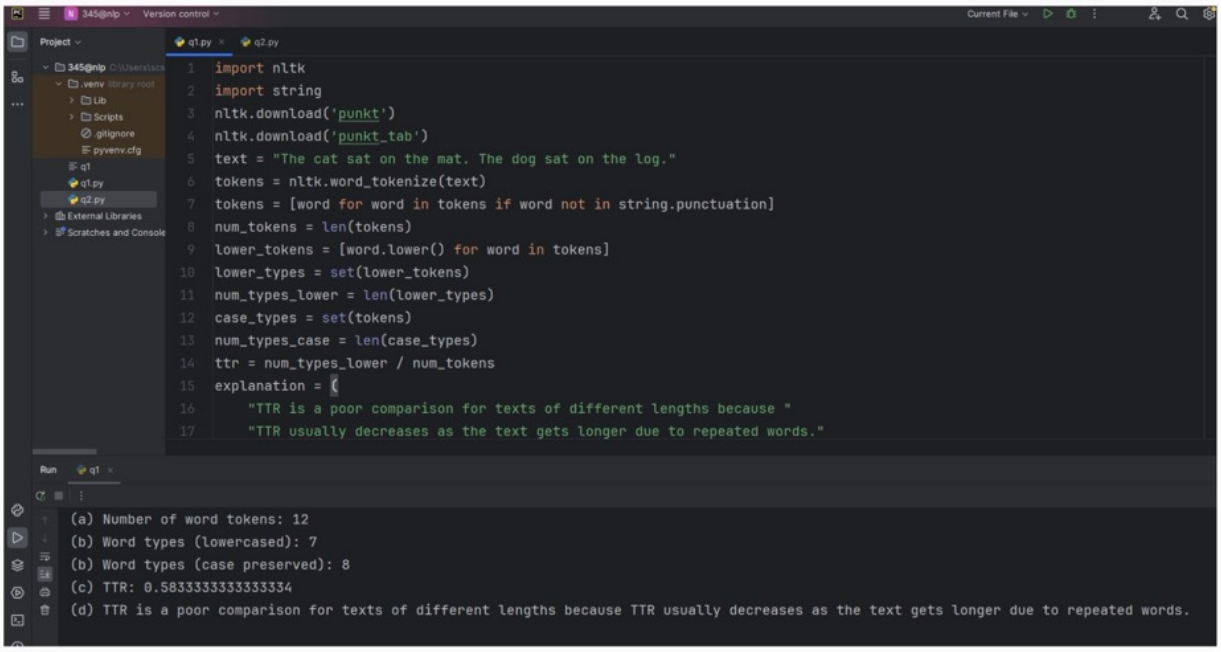
Q1. Vocabulary Statistics for a News Startup

Aim: To calculate word tokens, word types, and Type-Token Ratio (TTR) of a given text using NLTK.

Algorithm:

1. Import `nltk` and `string`.
2. Tokenize the given text.
3. Remove punctuation.
4. Count total word tokens.
5. Convert words to lowercase and find unique word types.
6. Find unique word types while preserving case.
7. Calculate TTR = Number of types / Number of tokens.

Program:



Result: the program was executed successfully

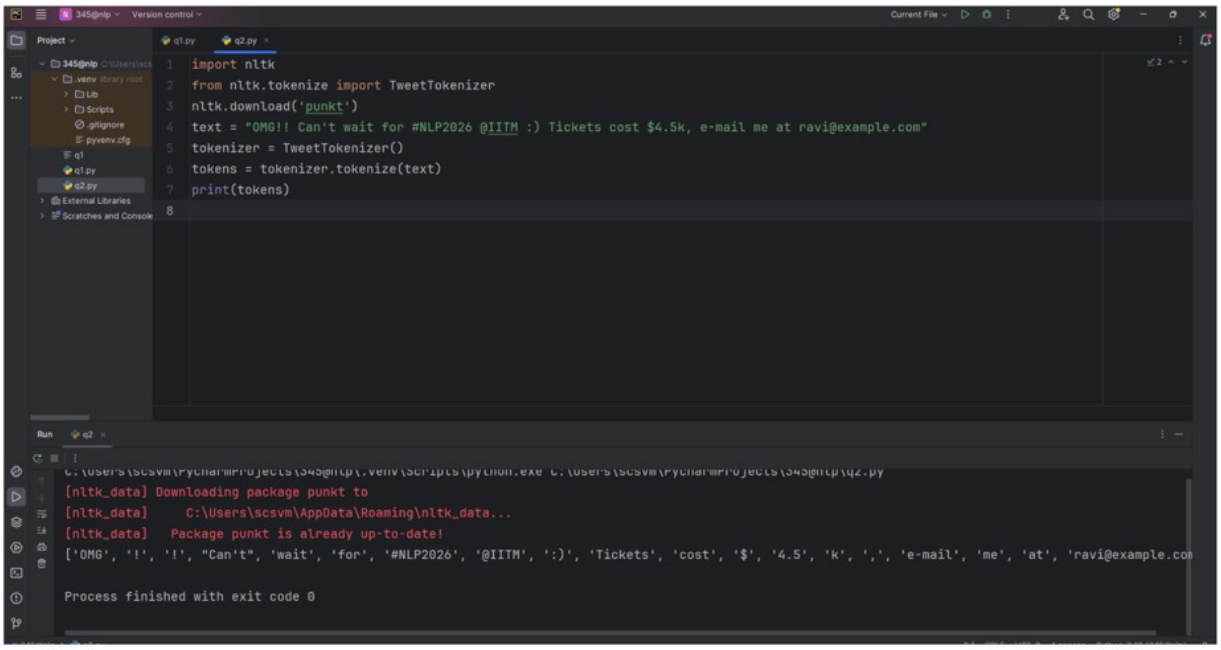
Q2. Tokenizing a Social-Media Post

Aim: To tokenize a social-media post effectively by handling hashtags, mentions, emoticons, currency, e-mails, and punctuation.

Algorithm:

1. Read the social-media text.
2. Identify special tokens such as `#hashtag`, `@mention`, emoticons, currency, and e-mail.
3. Keep these as meaningful tokens.
4. Separate punctuation where appropriate.
5. Apply Penn Treebank convention for contractions.

Program:



Result: the program was executed successfully

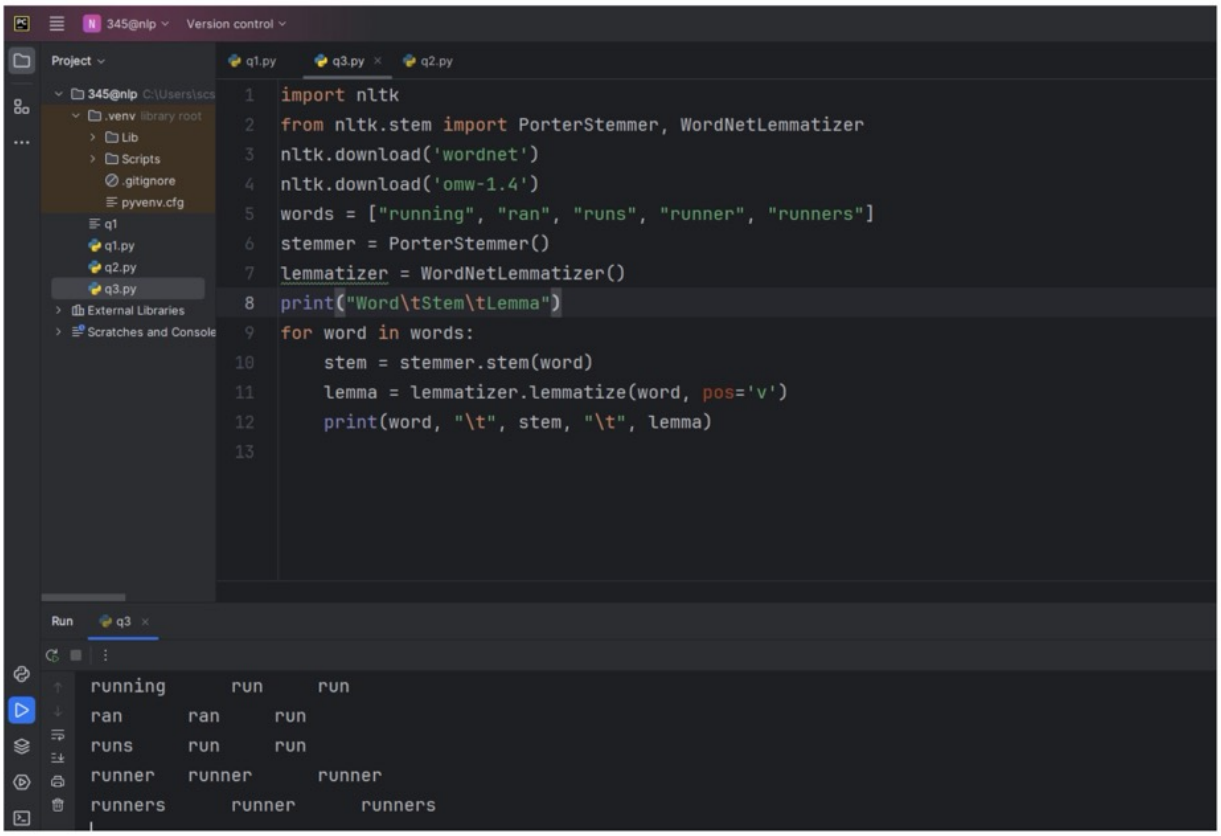
Q3. Query Normalisation in a Search Engine

Aim: To compare stemming and lemmatization for normalizing words in a search engine.

Algorithm:

1. Store the words `running, ran, runs, runner, runners`.
2. Apply the Porter Stemmer to each word.
3. Find the lemma of each word using WordNet.
4. Compare stemming and lemmatization.
5. Determine their effect on recall and precision.

Program:



Result: the program was executed successfully

Q4. Zipf's Law in a Customer-Review Corpus

Aim:

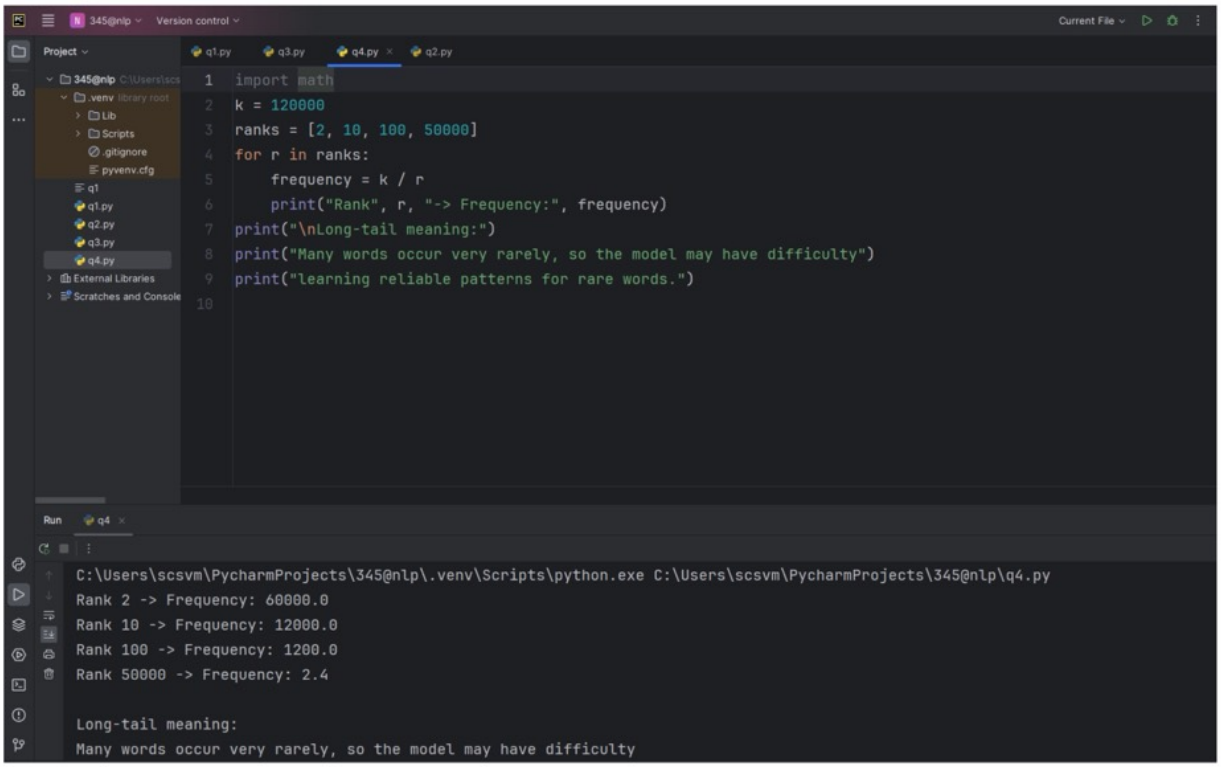
To apply Zipf's Law to estimate word frequencies at different ranks in a customer-review corpus.

Algorithm:

1. Take the frequency of the most frequent word (`the`) as 120,000.
2. Since rank 1 has frequency 120,000, set `k = 120,000`.
3. Use Zipf's formula:

4. f=kr
5. Calculate frequencies for ranks 2, 10, 100, and 50,000.
6. Interpret the long-tail effect on the model.

Program:



Result: the program was executed successfully

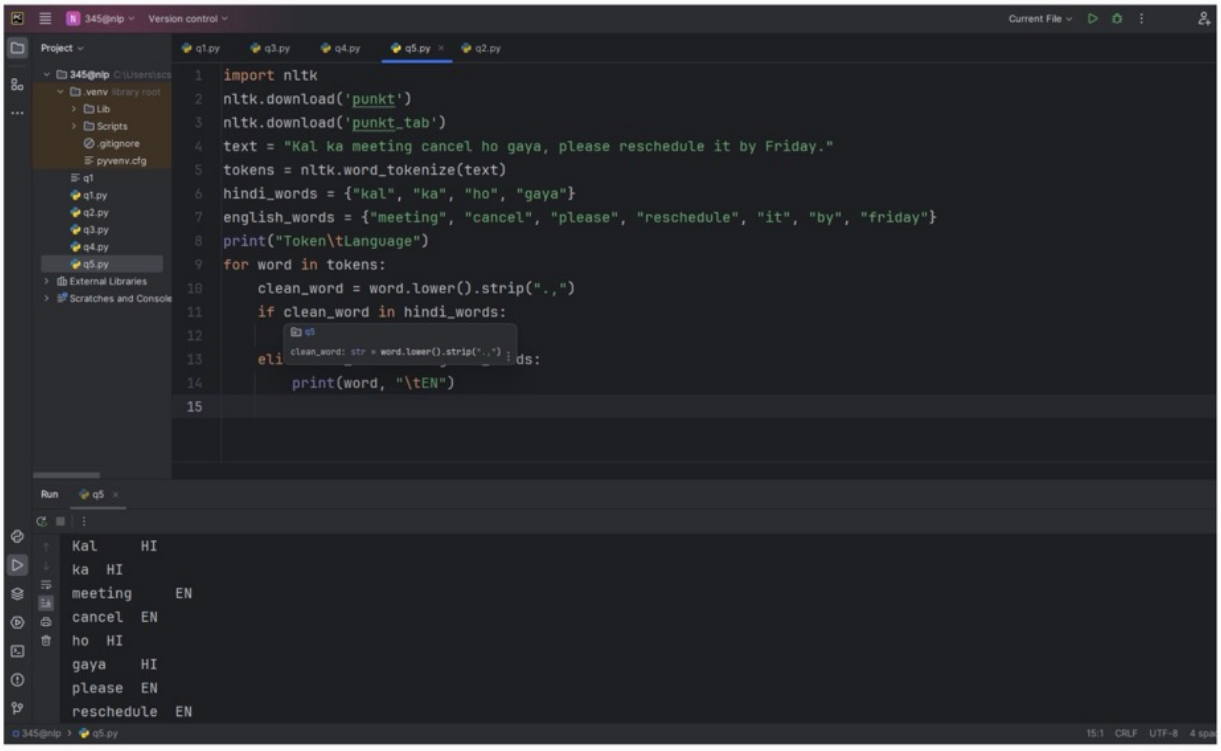
Q5. Code-Mixed Customer Chat

Aim: To identify Hindi-English code-mixed tokens and design a preprocessing approach for mixed-language customer chats.

Algorithm:

1. Store the code-mixed sentence.
2. Tokenize the sentence into individual words.
3. Assign HI to Romanised Hindi words and EN to English words.
4. Identify problems caused by Romanised Hindi and code-mixing.
5. Apply language identification, normalization, and language-specific NLP processing.

Program:



Result: the program was executed successfully

Unit 2: Word Generation

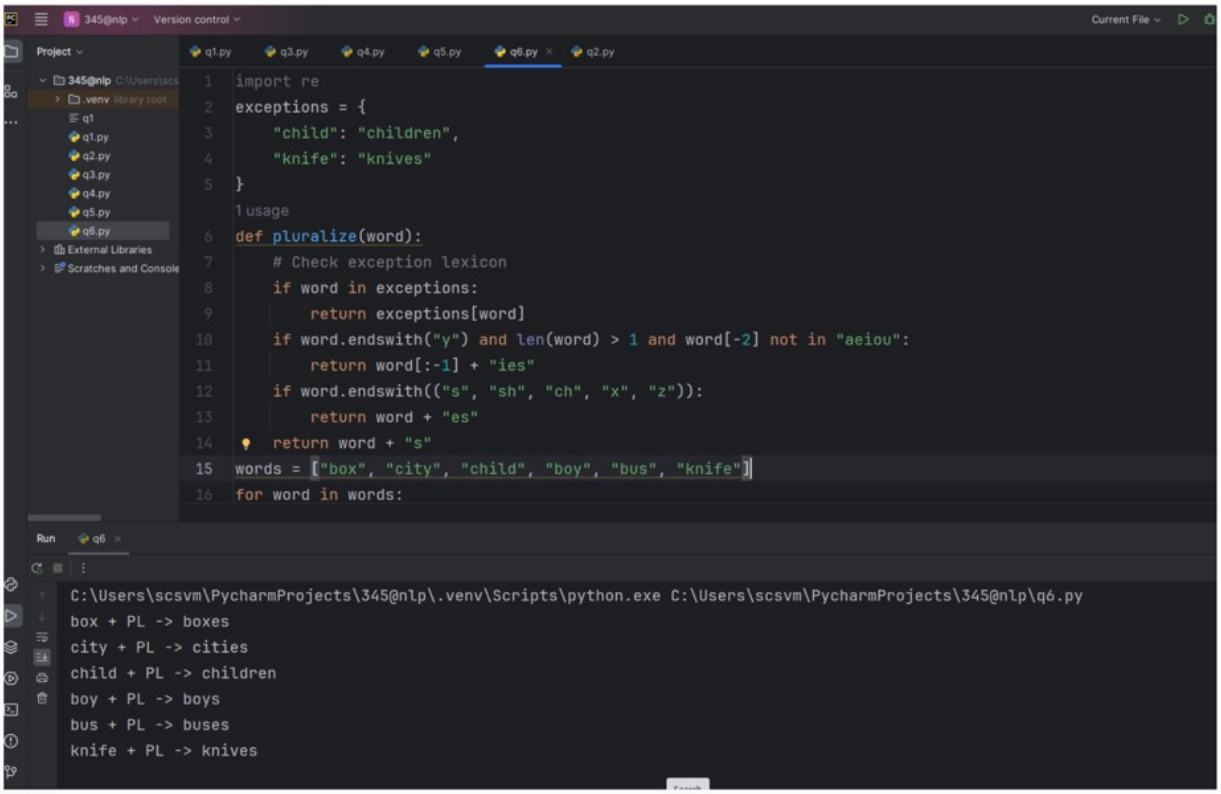
Q6. Plural Generator for an E-Commerce Catalogue

Aim: To generate plural forms of English nouns using finite-state transducer (FST)-style rules and identify irregular forms.

Algorithm:

1. Take the noun lemma with the feature +PL.
2. Apply general pluralization rules:
    - consonant + y → `ies`
    - s, sh, ch, x, z → `es`
    - otherwise → `s`
3. Check a lexicon for irregular nouns such as `child → children` and `knife → knives`.
4. Print the generated plural forms.

Program:



Result: the program was executed successfully

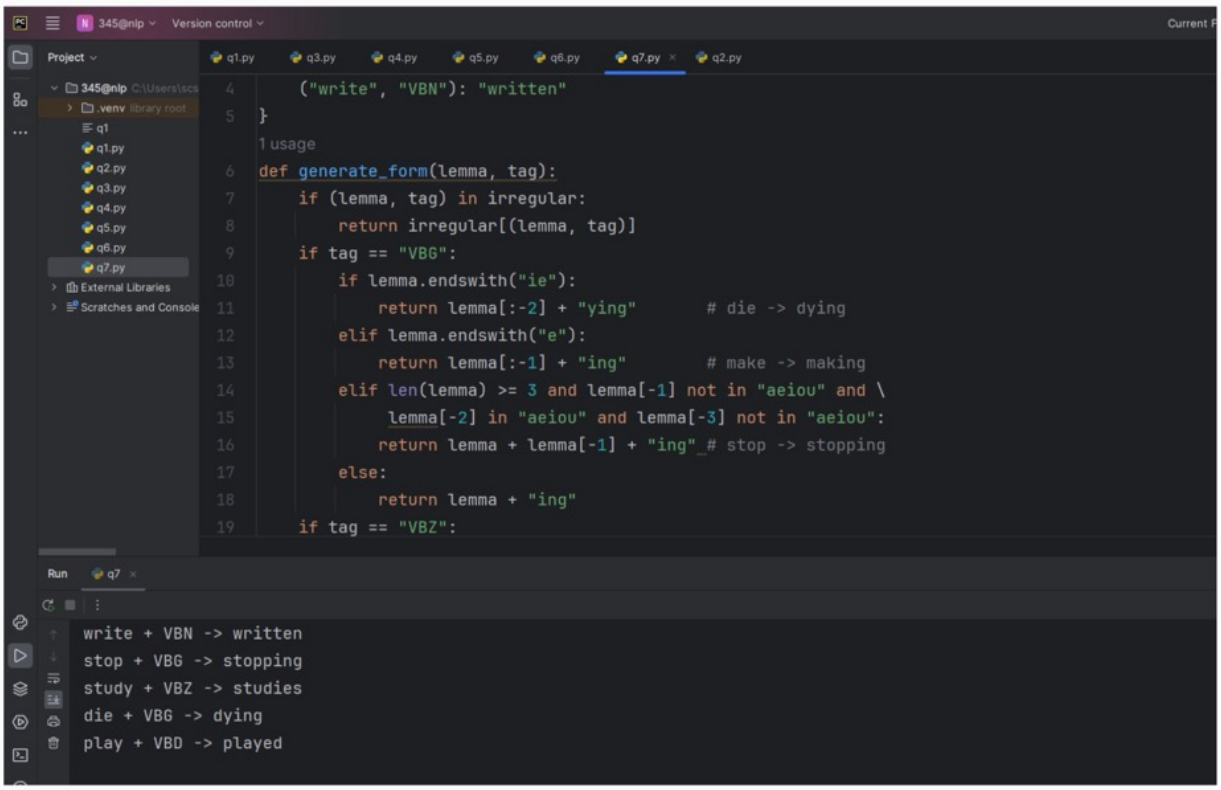
Q7. Verb Forms for a Grammar-Correction Tool

Aim: To generate correct English verb forms from a lemma and morphological tag using spelling rules and an irregular-forms table.

Algorithm:

1. Store irregular verb forms in a dictionary.
2. Read the lemma and morphological tag.
3. Check the irregular-form table first.
4. If no irregular form exists, apply the appropriate spelling rule.
5. Generate and display the surface form.

Program:



Result: the program was executed successfully

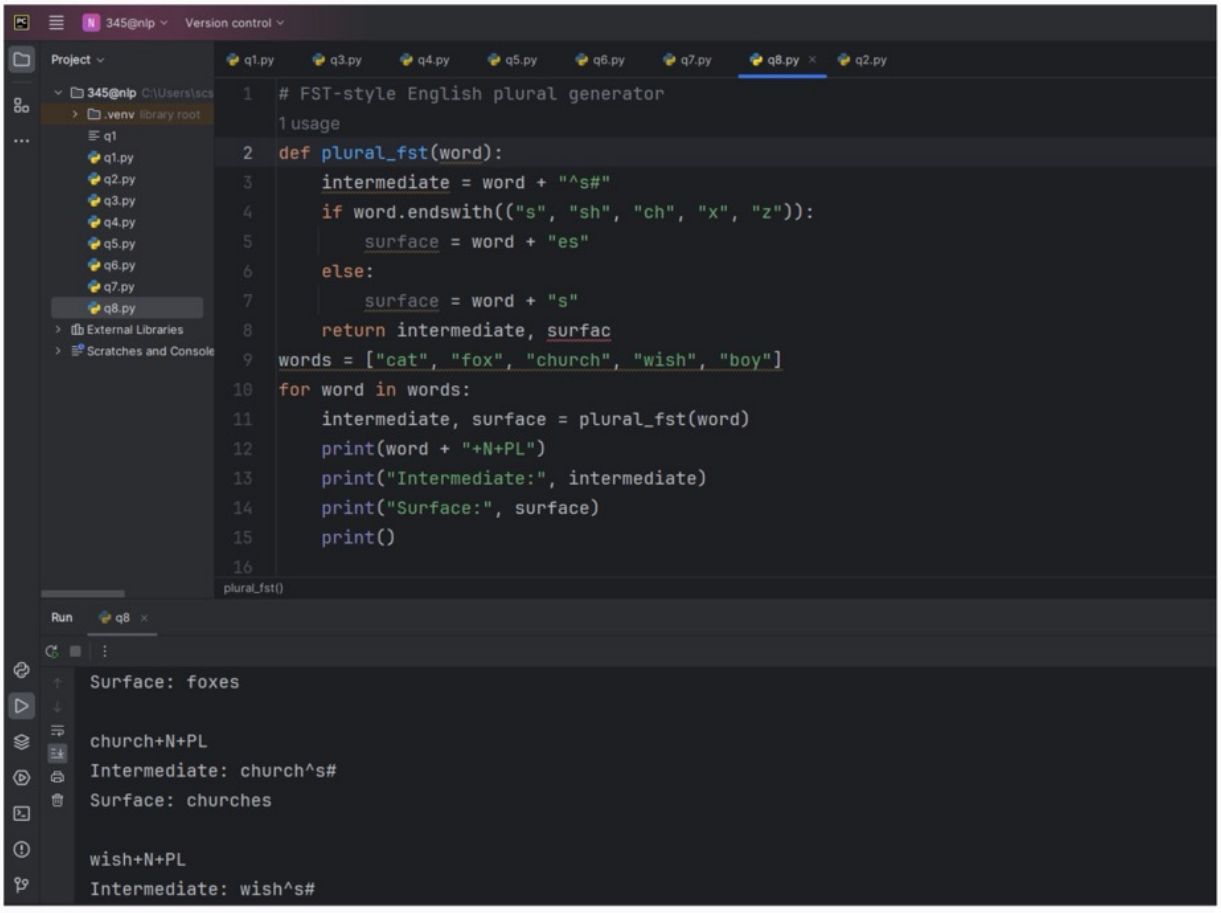
Q8. Finite-State Transducer for English Plurals

Aim: To implement a simple Finite-State Transducer (FST) for generating English plural forms from lexical forms.

Algorithm:

1. Take a noun with +N+PL.
2. Convert it to an intermediate form using ^s#.
3. Insert e before s when the noun ends in s, sh, ch, x, z.
4. Remove morphological boundary symbols.
5. Produce the surface plural form.

Program:



Result: the program was executed successfully

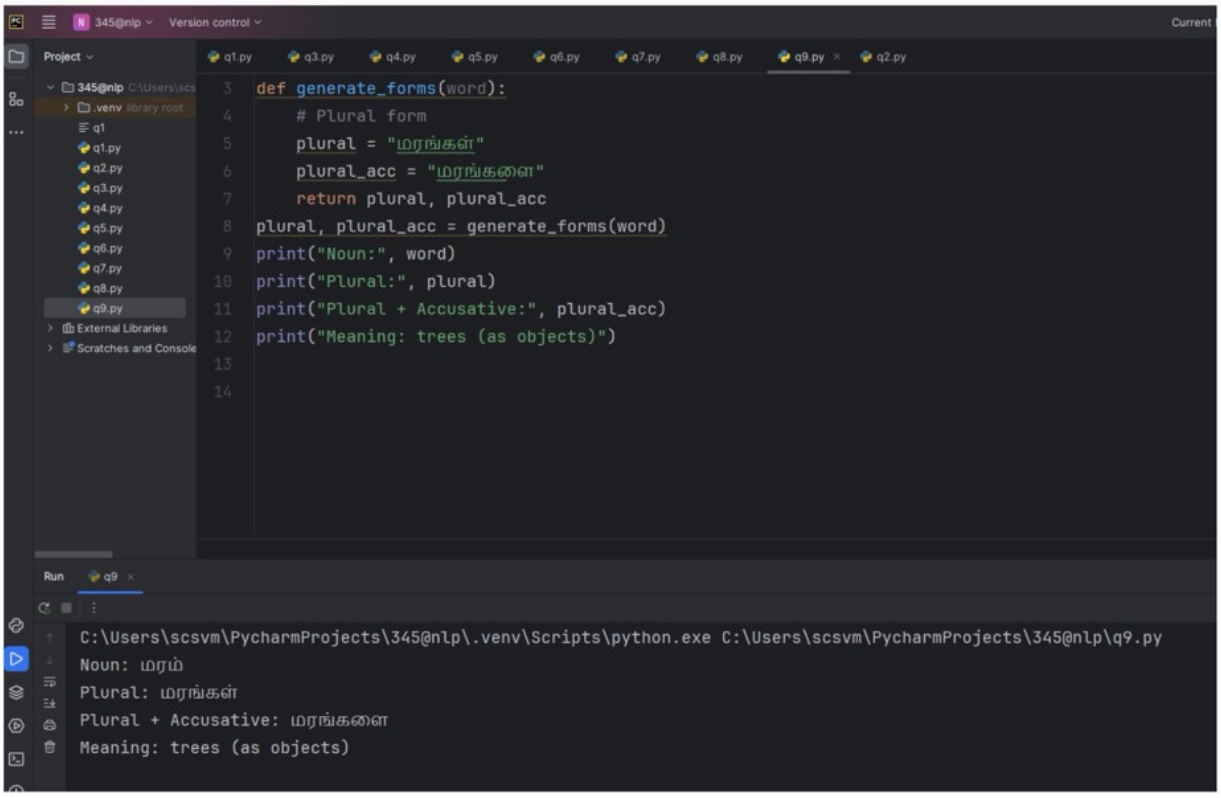
Q9. Tamil Noun Forms for an e-Governance Chatbot

Aim: To generate Tamil noun forms by applying plural and case suffixes with appropriate stem changes and sandhi rules.

Algorithm:

1. Store the noun stem
2. Apply the plural rule to form
3. Apply the accusative suffix –
4. Apply the required stem alternation/sandhi.
5. Generate the combined

Program:



Result: the program was executed successfully

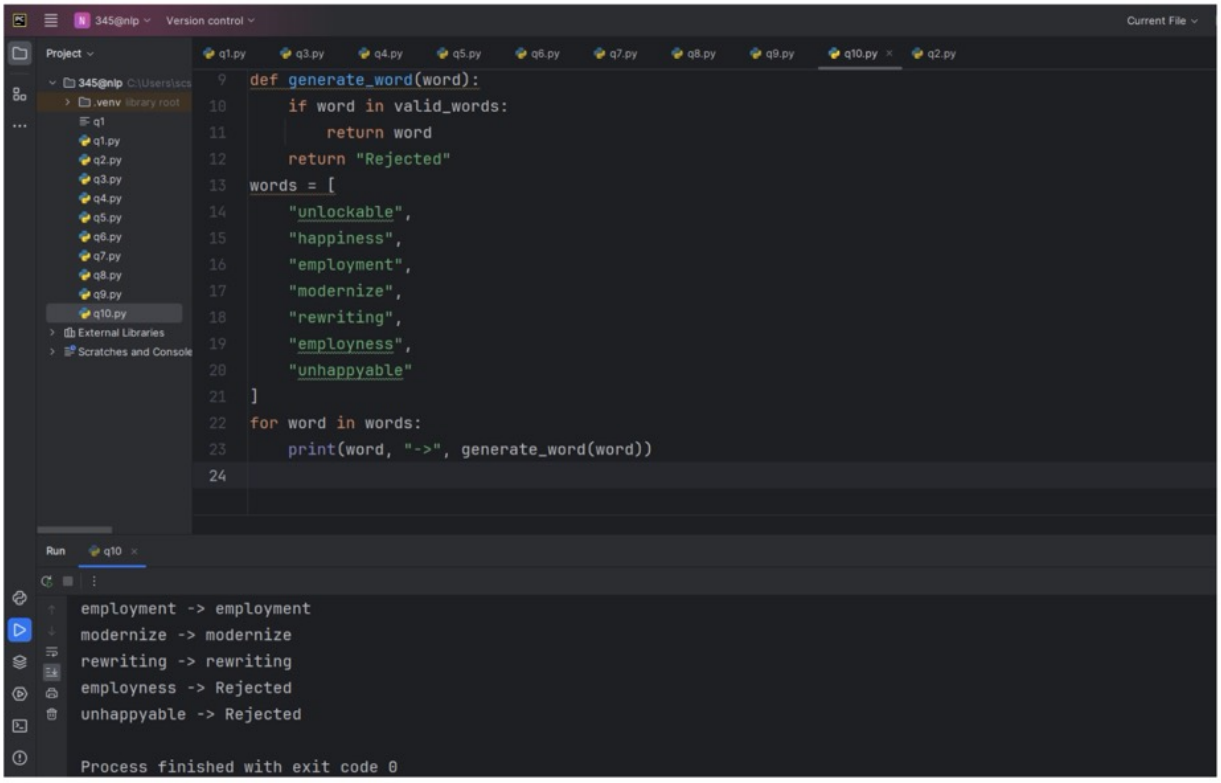
Q10. Derivational Word Generator for Brand Names

Aim: To generate valid derived words using prefixes and suffixes while rejecting invalid or inappropriate affix combinations.

Algorithm:

1. Store valid derivational patterns and exceptions.
2. Combine the given root words with their affixes.
3. Check whether the generated word is present in a valid-word dictionary.
4. Reject combinations that violate morphological constraints.
5. Handle ambiguous words separately.

Program:



Result: the program was executed successfully

Unit 3: Morphology

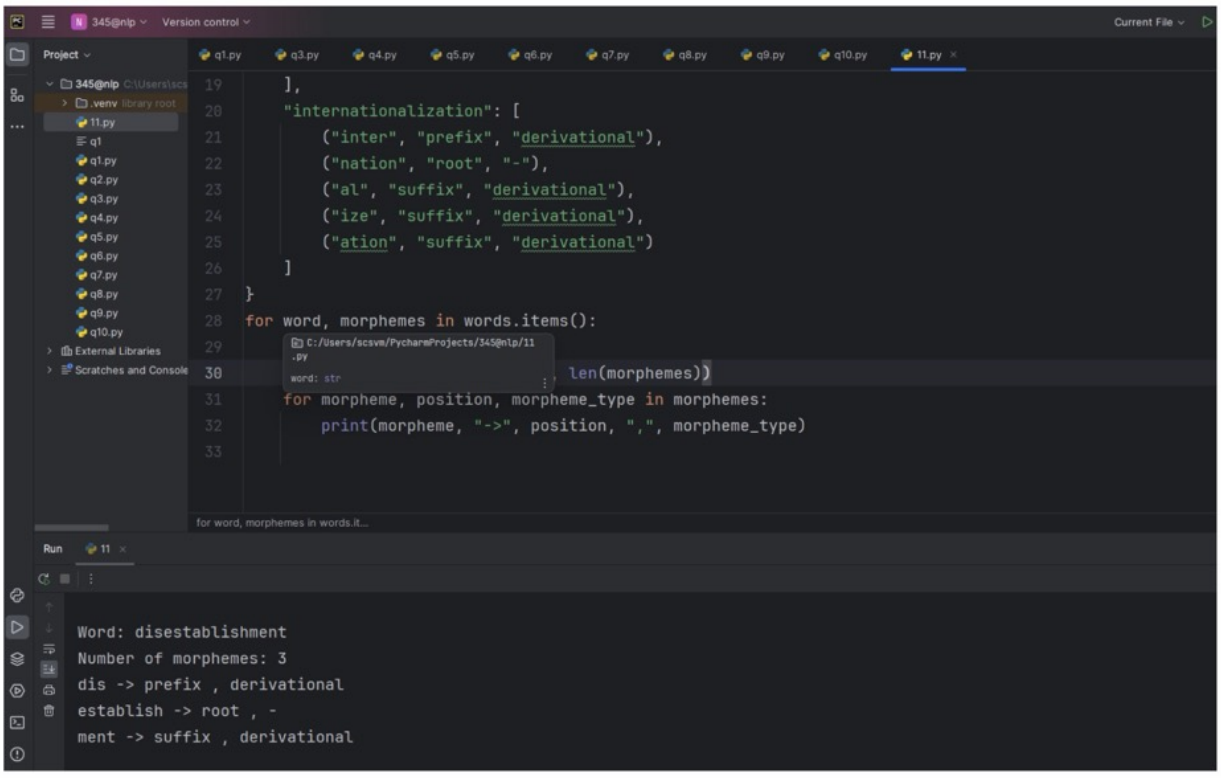
Aim: To segment words into morphemes and identify prefixes, roots, and suffixes as derivational or inflectional.

Algorithm:

1. Store the given words.
2. Split each word into its prefix, root, and suffixes.
3. Label each morpheme according to its function.
4. Count the number of morphemes.
5. Identify whether each suffix is derivational or inflectional

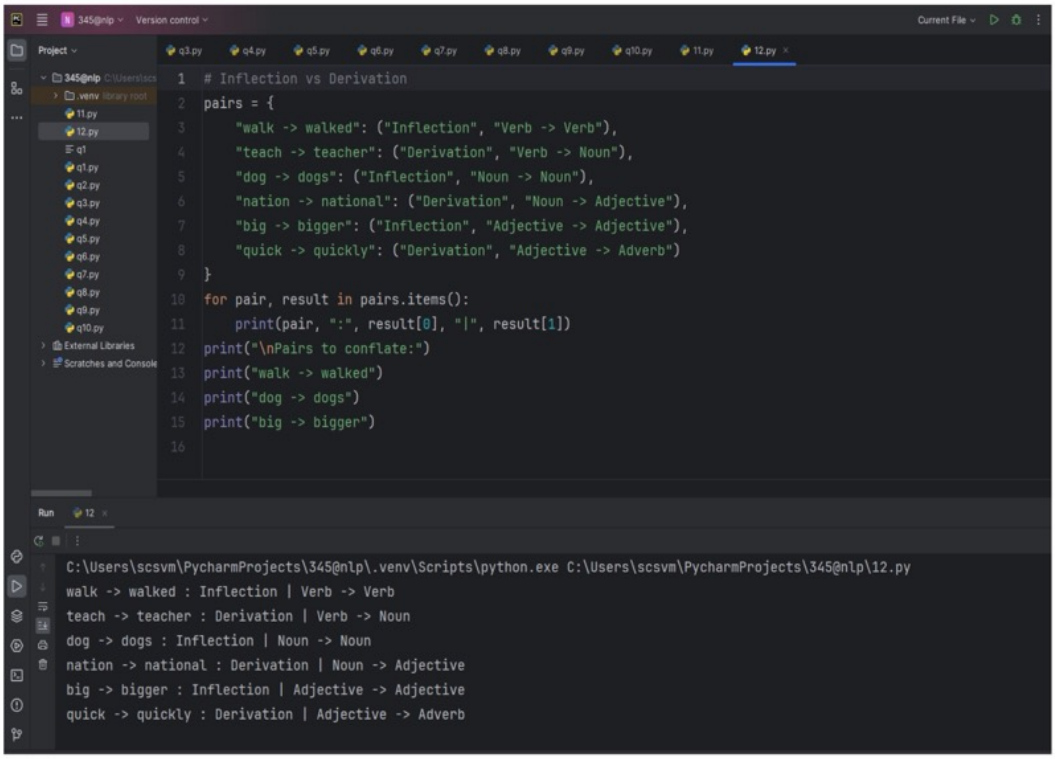
.

Program:



Result: the program was executed successfully

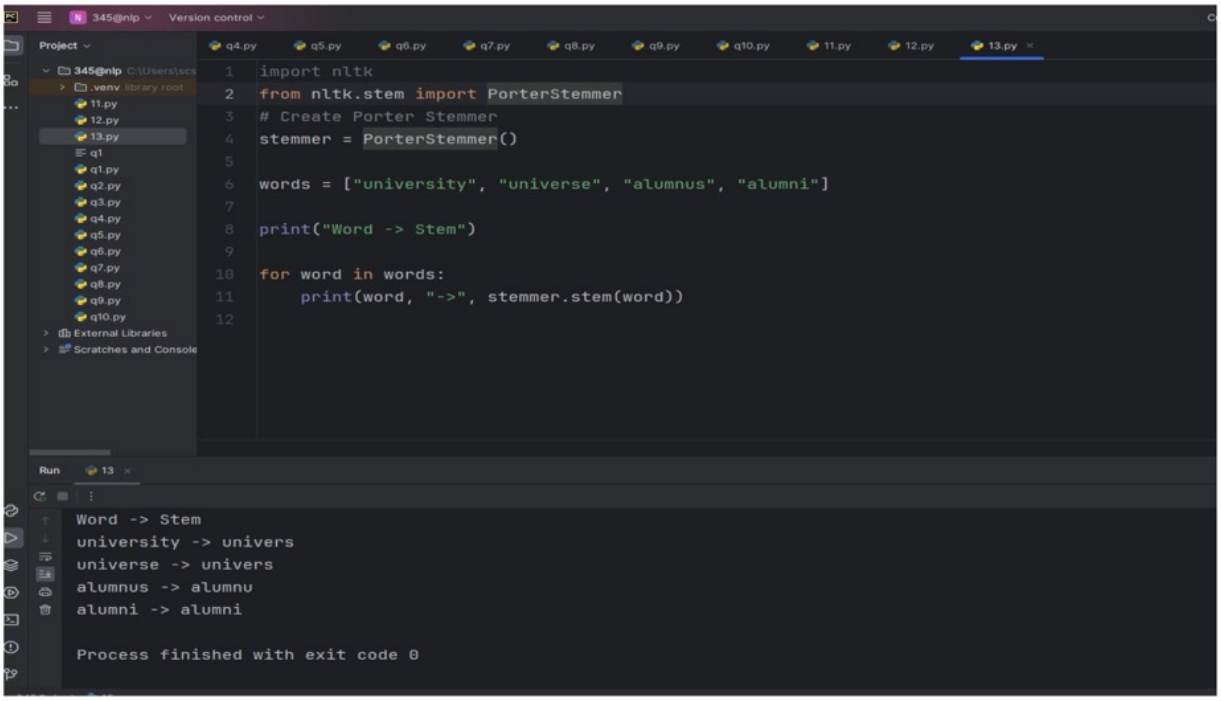
Q12. Inflection versus Derivation



```python
# Inflection vs Derivation
pairs = {
    "walk -> walked": ("Inflection", "Verb -> Verb"),
    "teach -> teacher": ("Derivation", "Verb -> Noun"),
    "dog -> dogs": ("Inflection", "Noun -> Noun"),
    "nation -> national": ("Derivation", "Noun -> Adjective"),
    "big -> bigger": ("Inflection", "Adjective -> Adjective"),
    "quick -> quickly": ("Derivation", "Adjective -> Adverb")
}
for pair, result in pairs.items():
    print(pair, ":", result[0], "|", result[1])
print("\nPairs to conflate:")
print("walk -> walked")
print("dog -> dogs")
print("big -> bigger")
```

```
C:\Users\scsvm\PycharmProjects\345@nlp\.venv\Scripts\python.exe C:\Users\scsvm\PycharmProjects\345@nlp\12.py
walk -> walked : Inflection | Verb -> Verb
teach -> teacher : Derivation | Verb -> Noun
dog -> dogs : Inflection | Noun -> Noun
nation -> national : Derivation | Noun -> Adjective
big -> bigger : Inflection | Adjective -> Adjective
quick -> quickly : Derivation | Adjective -> Adverb
```

Q13. Stemmer Errors in Legal Search



```python
import nltk
from nltk.stem import PorterStemmer
# Create Porter Stemmer
stemmer = PorterStemmer()

words = ["university", "universe", "alumnus", "alumni"]

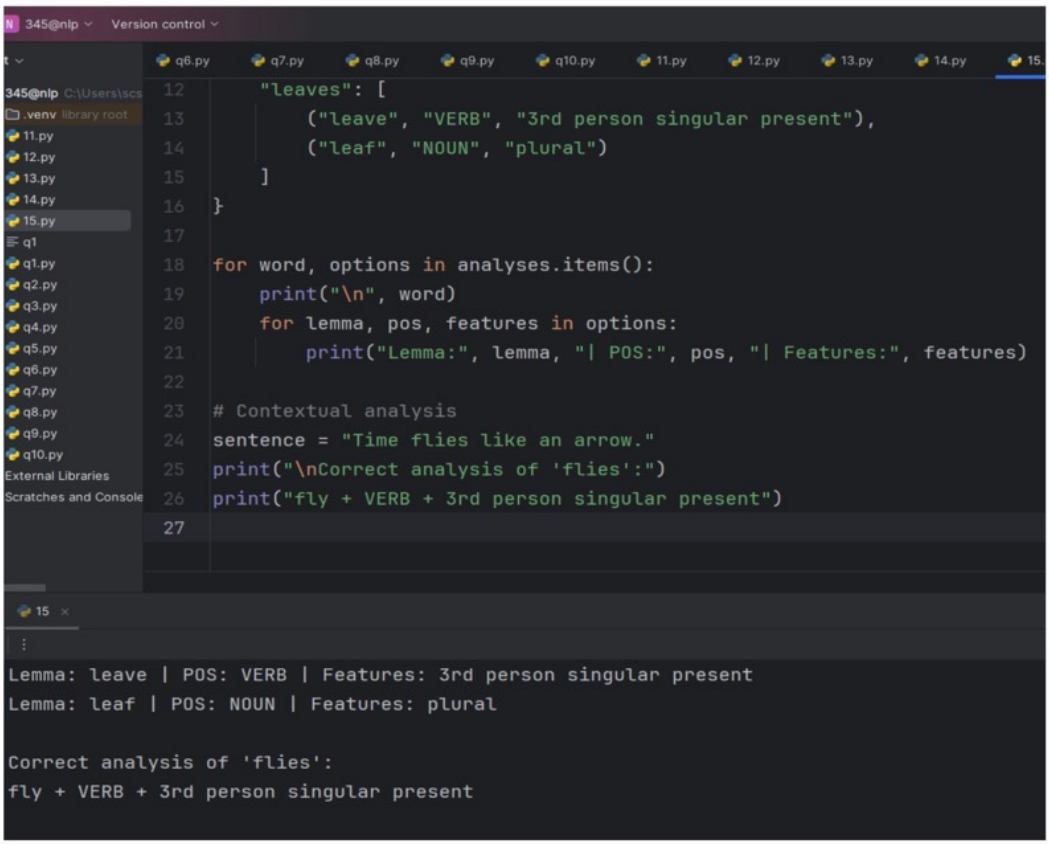
print("Word -> Stem")

for word in words:
    print(word, "->", stemmer.stem(word))
```

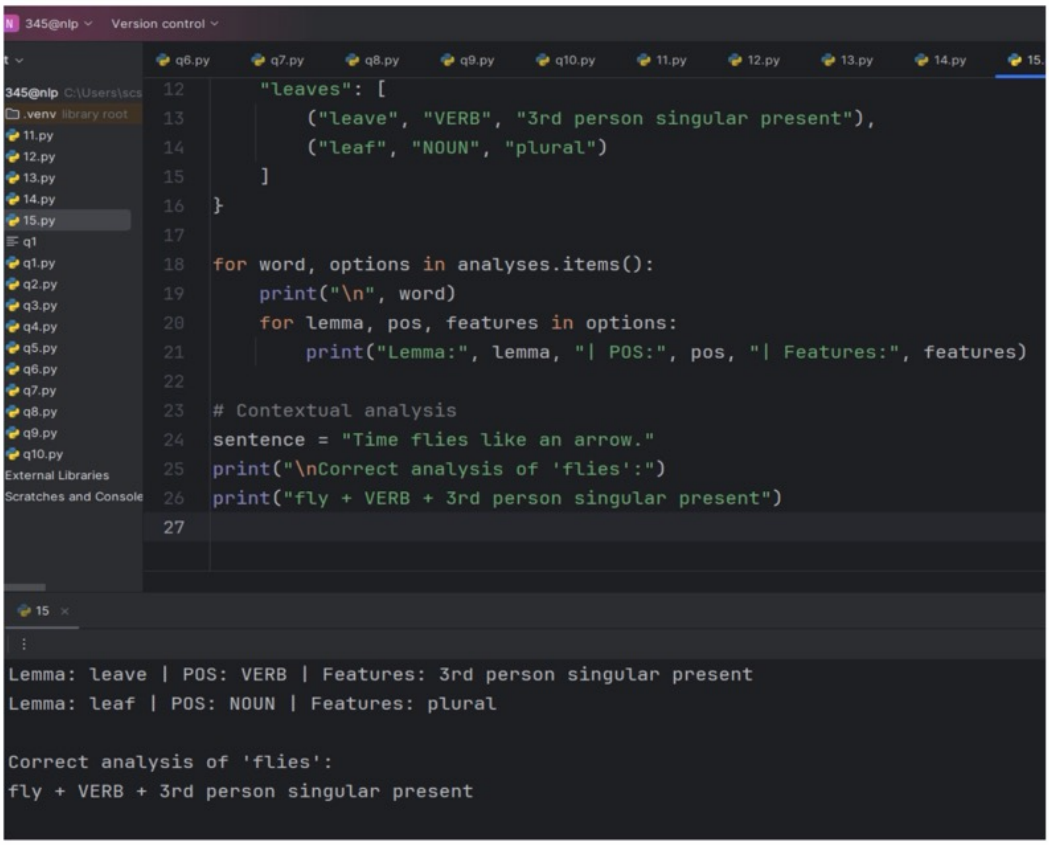
```
Word -> Stem
university -> univers
universe -> univers
alumnus -> alumnu
alumni -> alumni

Process finished with exit code 0
```

Q14. Agglutinative Turkish and Vocabulary Explosion

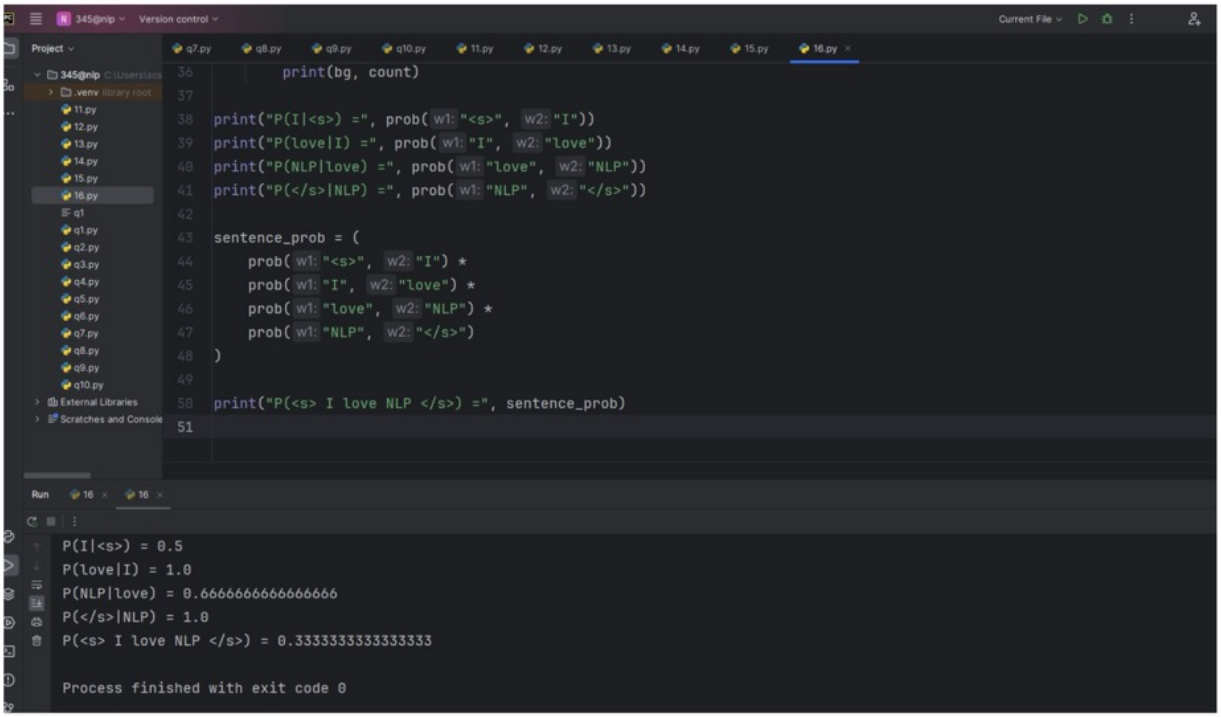


Q15. Ambiguous Analyses from a Morphological Analyzer

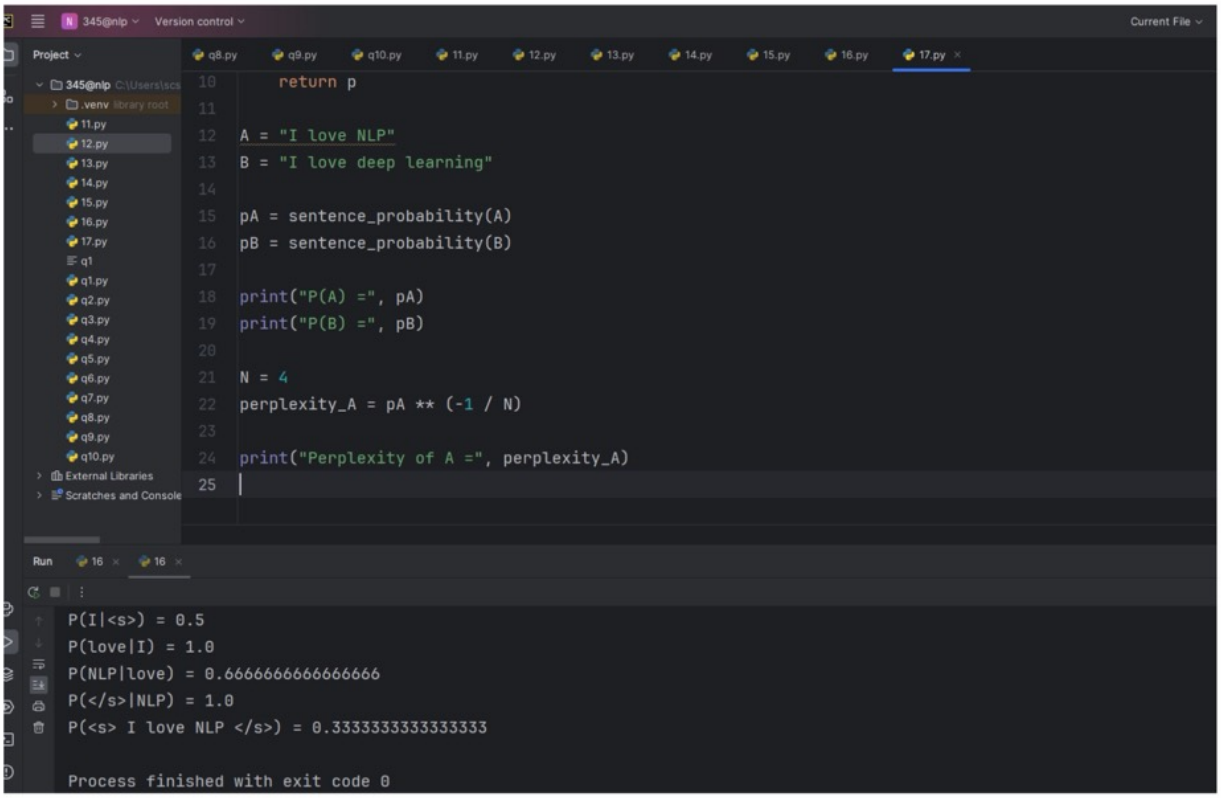


Unit 4: N-Grams

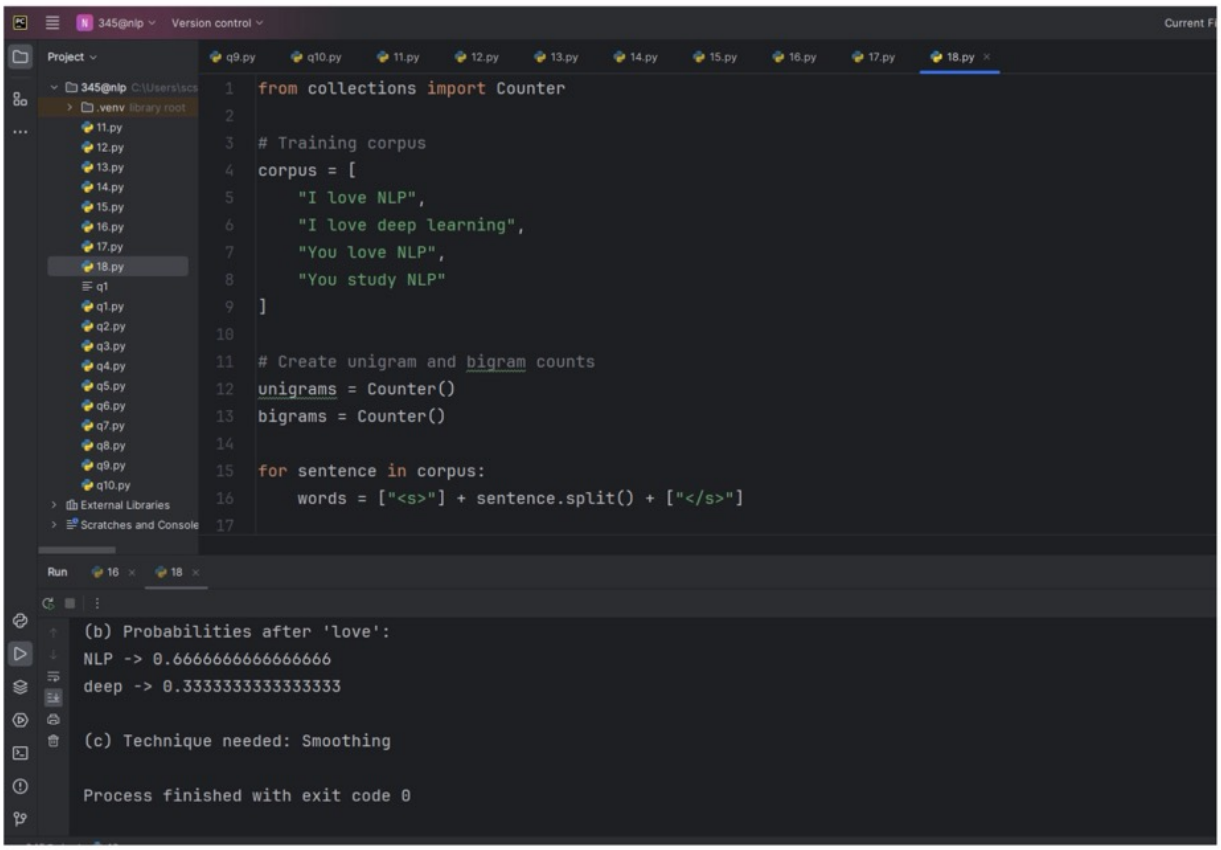
Q16. Building a Bigram Model



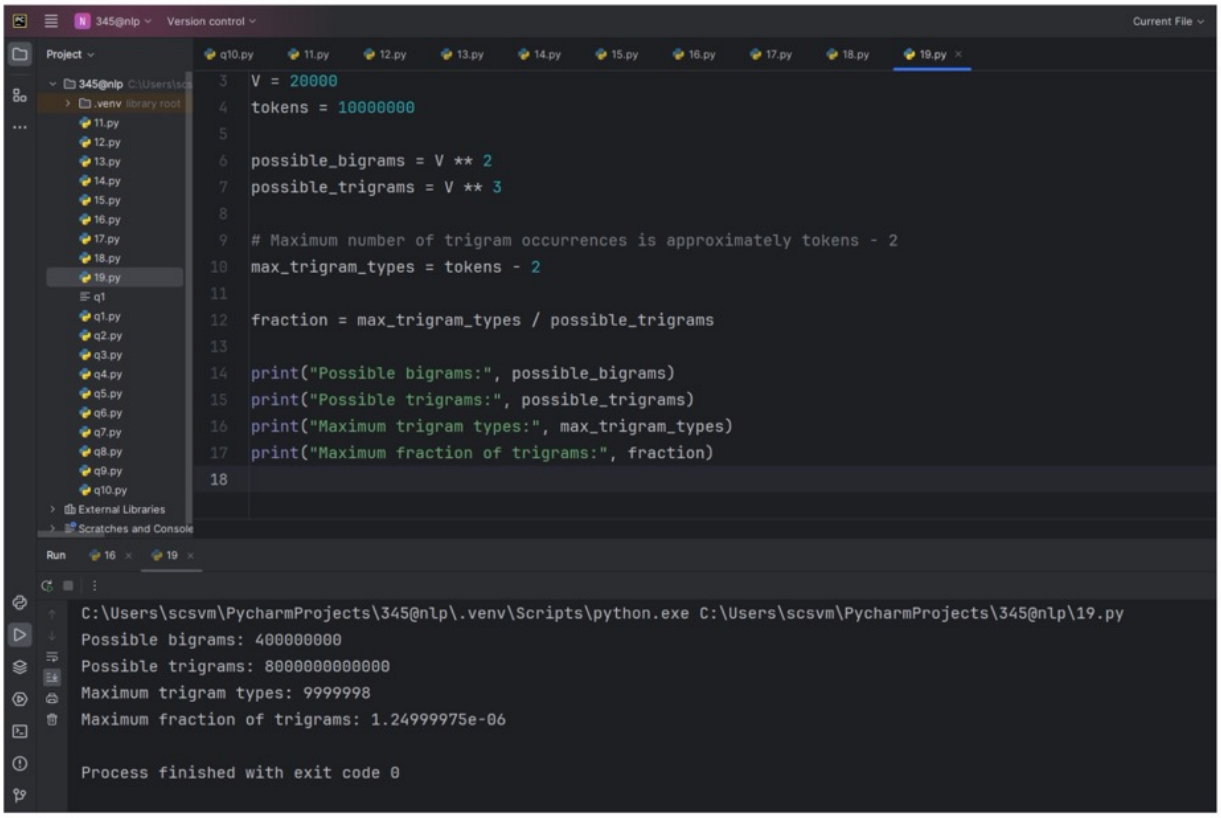
Q17. Comparing Two Candidate Sentences



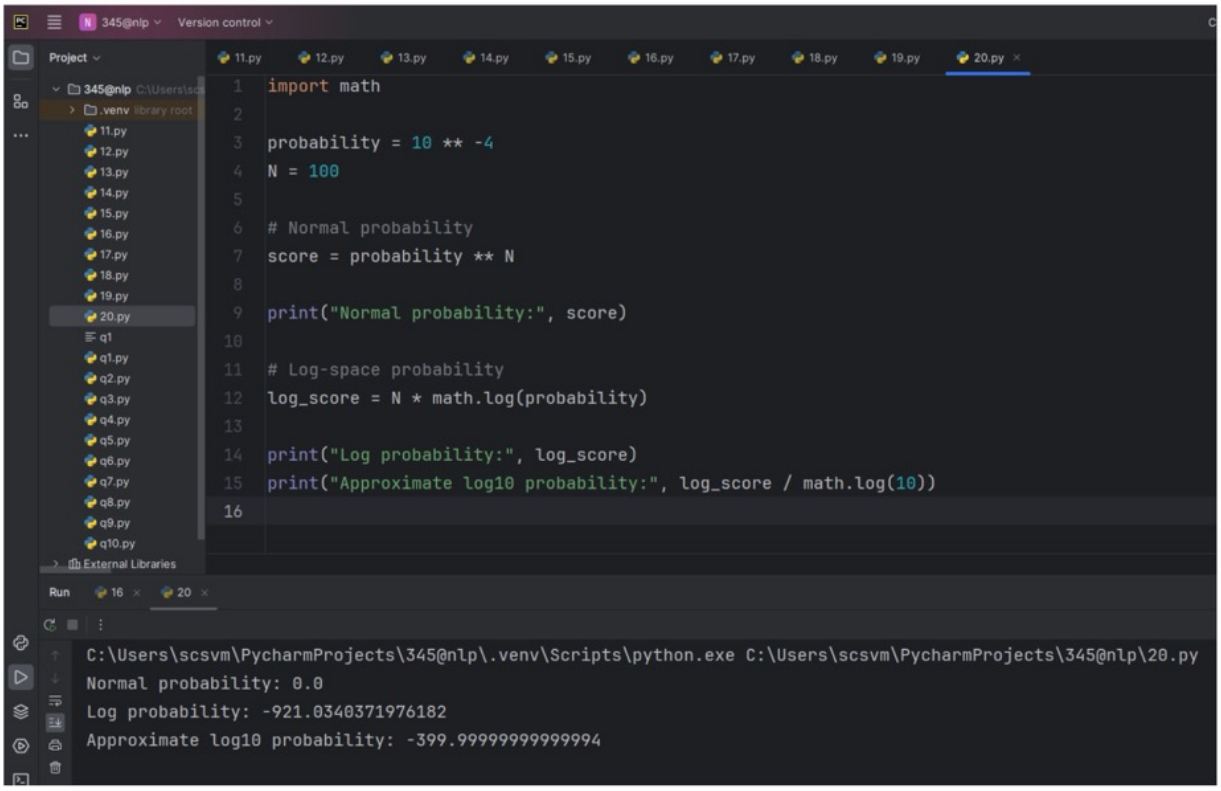
Q18. The Zero-Probability Problem



Q19. Model Size and Data Sparsity on a Phone

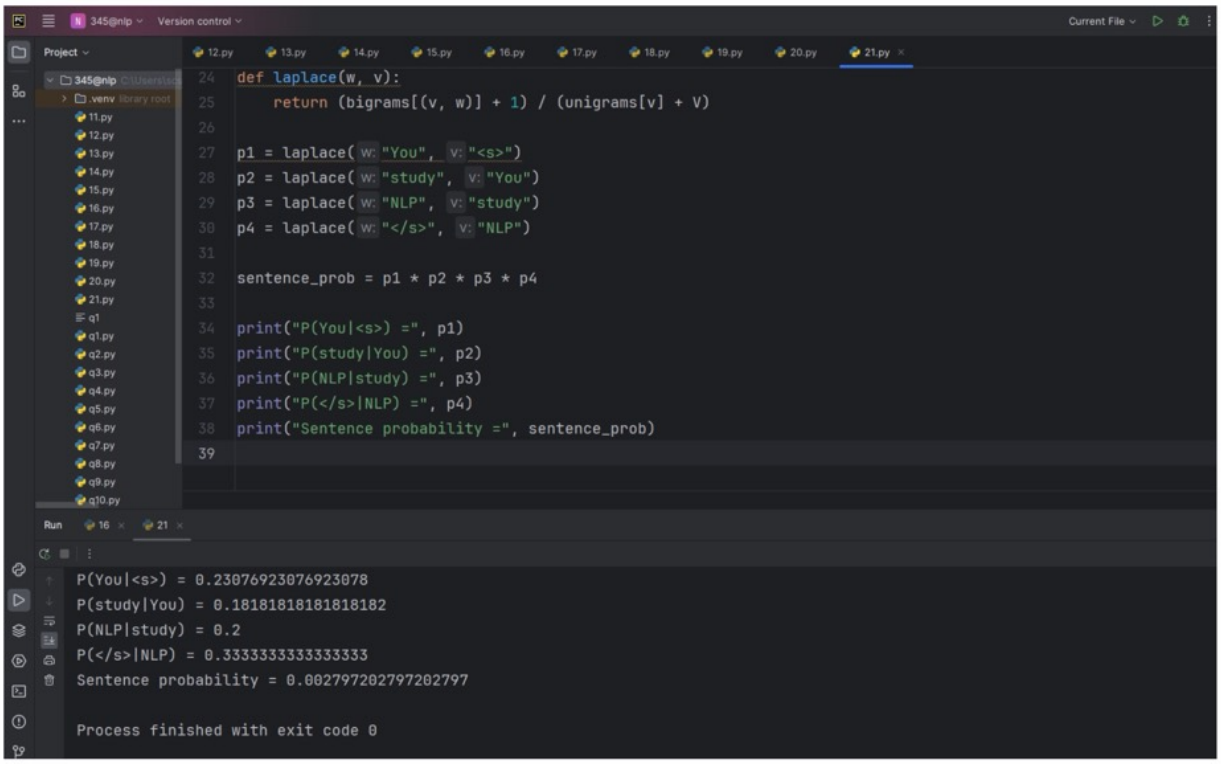


Q20. Underflow and Language Identification

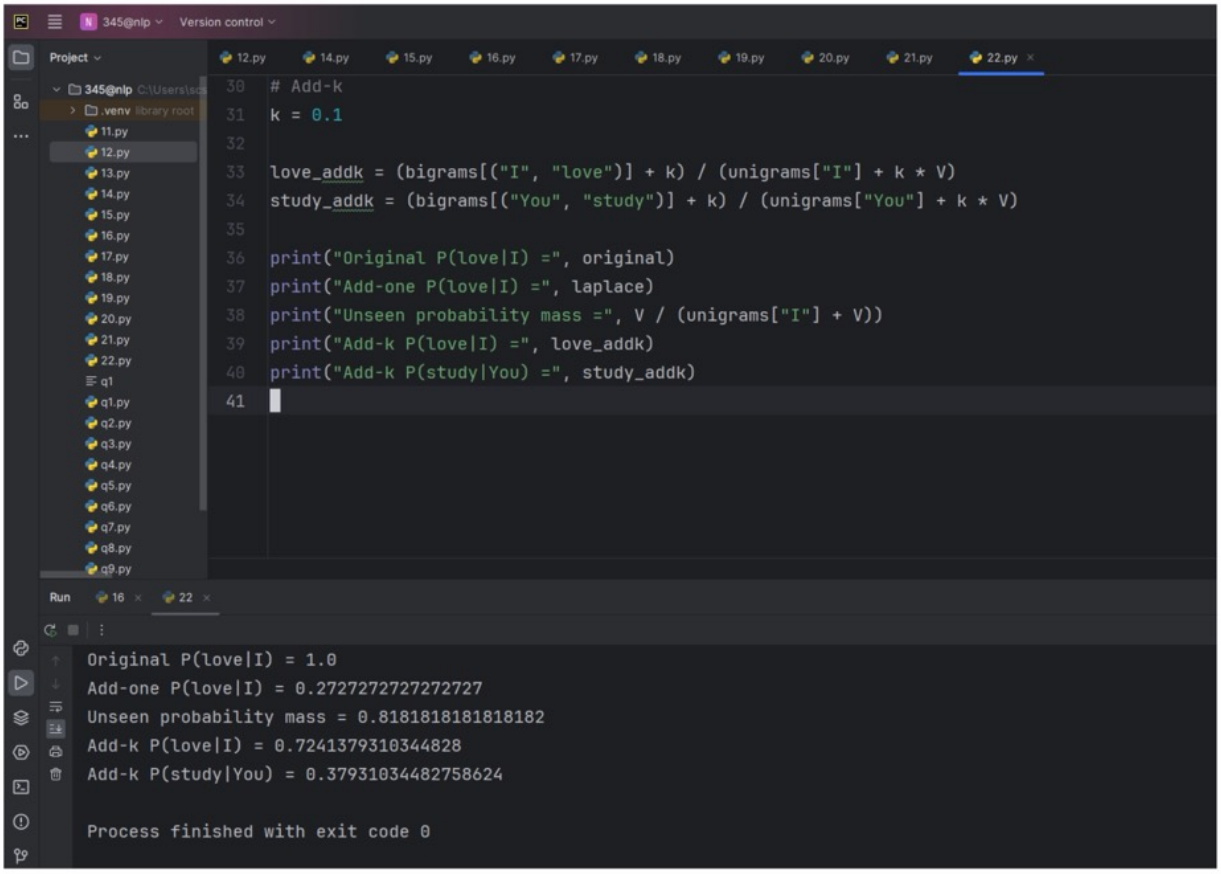


Unit 5: N-Gram Smoothing

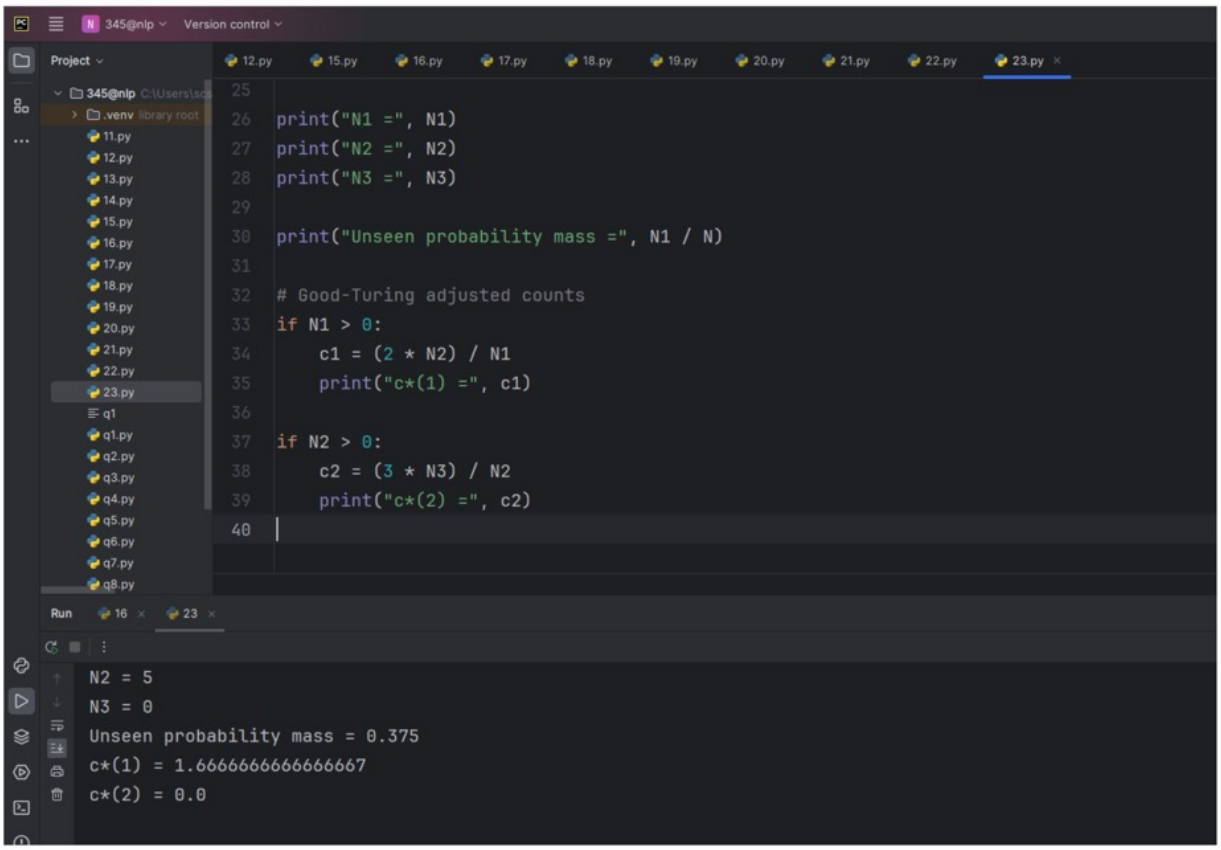
Q21. Laplace Smoothing of an Unseen Bigram



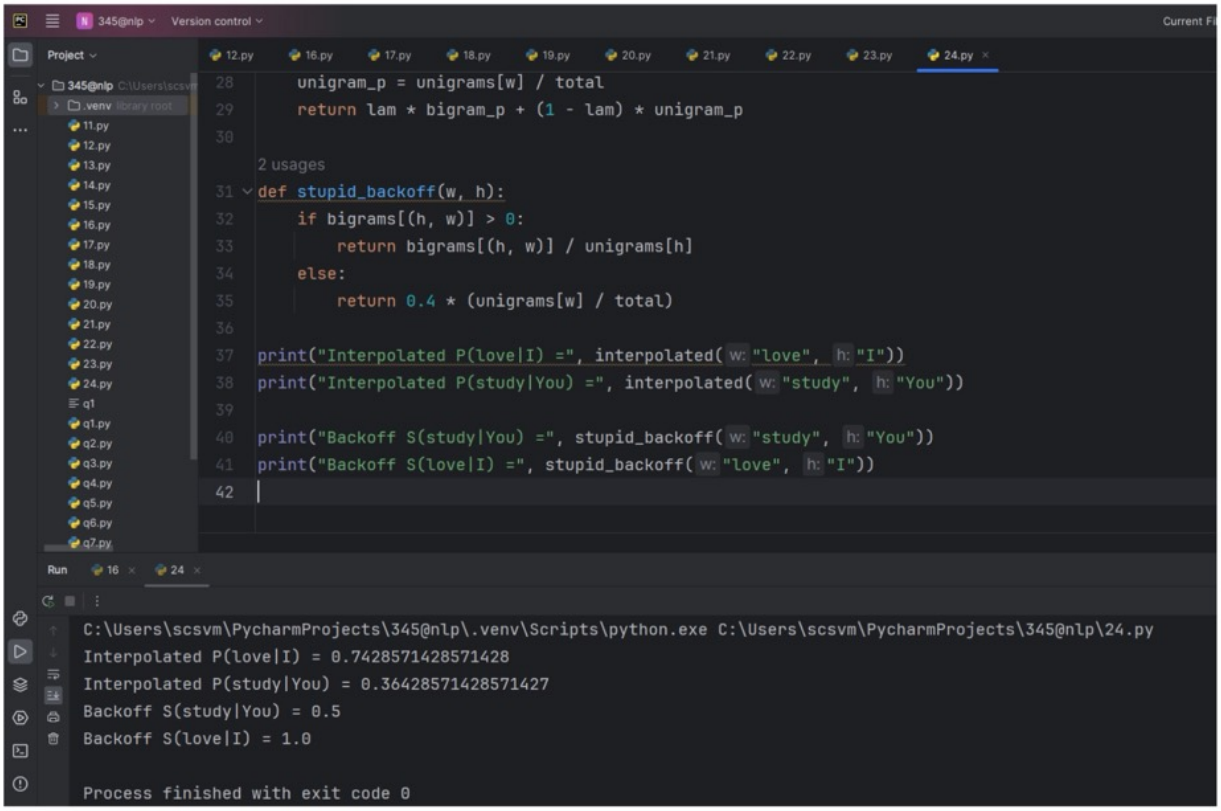
Q22. How Much Mass Does Laplace Steal?



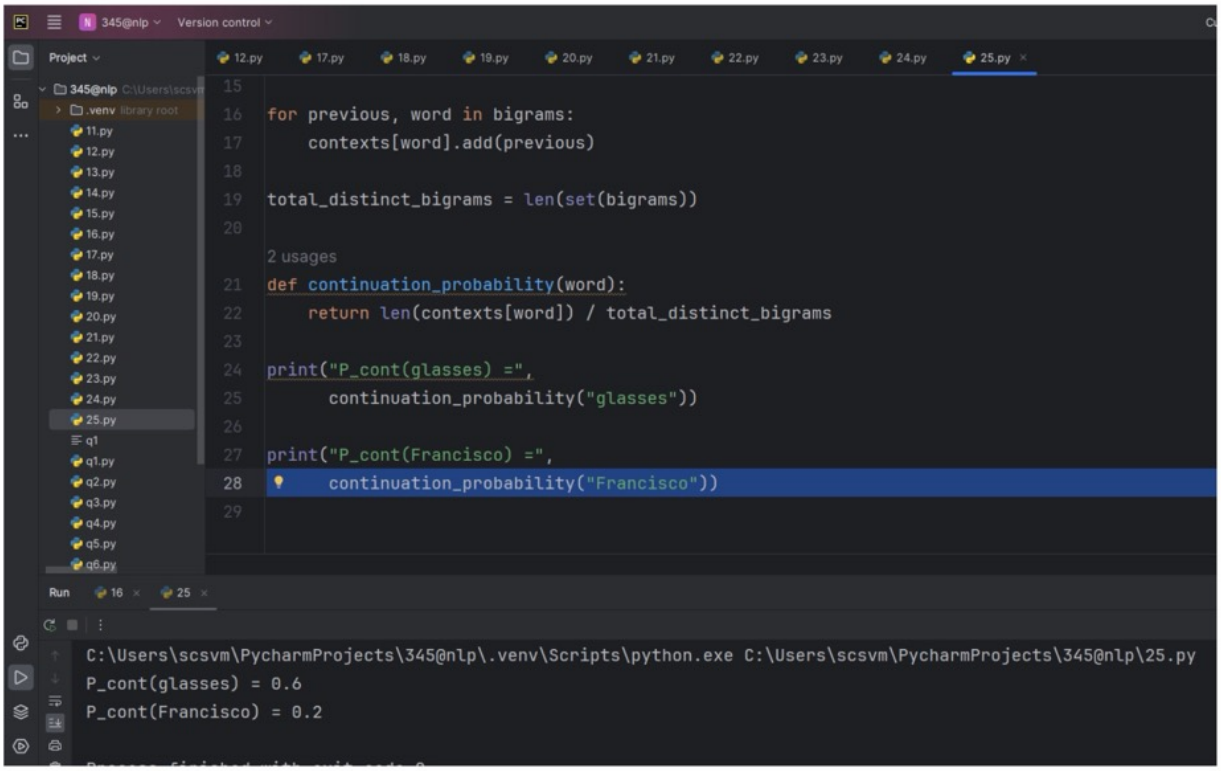
Q23. Good-Turing Discounting



Q24. Interpolation and Backoff

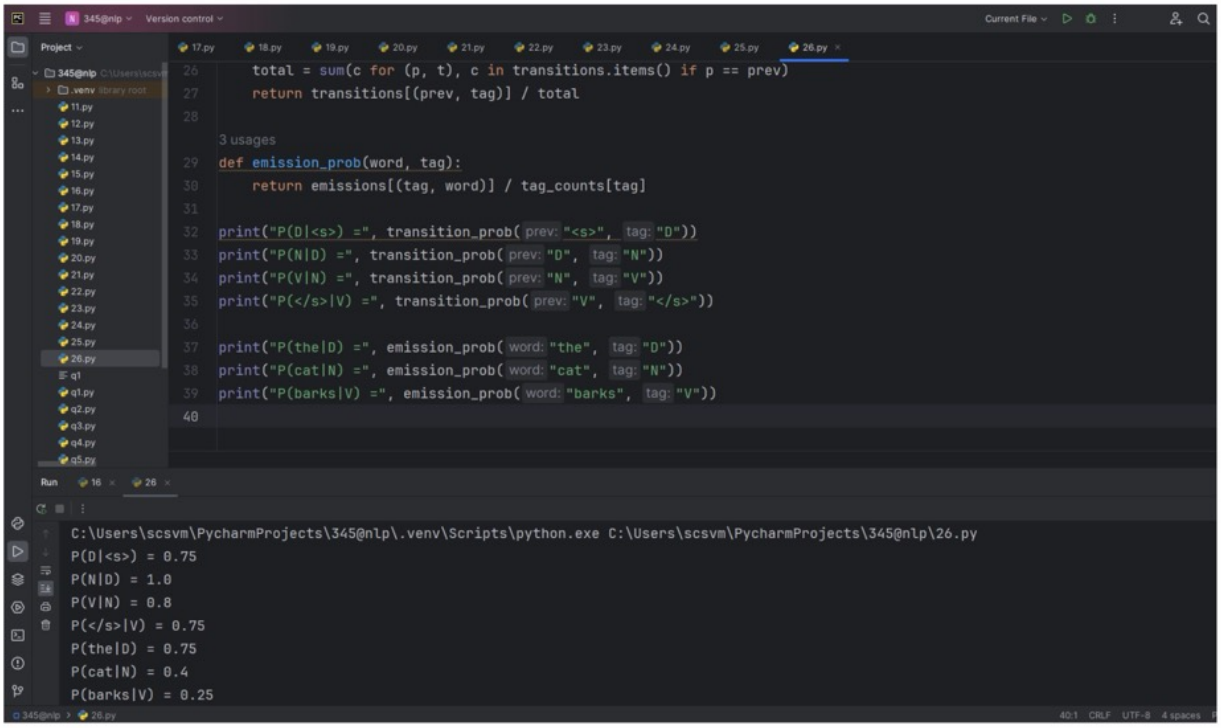


Q25. Why "Francisco" Should Not Follow "reading"

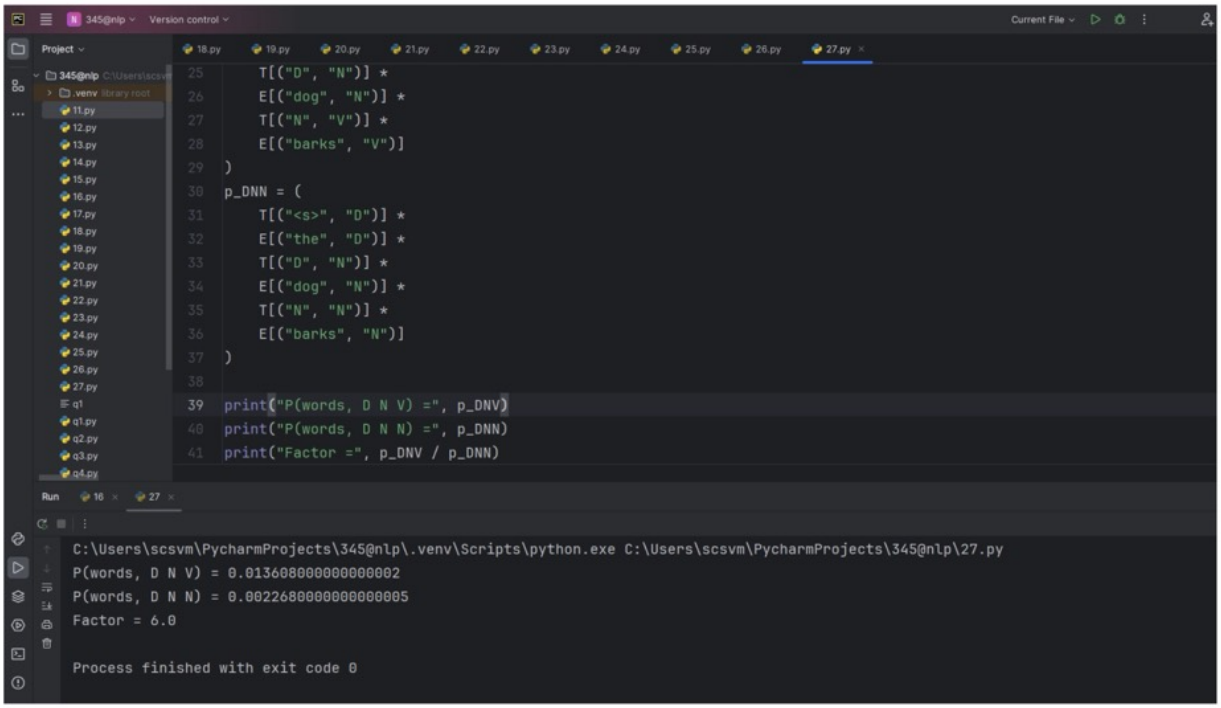


Unit 6: POS Tagging: Hidden Markov Model

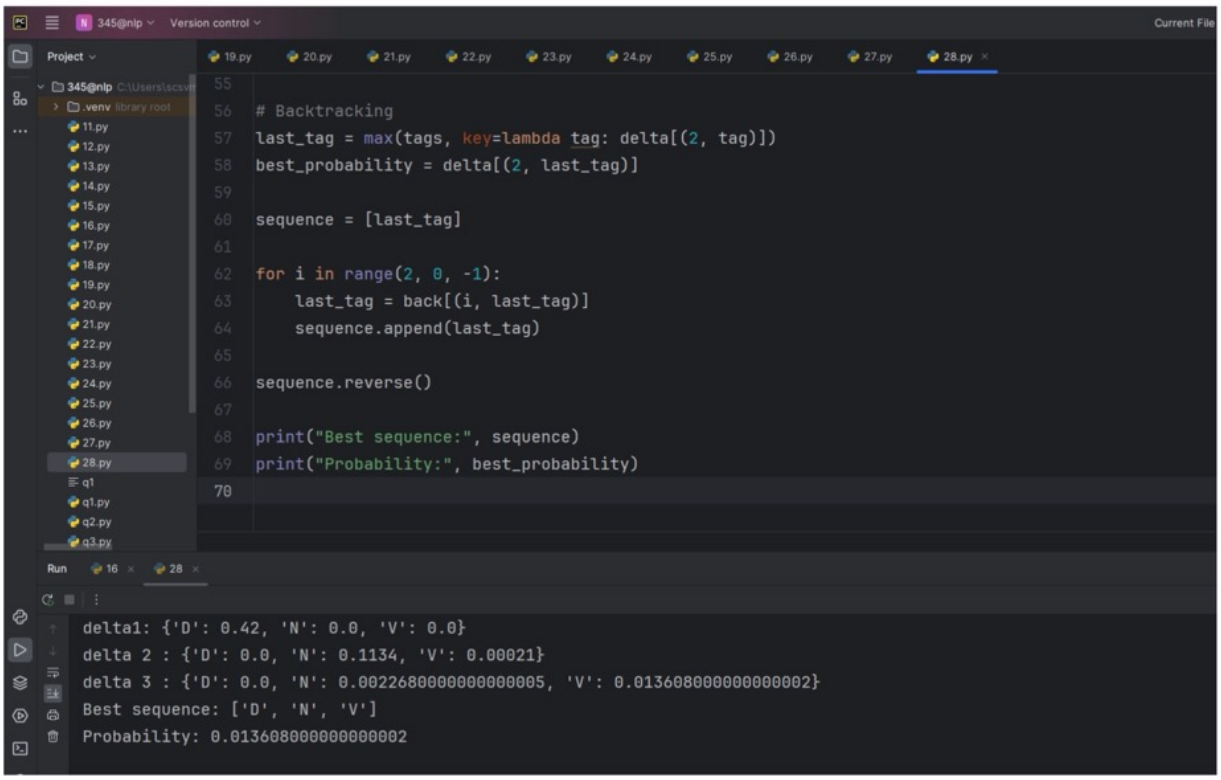
Q26. Training an HMM Tagger from a Tagged Corpus



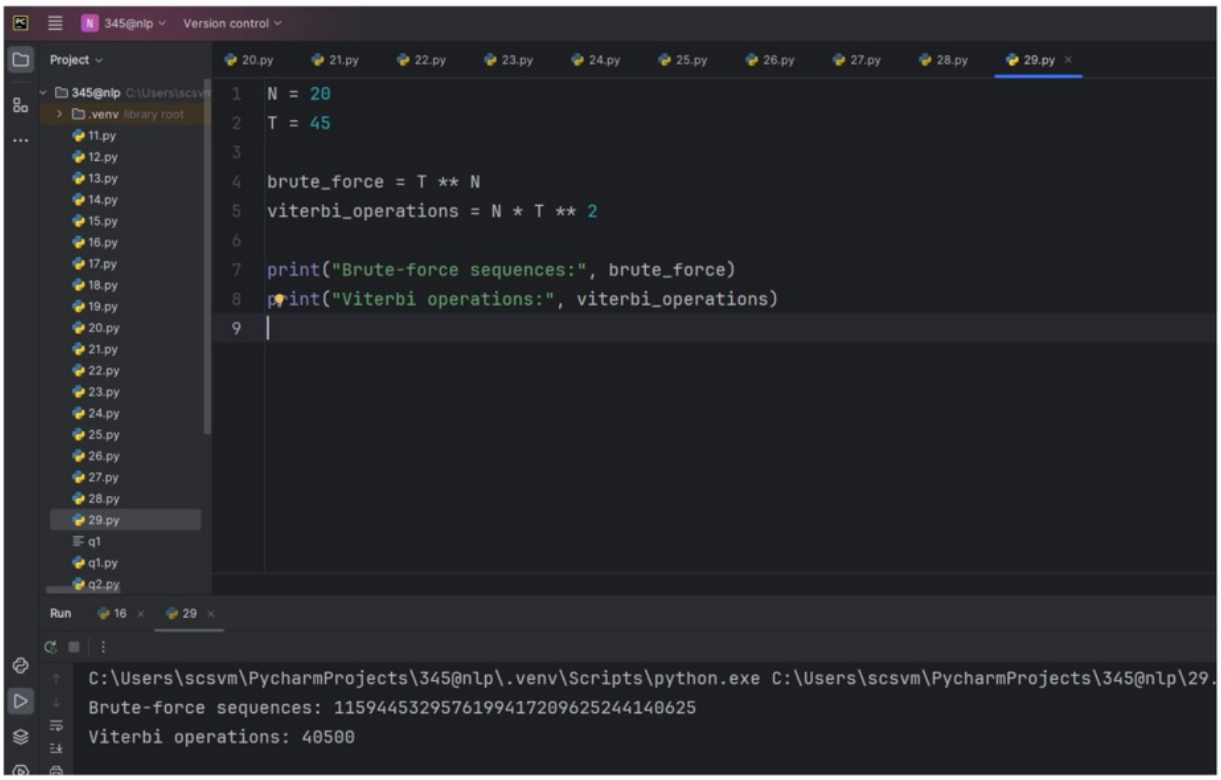
Q27. Joint Probability of Competing Tag Sequences



Q28. Running the Viterbi Algorithm



Q29. Why Viterbi Instead of Brute Force?



Q30. Unknown Words and Model Limitations

